

Linux 平台下 Tiny6410 裸机程序开发指南

版本：2013-12-1

(本手册正在不断更新中，建议您到网站下载最新版本)



Copyright@2013



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

版 权 声 明

本手册版权归属广州友善之臂计算机科技有限公司（以下简称“友善之臂”）所有， 并保留一切权力。非经友善之臂同意(书面形式)，任何单位及个人不得擅自摘录本手册部分或全部，违者我们将追究其法律责任。

敬告：

在售开发板的手册会经常更新，请在 <http://www.arm9home.net> 网站查看最近更新，并下载最新手册，不再另行通知。

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)



追 求 卓 越 创 造 精 品
TO BE BEST TO DO GREAT

广州友善之臂计算机科技有限公司

更新说明:

日期	说明
2013-12-1	本手册第一次发布，任何问题请反馈至capbily@163.com



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

第一章	简介	7
第一节	起源	7
第二节	关于开发环境	7
第三节	文档涉及的裸机程序	7
第二章	汇编点亮 LED	8
第一节	查阅原理图	8
第二节	程序相关讲解	9
第三节	编译代码和烧写运行	10
第四节	实验现象	12
第三章	设置栈和 C 语言点亮 LED	13
第一节	为什么调用 C 函数要设置栈	13
第二节	程序相关讲解	14
第三节	编译代码和烧写运行	15
第四节	实验现象	16
第四章	C 语言中调用汇编函数	17
第一节	为什么要在 C 语言中调用汇编函数	17
第二节	程序相关讲解	17
第三节	编译代码和烧写运行	18
第四节	实验现象	19
第五章	控制 icache	20
第一节	什么是 cache	20
第二节	程序相关讲解	20
第三节	编译代码和烧写运行	20
第四节	实验现象	21
第六章	查询方式检测按键	23
第一节	查看原理图	23
第二节	程序相关讲解	23
第三节	编译代码和烧写运行	24
第四节	实验现象	25
第七章	初始化时钟	26
第一节	S3C6410 时钟体系	26
第二节	程序相关讲解	27
第三节	编译代码和烧写运行	34
第四节	实验现象	35
第八章	串口设置之输入输出字符	36
第一节	S3C6410 UART 相关说明	36
第二节	程序相关讲解	37
第三节	编译代码和烧写运行	42
第四节	实验现象	43
第九章	重定位代码到 SRAM+4096	45

地址：广州市天河区龙口西路龙苑大厦A1栋1705

网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

第一节	两个不同的地址概念	45
第二节	程序相关讲解	45
第三节	编译代码和烧写运行	47
第四节	实验现象	48
第十章	重定位代码到 DRAM	49
第一节	关于 DRAM	49
第二节	程序相关讲解	50
第三节	编译代码和烧写运行	51
第四节	实验现象	52
第十一章	NAND Flash 控制器	53
第一节	关于 NAND Flash	53
第二节	程序相关讲解	53
第三节	编译代码和烧写运行	56
第四节	实验现象	57
第十二章	内存管理单元 MMU	58
第一节	关于 MMU	58
第二节	程序相关讲解	58
第三节	编译代码和烧写运行	60
第四节	实验现象	61
第十三章	移植 printf 和 scanf 功能	62
第一节	移植的途径	62
第二节	移植步骤	62
第三节	程序相关讲解	62
第四节	编译代码和烧写运行	64
第五节	实验现象	65
第十四章	控制蜂鸣器	67
第一节	查阅原理图	67
第二节	程序相关讲解	67
第三节	编译代码和烧写运行	68
第四节	实验现象	70
第十五章	S3C6410 中断体系	71
第一节	关于 S3C6410 的中断体系	71
第二节	程序相关讲解	71
第三节	编译代码和烧写运行	73
第四节	实验现象	74
第十六章	PWM 定时器	75
第一节	S3C6410 的 PWM 定时器	75
第二节	程序相关讲解	76
第三节	编译代码和烧写运行	78
第四节	实验现象	79



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

第十七章	看门狗定时和复位	80
第一节	S3C6410 的看门狗定时器	80
第二节	程序相关讲解	80
第三节	编译代码和烧写运行	82
第四节	实验现象	82
第十八章	RTC 读写时间	84
第一节	S3C6410 的 RTC	84
第二节	程序相关讲解	84
第三节	编译代码和烧写运行	86
第四节	实验现象	87
第十九章	LCD 绘图和打印字符	89
第一节	S3C6410 LCD 控制器	89
第二节	程序相关讲解	89
第三节	编译代码和烧写运行	98
第四节	实验现象	99
第二十章	测试 ADC 转换	101
第一节	S3C6410 的 ADC	101
第二节	程序相关讲解	102
第三节	编译代码和烧写运行	104
第四节	实验现象	105
第二十一章	增加命令功能	107
第一节	关于命令功能	107
第二节	程序详细讲解	107
第三节	编译代码和烧写运行	108
第四节	实验现象	109

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)



第一章 简介

第一节 起源

对于很多嵌入式开发者和爱好者，特别是初学者，如何从底层开始了解和学习 ARM，绝非是一件容易的事！为此，友善之臂的工程师，花了很多时间和心血，基于 Tiny6410 开发板编写了这份教程，以帮助广大嵌入式爱好者更深入了解 S3C6410 的详细启动过程，同时让更多初学者能从裸机程序开始学习 S3C6410。鉴于每个人的认知水平不同，以及我们平时的开发任务比较紧张，我们并不提供关于该教程的任何技术支持。如果你对本教程的内容有任何疑问，可以到 arm9 之家论坛(<http://www.arm9home.net>) 反馈，并和其他网友交流讨论。需要说明的是，本教程也适用于友善之臂出品的 mini6410 等其他型号基于 S3C6410 的开发板平台。友善之臂将对本教程作不定期的维护和补充，请时常留意论坛的更新信息，不再另行通知。友善之臂(<http://www.arm9.net>) 保留本教程的一切解释权。

第二节 关于开发环境

学习前提：学习过简单的 C 语言和 ARM 汇编语言

开发平台：Windows XP + 虚拟机 fedora15，使用 eclipse 编写代码，代码均使用 utf8 进行编写。

交叉编译器：arm-linux-gcc-4.5.1

注：交叉编译器的安装方法可参考开发板相关的用户手册。

配套开发板：所有程序均在 Tiny6410 上成功测试运行过，对于友善之臂其他 S3C6410 开发板，大部分代码仍然适用。

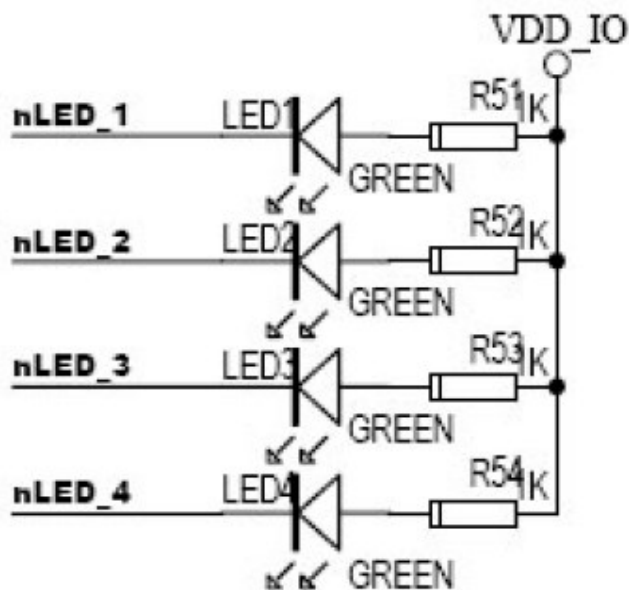
第三节 文档涉及的裸机程序

首先会从最基础的点亮 LED 讲起，先采用 DRAM 加载的方式，再采用 NAND 启动加载的方式，逐步讲解 S3C6410 的启动过程以及如何重定位代码，让用户了解 S3C6410 上电后运行的每一个步骤。用户可以通过查看书签可以了解本文档所涉及的硬件模块，本文档一共涉及 25 个裸机程序，并且以后将不断丰富出更多的裸机程序，也请众网友发挥各自的才智，在本文档的基础上写出更多简单实用的裸机程序并反馈给我们。

第二章 汇编点亮 LED

第一节 查阅原理图

Tiny6410 板上提供了 4 个可编程用户 LED，原理图如下：



LED 原理图

在原理图中搜索引脚“nLED_1”，可得：

ATA12/GPK12	N25	KP_ROW4	
ATA11/GPK11	P18	KP_ROW3	
ATA10/GPK10	P23	KP_ROW2	
ATA9/GPK9	P19	KP_ROW1	
ATA8/GPK8	P25	KP_ROW0	
ATA7/GPK7	R25	nLED_4	
ATA6/GPK6	R24	nLED_3	
ATA5/GPK5	R22	nLED_2	
ATA4/GPK4	R23	nLED_1	
ATA3/GPK3	T23	GPK3	
ATA2/GPK2	T24	GPK2	(NC)
ATA1/GPK1	T22	GPK1	
ATA0/GPK0	T25	GPK0	

LED 引脚图

可见，LED1, 2, 3, 4 分别使用的 CPU 端口资源为 GPK_4, 5, 6, 7。

第二节 程序相关讲解

完整代码见目录 1.led_s。(注：为方便用户阅读，本教程涉及的所有代码本身均含相当丰富的注释)

1. start.S

由原理图可知，点亮 Tiny6410 的 4 个 LED 需如下 2 个步骤：

把外设的基地址告诉 CPU；对于 6410 来说，内存的地址范围为 0x00000000~0x60000000，外设的地址范围为 0x70000000~0x7ffffff。

Table 2-1 Address map.

Start Address	End Address	Int. ROM	Stepping Stone (NAND Ctrl.)	SROM Ctrl.	One NAND Ctrl. 0	One NAND Ctrl. 1	DRAM Ctrl 1
0x00000000	0x07FFFFFF	O ¹	-	O ¹	O ¹	-	-
0x08000000	0x0BFFFFFF	O	-	-	-	-	-
0x0C000000	0x0FFFFFFF	-	O	-	-	-	-
0x10000000	0x17FFFFFF	-	-	O	-	-	-
0x18000000	0x1FFFFFFF	-	-	O	-	-	-
0x20000000	0x27FFFFFF	-	-	O ²	O ²	-	-
0x28000000	0x2FFFFFFF	-	-	O ²	-	O ²	-
0x30000000	0x37FFFFFF	-	-	O	-	-	-
0x38000000	0x3FFFFFFF	-	-	O	-	-	-
0x40000000	0x47FFFFFF	-	-	-	-	-	-
0x48000000	0x4FFFFFFF	-	-	-	-	-	-
0x50000000	0x5FFFFFFF	-	-	-	-	-	O
0x60000000	0x6FFFFFFF	-	-	-	-	-	O

关闭看门狗，防止程序不断重启；

设置寄存器 GPKCON0，使 GPK_4/5/6/7 四个引脚为输出功能；

往寄存器 GPKDAT 写 0，使 GPK_4/5/6/7 四个引脚输出低电平，4 个 LED 会亮；相反，

往寄存器 GPKDAT 写 1，使 GPK_4/5/6/7 四个引脚输出高电平，4 个 LED 会灭；

以上两个步骤即为 start.S 中的核心内容，start.S 里面涉及的汇编指令请自行学习 GNU 汇编指令集，这里不再进行赘述。

2. Makefile

```
led.bin: start.o
    arm-linux-ld -Ttext 0x50000000 -o led.elf $^
    arm-linux-objcopy -O binary led.elf led.bin
    arm-linux-objdump -D led.elf > led_elf.dis
```



```
%o : %.S
arm-linux-gcc -o $@ $< -c
%.o : %.c
arm-linux-gcc -o $@ $< -c
clean:
rm *.o *.elf *.bin *.dis -rf
```

关于 Makefile 网上的相关资料相当多, 这里推荐《跟我一起写 makefile》(光盘中有), 以方便用户后续的学习和查阅, 下面我们只是进行粗略地讲解。当用户在 Makefile 所在目录下执行 make 命令时, 系统会进行如下操作:

执行 arm-linux-gcc -o \$@ \$< -c 命令将当前目录下存在的汇编文件和 C 文件编译成 .o 文件;
执行 arm-linux-ld -Ttext 0x50000000 -o led.elf \$^ 将所有 .o 文件链接成 elf 文件, -Ttext 0x50000000 表示程序的运行地址是 0x50000000, 即程序只有位于该地址上才能正常运行;
执行 arm-linux-objcopy -O binary led.elf led.bin 将 elf 文件抽取为可在开发板上运行的 bin 文件;
执行 arm-linux-objdump -D led.elf > led_elf.dis 将 elf 文件反汇编后保存在 dis 文件中, 调试程序时可能会用到;

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

```
# cd 1.led_s
# make
```

执行 make 后会生成 led.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上。MiniTools 提供了两种烧写裸机程序的方式: 一种是直接下载到内存 DRAM, 另外一种下载到 NAND Flash。注意: 请先在 SD 卡中烧写带有 USB 下载功能的 Superboot。

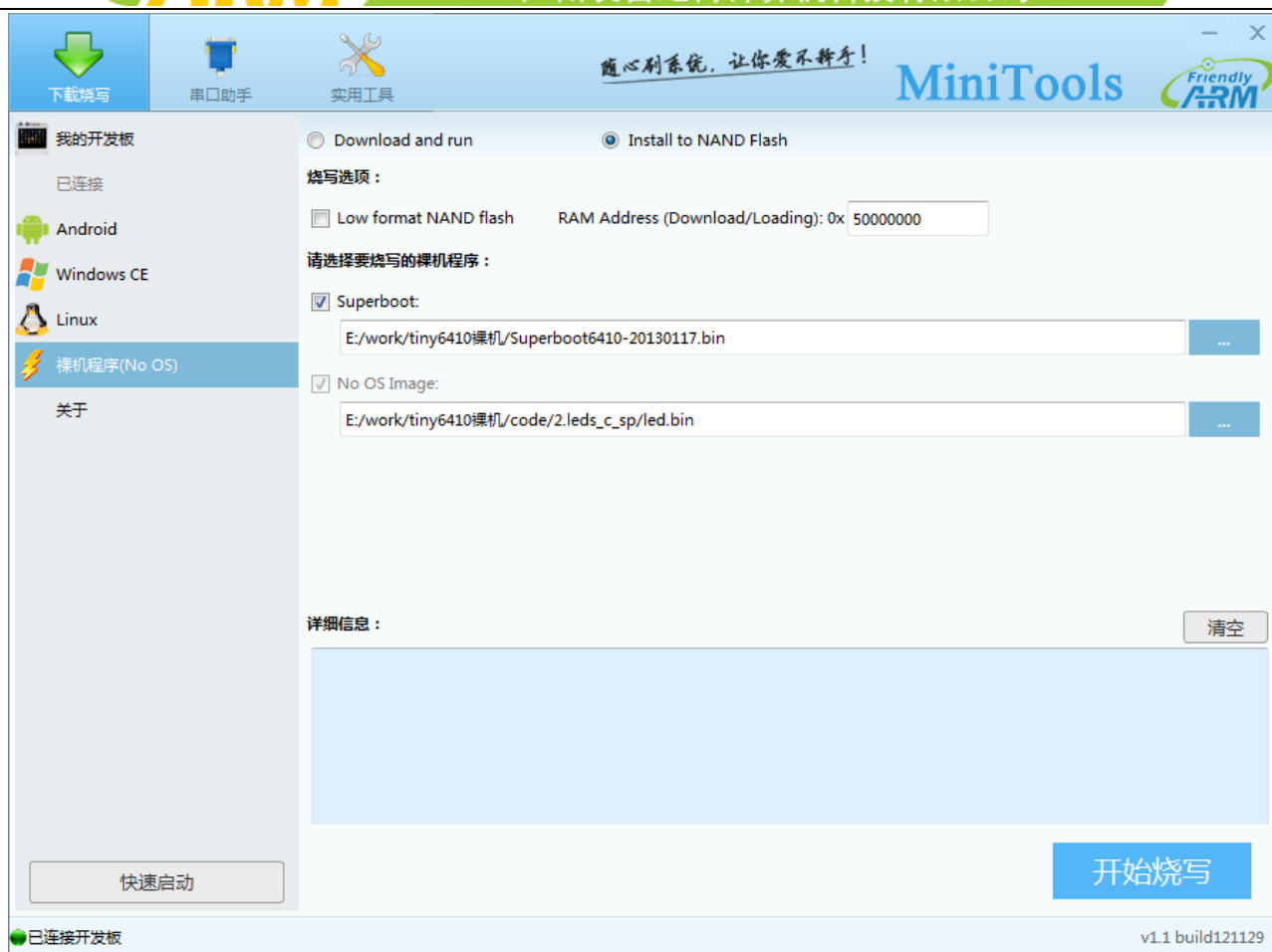
方式一 下载到 DRAM, 其设置方式如下:



先选中上方的” Download and run “，设置好下载地址”RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击“下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击“下载运行”就可以马上看到开发板上裸机程序的运行效果了。

注意：不同的 CPU 会有不同的内存起始地址，在此为 0x50000000

方式二 下载到 NANDFlash，其设置方式如下：



先选中“Install to NAND Flash”，设置好加载地址“RAM Address(Download/Loading)”，选择使用 Superboot 加载裸机程序，并选择要加载的裸机程序，点击“开始烧写”；MiniTools 首先把 Superboot 和裸机程序都烧写到 NAND Flash 中，以后只要选择 NAND 启动时，NAND Flash 中的 Superboot 都会将 NAND Flash 中的裸机程序加载到 DRAM 中运行了。

上面两种方式都可以烧写运行我们的裸机程序，由于方式一较为便捷，并且不会损坏 NAND Flash 中的原有数据，本文档所涉及的所有的程序都将统一采用这种方式进行烧写和运行。

第四节 实验现象

可以看到开发板上的 4 个 LED 不停地闪烁。

第三章 设置栈和 C 语言点亮 LED

第一节 为什么调用 C 函数要设置栈

栈有三个作用，包括：保存现场；传递参数：汇编代码调用 C 函数时，需传递参数；保存临时变量：包括函数的非静态局部变量以及编译器自动生成的其他临时变量；

1) 保存现场

现场，意思就相当于案发现场，总有一些现场的情况，要记录下来的，否则被别人破坏掉之后，你就无法恢复现场了。而此处说的现场，就是指 CPU 运行的时候，用到了一些寄存器，比如 r0, r1 等等，对于这些寄存器的值，如果你不保存而直接跳转到子函数中去执行，那么很可能就被其破坏了，因为其函数执行也要用到这些寄存器。因此，在函数调用之前，应该将这些寄存器等现场，暂时保持起来(入栈 push)，等调用函数执行完毕返回后(出栈 pop)，再恢复现场。这样 CPU 就可以正确的继续执行了。

保存寄存器的值，一般用的是 push 指令，将对应的某些寄存器的值，一个个放到栈中，把对应的值压入到栈里面，即所谓的压栈。然后待被调用的子函数执行完毕的时候，再调用 pop，把栈中的一个一个的值，赋值给对应的那些你刚开始压栈时用到的寄存器，把对应的值从栈中弹出去，即所谓的出栈。

其中保存的寄存器中，也包括 lr 的值（因为用 bl 指令进行跳转的话，那么之前的 PC 的值是存在 lr 中的），然后在子程序执行完毕的时候，再把栈中的 lr 的值 pop 出来，赋值给 PC，这样就实现了子函数的正确的返回。

2) 传递参数

C 语言进行函数调用的时候，常常会传递给被调用的函数一些参数，对于这些 C 语言级别的参数，被编译器翻译成汇编语言的时候，就要找个地方存放一下，并且让被调用的函数能够访问，否则就没发实现传递参数了。对于找个地方放一下，分两种情况。一种情况是，本身传递的参数不多于 4 个，就可以通过寄存器 r0~r3 传送参数。因为在前面的保存现场的动作中，已经保存好了对

应的寄存器的值，那么此时，这些寄存器就是空闲的，可以供我们使用的了，那就可以放参数。

另一种情况是，参数多于 4 个时，寄存器不够用，就得用栈了。

3) 临时变量保存在栈中

包括函数的非静态局部变量以及编译器自动生成的其他临时变量。

第二节 程序相关讲解

完整代码见目录 2.led_c_sp。

1. start.S

本章我们将使用 C 函数来实现点灯和延时的功能。在代码 2.led_c_sp 中，start.S 的作用如下：

- 1) 把外设的基地址告诉 CPU；
- 2) 关闭看门狗；
- 3) 设置栈，其实就是设置 SP 寄存器，让其指向一块可用的内存。SD 启动时，S3C6410 的内部 8K 的 SRAM 被映射到 0X0C000000，而 ARM 默认的栈是递减的，所以可以让 SP 指向 0X0C002000；
- 4) 调用 c 函数 main()，实现 LED 闪烁；

2. main.c

main.c 中编写了两个 c 函数，main() 实现 LED 闪烁，delay() 实现延时的功能，代码如下：

```
void delay()
{
    volatile int i = 0x10000;
    while (i--);
}

int main()
{
    int i = 0x10;
    volatile unsigned long *gpkcon0 = (volatile unsigned long *)0x7F008800;
    volatile unsigned long *gpkdat = (volatile unsigned long *)0x7F008808;
    *gpkcon0 = 0x11110000;
    while (1)
    {
        *gpkdat = i;
        i++;
        if (i == 0x100 )
```




```
        i = 0x10;  
        delay();  
    }  
  
    return 0;  
}
```

相比起汇编，使用 C 语言来操作 LED 显得简洁容易了许多，后面的代码我们尽可能地使用 C 语言编写。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

```
# cd 2.led_c_sp
```

```
# make
```

执行 make 后会生成 led.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址”RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击“下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上一点击“下载运行”就可以马上看到开发板上裸机程序的运行效果了。

第四节 实验现象

可以看到 4 个 LED 轮流地被点亮。设置栈以后，我们的程序就能调用 C 函数，这样将大大提升我们编写代码的速度，后面的代码我们将尽可能使用 C 语言编写。

第四章 C 语言中调用汇编函数

第一节 为什么要在 C 语言中调用汇编函数

- 1) 汇编执行的代码效率更高;
- 2) 某些操作使用汇编编写代码更方便, 如对协处理器的操作;

第二节 程序相关讲解

完整代码见目录 3. led_c_call_s。

1. start.S

本章我们将使用 C 函数来实现点灯和使用汇编实现延时的功能。在代码 3. led_c_call_s 中, start.S 的作用和上一章一样, 但是多了一个汇编编写的延时函数, 函数名叫 delay(), 代码如下:

```
delay:
delay_loop:
    cmp r0, #02
    sub r0, r0, #1
    bne delay_loop
    mov pc, lr
```

2. main.c

main.c 中的 main() 函数实现了流水灯的功能, 其中延时部分的代码调用的是 start.S 中的 delay() 函数, 参数通过 r0 传递, 原理和汇编调用 c 函数是一样的, 具体代码如下:

```
void delay(int count);           // 函数声明
int main()                       // 流水灯
{
    int i = 0x10;                 // 配置引脚
    volatile unsigned long *gpkcon0 = (volatile unsigned long *)0x7F008800;
    volatile unsigned long *gpkdat = (volatile unsigned long *)0x7F008808;

    *gpkcon0 = 0x11110000;
    while (1)
    {
        *gpkdat = i;
        i++;
        if (i == 0x100 )
```

```
        i = 0x10;  
        delay(0x10000);  
    }  
    return 0;  
}
```

C 语言里调用汇编函数有两个需要注意的地方：

- 1) 汇编文件里需用“.global 函数名”导出函数；
- 2) C 文件里调用汇编函数前需要先声明函数，如 void delay(int count);

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

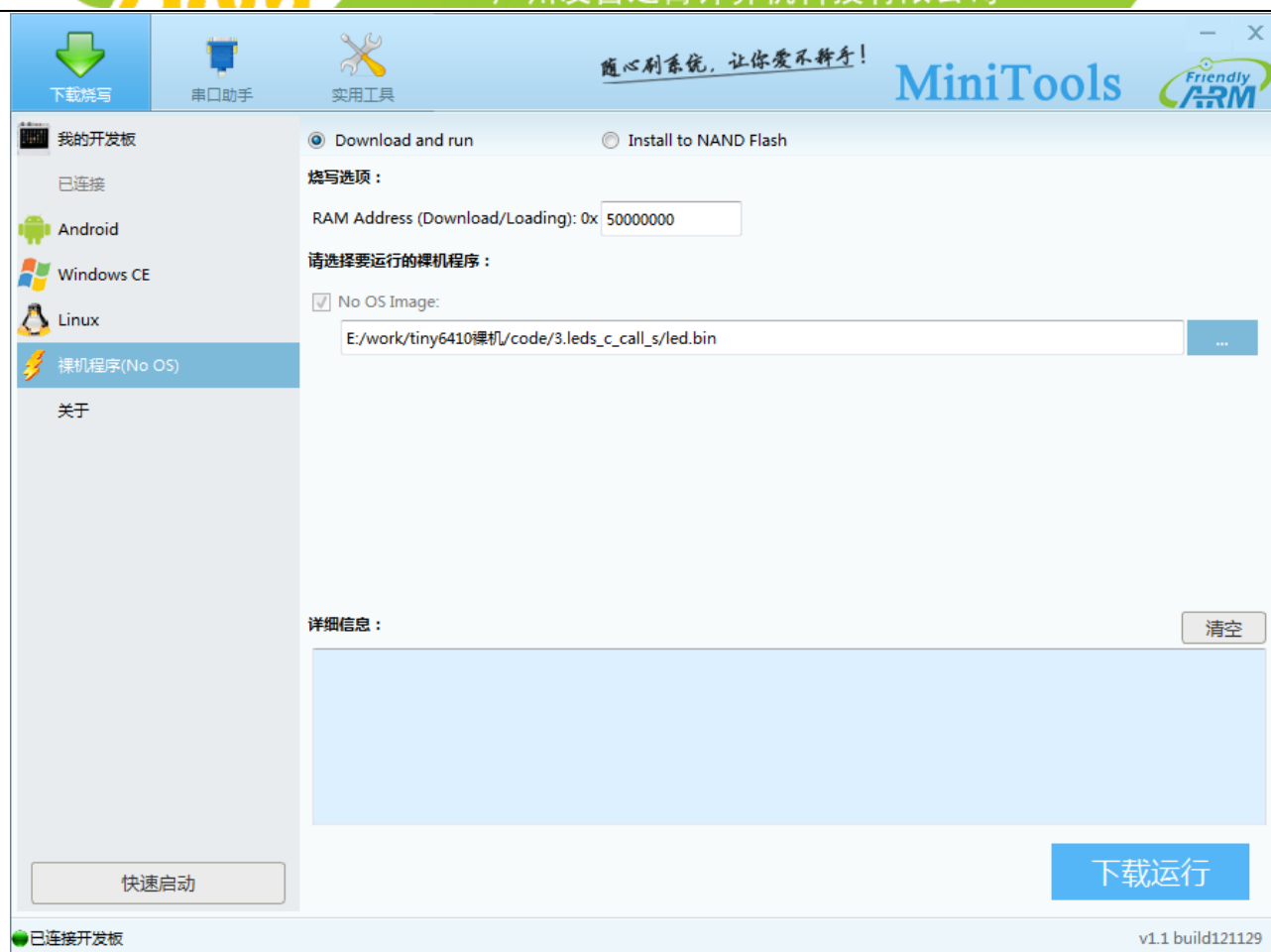
```
# cd 3.leds_c_call_s
```

```
# make
```

执行 make 后会生成 led.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址”RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击“下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击“下载运行”就可以马上看到开发板上裸机程序的运行效果了。

第四节 实验现象

代码的运行效果和上一章一样，都是流水灯，但是本章代码的 LED 闪烁明显比上一章的 LED 闪烁要快得多，这说明用汇编编写的代码执行起来效果更高。

第五章 控制 icache

第一节 什么是 cache

基于程序访问的局限性，在主存和 CPU 通用寄存器之前设置了一类高速的、容量较小的存储器，把正在执行的指令地址附件的一部分指令或数据从主存调入这类存储器，供 CPU 在一段时间内使用，这对提高程序的运行速度有很大的作用。这类介于主存和 CPU 之间的高速小容量存储器称作高速 cache。

比较常见的 cache 包括 icache 和 dcache。icach 的使用比较简单，系统刚上电时，icach 中的内容是无效的，并且 icache 的功能是关闭的，往 CP15 协处理器中的寄存器 1 的 bit[12] 写 1 可以启动 icache，写 0 可以停止 icache。icach 关闭时，CPU 每次取指都要读主存，性能非常低。因为 icache 可随时启动，越早开 icache 越好。

与 icache 相似，系统刚上电时，dcache 中的内容是无效的，并且 dcache 的功能是关闭的，往 CP15 协处理器中的寄存器 1 的 bit[2] 写 1 可以启动 dcache，写 0 可以停止 dcache。因为 dcache 必须在启动 mmu 后才能被启动，而对于裸机而言，没必要开 mmu，所以本教程的程序将不会启动 dcache。

第二节 程序相关讲解

完整代码见目录 4. led_c_icache。相比代码 2. leds_c_sp，代码 4. leds_c_icache 与它的唯一区别在于在 start.S 中打开了 icaches。

1. start.S

```
#ifdef CONFIG_SYS_ICACHE_OFF
bic r0, r0, #0x00001000           // 关闭 icache
#else
orr r0, r0, #0x00001000           // 打开 icache
#endif
mcr p15, 0, r0, c1, c0, 0
```

CONFIG_SYS_ICACHE_OFF 为 0 时启动 icache，为 1 时关闭 icache。至于协处理器的相关指令，需查阅 s3c2410 的芯片手册或者《arm 体系结构与编程》一书。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

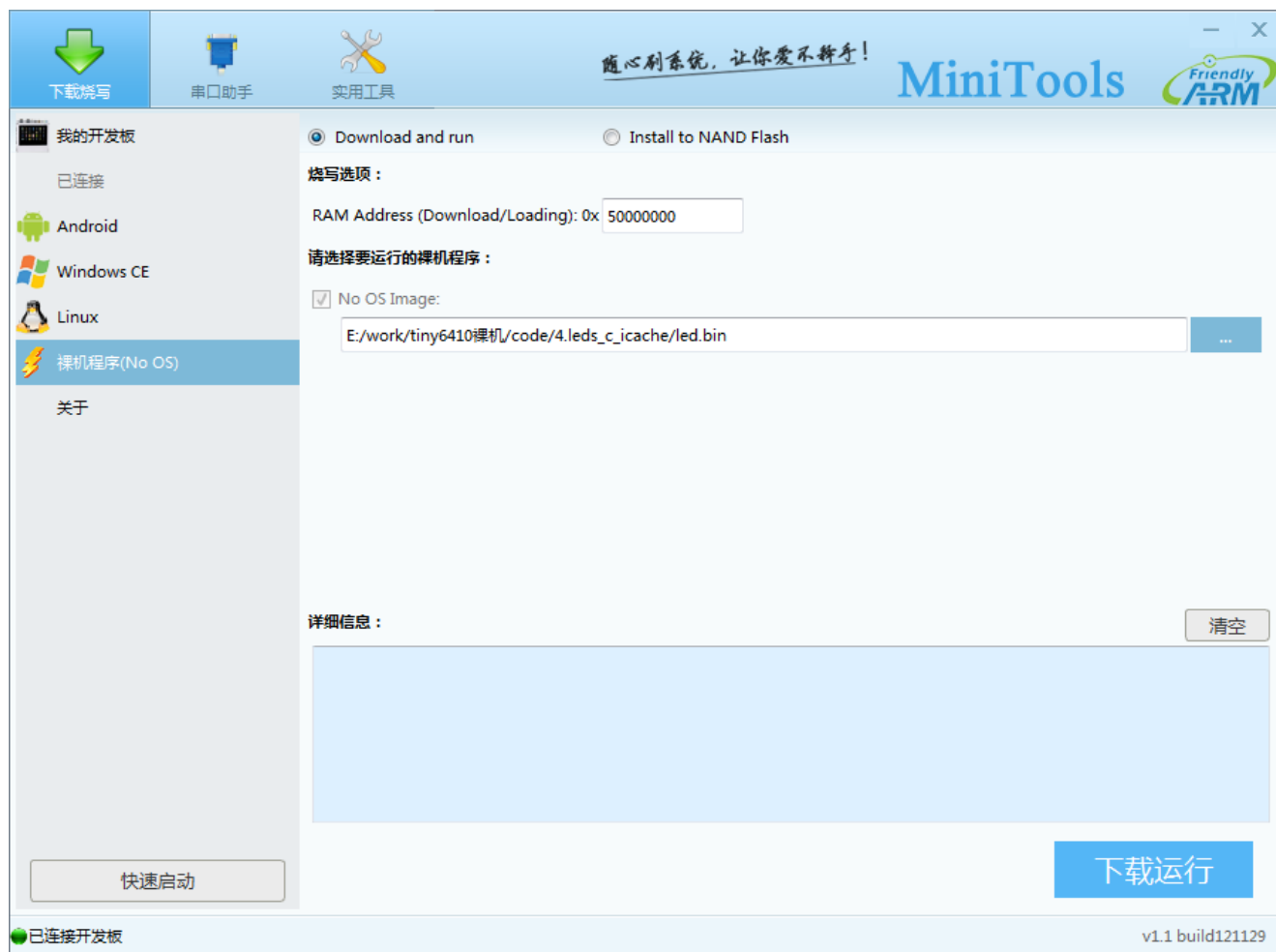

```
# cd 4.leds_c_icache
```

```
# make
```

执行 make 后会生成 led.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址” RAM Address (Download/Loading) ”，选择要运行的裸机程序，再点击” 下载运行 “就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击” 下载运行 “就可以马上看到开发板上裸机程序的运行效果了。

第四节 实验现象

相比程序 2.leds_c_sp，肉眼难以看到 LED 闪烁变快，这是因为 MiniTools 帮我们初始化了 6410 的系统时钟，如果想看出区别，需要手动降低时钟，后面将会有专门的章节介绍如何初始化



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

系统时钟。由于 icache 可以提高 CPU 的取指速度以及可以随时打开，所以程序里应该尽早的打开 icache。

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

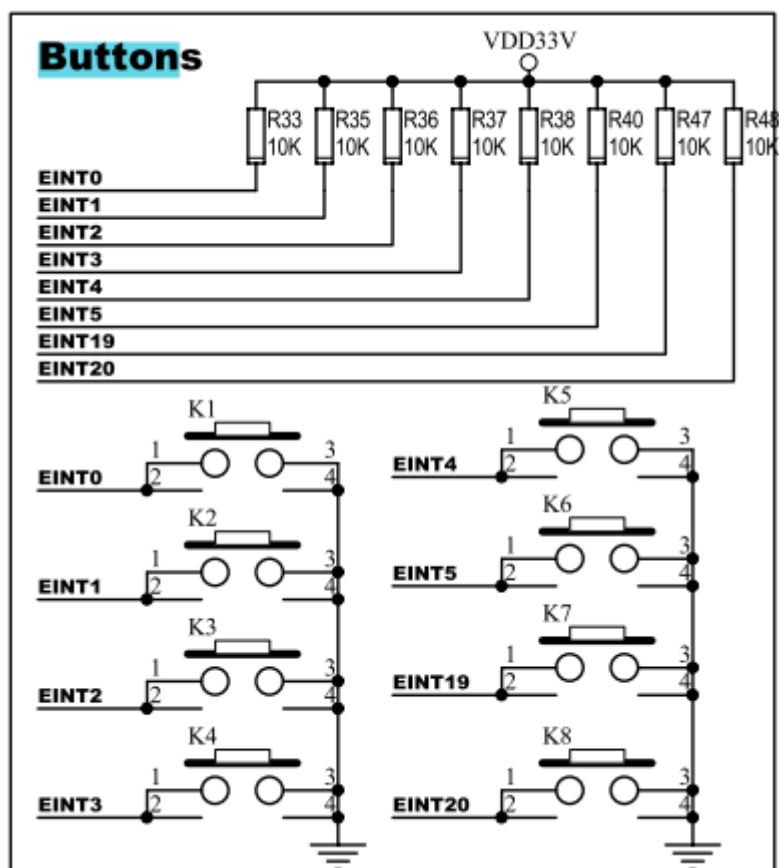
电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第六章 查询方式检测按键

第一节 查看原理图

Tiny6410 中共有 8 个用户按键，原理图如下：



按键原理图

相关的引脚信息如下图：

按键	K1	K2	K4	K4	K5	K6	K7	K8
对应的中断	EINT0	EINT1	EINT2	EINT3	EINT4	EINT5	EINT19	EINT20
可复用为 GPIO	GPN0	GPN1	GPN2	GPN3	GPN4	GPN5	GPL11	GPL12

按键引脚图

第二节 程序相关讲解

完整代码见目录 5.key_led。启动代码与前面无异，主要区别在于 main.c。

1. main.c

代码如下：

```
void main(void)
{
    int dat = 0;
    GPKCON0 = 0x11110000;    // 配置引脚 GPK4, 5, 6, 7 为输出功能
    GPKDAT = 0x000000f0;    // 所有 LED 熄灭
    GPNCON = 0;              // 配置引脚 GPN0, 1, 2, 3 为输入功能
    while(1)
    {
        dat = GPNDAT;
        if(dat & (1<<0))    // KEY1 被按下，则 LED1 亮，否则 LED1 灭
            GPKDAT |= 1<<4;
        else
            GPKDAT &= ~(1<<4);
        if(dat & (1<<1))    // KEY2 被按下，则 LED2 亮，否则 LED2 灭
            GPKDAT |= 1<<5;
        else
            GPKDAT &= ~(1<<5);
        if(dat & (1<<2))    // KEY3 被按下，则 LED3 亮，否则 LED3 灭
            GPKDAT |= (1<<6);
        else
            GPKDAT &= ~(1<<6);
        if(dat & (1<<3))    // KEY4 被按下，则 LED4 亮，否则 LED4 灭
            GPKDAT |= 1<<7;
        else
            GPKDAT &= ~(1<<7);
    }
}
```

程序很简单，首先配置 GPK 引脚为输出功能(输出低电平 LED 亮)以及配置 GPN 引脚为输入功能(按键被按下时，相应的引脚值为 1)，然后使用轮询的方式不断的读 GPN 引脚，当某个按键被按下时，对应的 GPN 引脚为高，此时我们点亮对应的 LED，否者 LED 保持熄灭的状态。

第三节 编译代码和烧写运行

编译代码

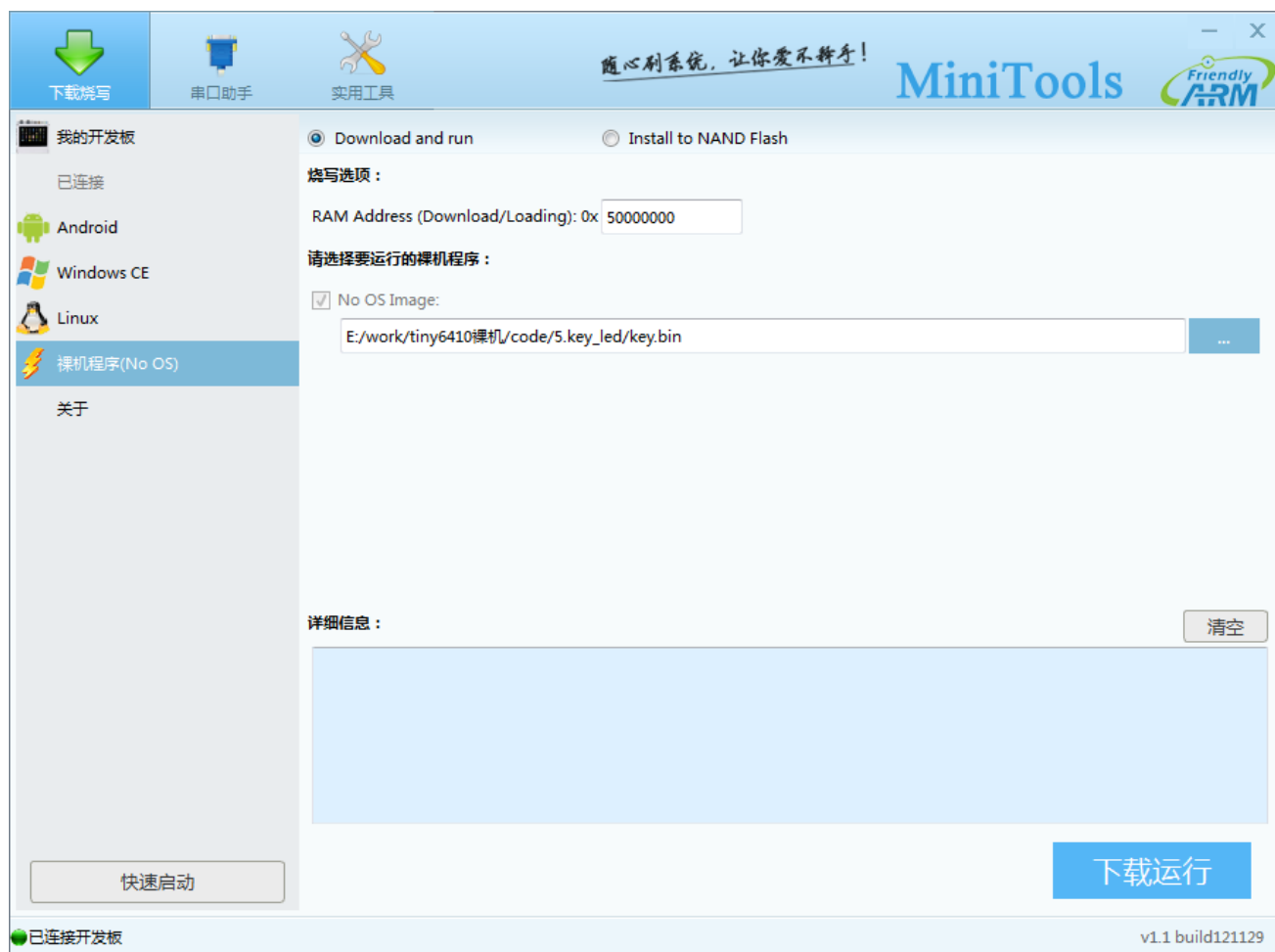
在 Fedora 终端执行如下命令：

```
# cd 5.key_led
# make
```

执行 make 后会生成 key.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址” RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击”下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击”下载运行”就可以马上看到开发板上裸机程序的运行效果了。

第四节 实验现象

未按下任何按键时，所有 LED 保持熄灭状态。当按下按键 KEY1 时，LED1 被点亮，松开按键 KEY1 时，LED1 熄灭。对于其余 3 个按键，也是相同情况。用查询的方式来检测按键太占 CPU 使用率了，除了检测按键，CPU 无法进行其他工作，后面的学习中中断的知识后，我们可以使用中断的方式来检测按键中断，这将大大降低 CPU 的使用率。

第七章 初始化时钟

第一节 S3C6410 时钟体系

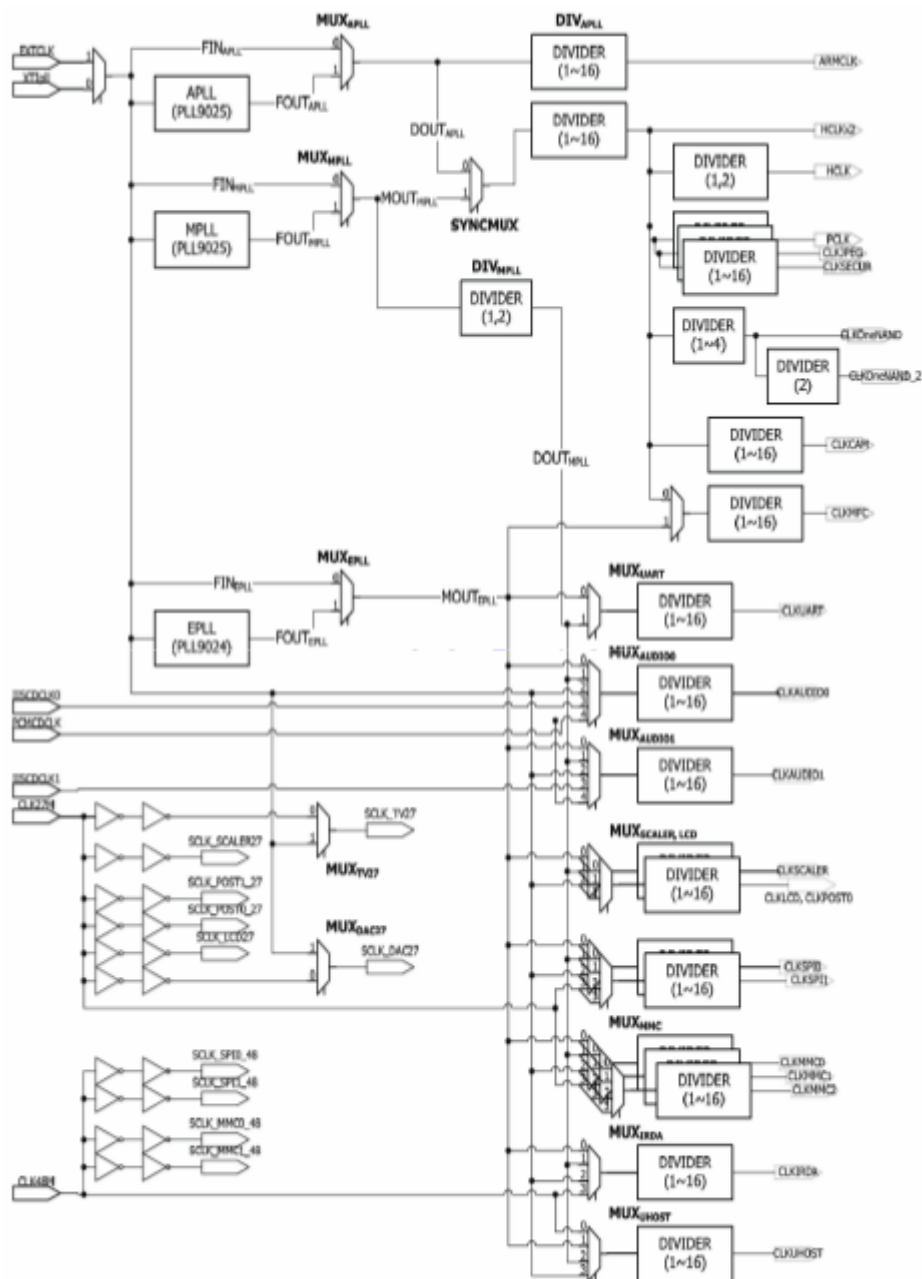
在 S3C6410 中生成所需的系统时钟信号，用于 CPU 的 ARMCLK，AXI/AHB 总线外设的 HCLK 和 APB 总线外设的 PCLK。在 S3C6410 中有三个 PLL。一个仅用于 ARMCLK，一个用于 HCLK 和 PCLK，最后一个用于外设，特别用于音频相关的时钟。通过外部提供的时钟源，时钟控制逻辑产生慢速时钟信号 ARMCLK，HCLK 和 PCLK。该每个外设块的时钟信号可能被启用或禁用，由软件控制以减少电源消耗。如何确定各类时钟的值，芯片手册上给出了参考值，我们按照参考值设置即可，各类时钟的参考值如下图：

Table 3-2. Typical value setting for clock dividers (SFR setting value / output frequency)

APLL	MPLL	DIV _{ARM}	DIV _{HCLKX2}	DIV _{HCLK}	DIV _{PCLK}	DIV _{CLKJPEG}	DIV _{CLKSECUR}
266MHz	266MHz	0 / 266MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
400MHz	266MHz	0 / 400MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
533MHz	266MHz	0 / 533MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
667MHz	266MHz	0 / 667MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz

S3C6410 时钟设置参考值图

具体如何设置上述各种各样的时钟，可参考下图：



S3C6410 时钟设置参考图

该图十分重要，依据上图我们就可以设置好所有硬件部件所需的工作时钟（实际上我们并不需要设置好所有部件的工作时钟，我们只需设置好我们需要使用的硬件部件的工作时钟即可），在本章第二节中，我们将以上图为基础，通过设置时钟相关的寄存器，达到初始化时钟的目的。

第二节 程序相关讲解

完整代码见目录 6.clock_s 和 7.clock_c。本章涉及的代码有两套,包括 6.clock_s(使用汇编初始化时钟)和 7.clock_c(使用 c 语言初始化时钟),两者的本质是一样的,初始化时钟所达到的效果也一样,想巩固汇编指令的话,可以阅读 6.clock_s 里的相关代码。由于 C 语言编写的代码思路更清晰,更便于阅读和理解,下面将以 7.clock_c 里的代码为参考进行讲解。

1. start.S

在调用 main 函数之前,调用了时钟初始化函数 clock_init,进行时钟相关的设置。

2. clock.c

时钟初始化函数 clock_init()

```
void clock_init(void)
```

```
{
```

```
    /* 1. 设置各 PLL 的 LOCK_TIME, 使用默认值 */
```

```
    APLL_LOCK = 0xffff;
```

```
    MPLL_LOCK = 0xffff;
```

```
    EPLL_LOCK = 0xffff;
```

```
    /* 2. 设置为异步模式(Asynchronous mode) */
```

```
    OTHERS &= ~0xc0;
```

```
    while ((OTHERS & 0xf00) != 0);
```

```
    /* 3. 设置分频系数 */
```

```
    CLK_DIV0 = (ARM_RATIO) | (MPLL_RATIO << 4) | (HCLK_RATIO << 8) | (HCLKX2_RATIO <<
```

```
9) | (PCLK_RATIO << 12);
```

```
    /* 4. 设置 PLL, 放大时钟 */
```

```
    APLL_CON = APLL_CON_VAL;
```

```
    MPLL_CON = MPLL_CON_VAL;
```

```
    /* 5. 选择 PLL 的输出作为时钟源 */
```

```
    CLK_SRC = 0x03;
```

```
}
```

上述代码共有 5 个步骤,下面我们来一一讲解每一个步骤:

1) 设置锁定时间

设置 PLL 后,时钟从 Fin 提升到目标频率时,需要一定的时间,即锁定时间。我们设置为最大值。

2) 设置为异步模式(Asynchronous mode)

S3C6410 硬性规定的,用 MPLL 作为 HCLK 和 PCLK 的 Source 需设置为异步 (ASYNC) 模式,若用 APLL,则设置为同步 (SYNC) 模式;



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

3.4.2.14 Others control register

REGISTER	ADDRESS	R/W	DESCRIPTION	RESET VALUE
OTHERS	0x7E00_F900	R/W	Others control register	0x0000_801E

OTHERS	BIT	DESCRIPTION	RESET VALUE
RESERVED	[31:24]	RESERVED	0x0000
STABLE COUNTER TYPE	[23]	Indicate OSC_STABLE, PWR_STABLE counter type 0 : Exponential Scale, 1 : Set by SFR	0
RESERVED	[22:17]	DO NOT CHANGE	0x0000
USB_SIG_MASK	[16]	USB signal mask to prevent unwanted leakage. (This bit must set before USB PHY is used.)	0
RESERVED	[15:14]	RESERVED	0x2
CLEAR_DBGACK	[13]	Clear DBGACK signal when this field has 1. ARM1176 asserts DBGACK signal to indicate the system has entered Debug state. If DBGACK is asserted, this state is store in SYSCON until software clear it using this field.	0
CLEAR_BATF_INT	[12]	Clear interrupt caused by battery fault when this bit is set.	0
SYNCACK	[11:8]	SYNC mode acknowledge (Read Only)	0x0
SYNCMODE	[7]	SYNCMODEREQ to ARM 0: Asynchronous mode, 1: Synchronous mode	0
SYNCMUXSEL	[6]	SYS CLOCK SELECT IN CMU	0

3) 设置分频

与分频相关的寄存器是 CLK_DIV0，见下图：

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

CLK_DIV0	BIT	DESCRIPTION	RESET VALUE
MFC_RATIO	[31:28]	MFC clock divider ratio $CLKMFC = CLKMFC_{IN} / (MFC_RATIO + 1)$	0x0
JPEG_RATIO	[27:24]	JPEG clock divider ratio, which must be odd value. In other words, S3C6410 supports only even divider ratio. $CLKJPEG = HCLKX2 / (JPEG_RATIO + 1)$	0x1
CAM_RATIO	[23:20]	CAM clock divider ratio $CLKCAM = HCLKX2 / (CAM_RATIO + 1)$	0x0
SECUR_RATIO	[19:18]	Security clock divider ratio, which must be 0x1 or 0x3. $CLKSECUR = HCLKX2 / (SECUR_RATIO + 1)$	0x1
RESERVED	[17:16]	RESERVED	0x1
PCLK_RATIO	[15:12]	PCLK clock divider ratio, which must be odd value. In other words, S3C6410 supports only even divider ratio. $PCLK = HCLKX2 / (PCLK_RATIO + 1)$	0x1
HCLKX2_RATIO	[11:9]	HCLKX2 clock divider ratio $HCLKX2 = HCLKX2_{IN} / (HCLKX2_RATIO + 1)$	0x0
HCLK_RATIO	[8]	HCLK clock divider ratio $HCLK = HCLKX2 / (HCLK_RATIO + 1)$	0
RESERVED	[7:5]	RESERVED	0x0
MPLL_RATIO	[4]	DIV _{MPLL} clock divider ratio $DOUT_{MPLL} = MOUT_{MPLL} / (MPLL_RATIO + 1)$	0
ARM_RATIO	[3:0]	DIV _{ARM} clock divider ratio $ARMCLK = DOUT_{APLL} / (ARM_RATIO + 1)$	0x0

现在我们来根据本章第一节中给出的时钟设置参考值来设置该寄存器。

Table 3-2. Typical value setting for clock dividers (SFR setting value / output frequency)

APLL	MPLL	DIV _{ARM}	DIV _{HCLKX2}	DIV _{HCLK}	DIV _{PCLK}	DIV _{CLKJPEG}	DIV _{CLKSECUR}
266MHz	266MHz	0 / 266MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
400MHz	266MHz	0 / 400MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
533MHz	266MHz	0 / 533MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz
667MHz	266MHz	0 / 667MHz	0 / 266MHz	1 / 133MHz	3 / 66MHz	3 / 66MHz	3 / 66MHz

- ARMCLK = 1000MHz = DOUT_{APLL} / (ARM_RATIO + 1), 经过在第四和第五步的设置后, DOUT_{APLL} 会被设置为 532MHz, 所以 ARM_RATIO = 0 即可;
 - DOUT_{MPLL} = MOUT_{MPLL} / (MPLL_RATIO + 1), 经过在第四和第五步的设置后 MOUT_{MPLL} = 532MHz, 所以 MPLL_RATIO = 0;
 - HCLKX2 = HCLKX2_{IN} / (HCLKX2_RATIO + 1) = 532 / (HCLKX2_RATIO + 1) = 266MHz, 所以 HCLKX2_RATIO = 1;
 - HCLK = HCLKX2 / (HCLK_RATIO + 1) = 266 / (HCLK_RATIO + 1) = 133MHz, 所以 HCLK_RATIO = 1;
 - PCLK = HCLKX2 / (PCLK_RATIO + 1) = 266 / (PCLK_RATIO + 1) = 66.5 MHz, 所以 PCLK_RATIO = 3;
- 所以 CLK_DIV0 = (ARM_RATIO) | (MPLL_RATIO << 4) | (HCLK_RATIO << 8) | (HCLKX2_RATIO << 9) | (PCLK_RATIO << 12);



4) 设置 PLL

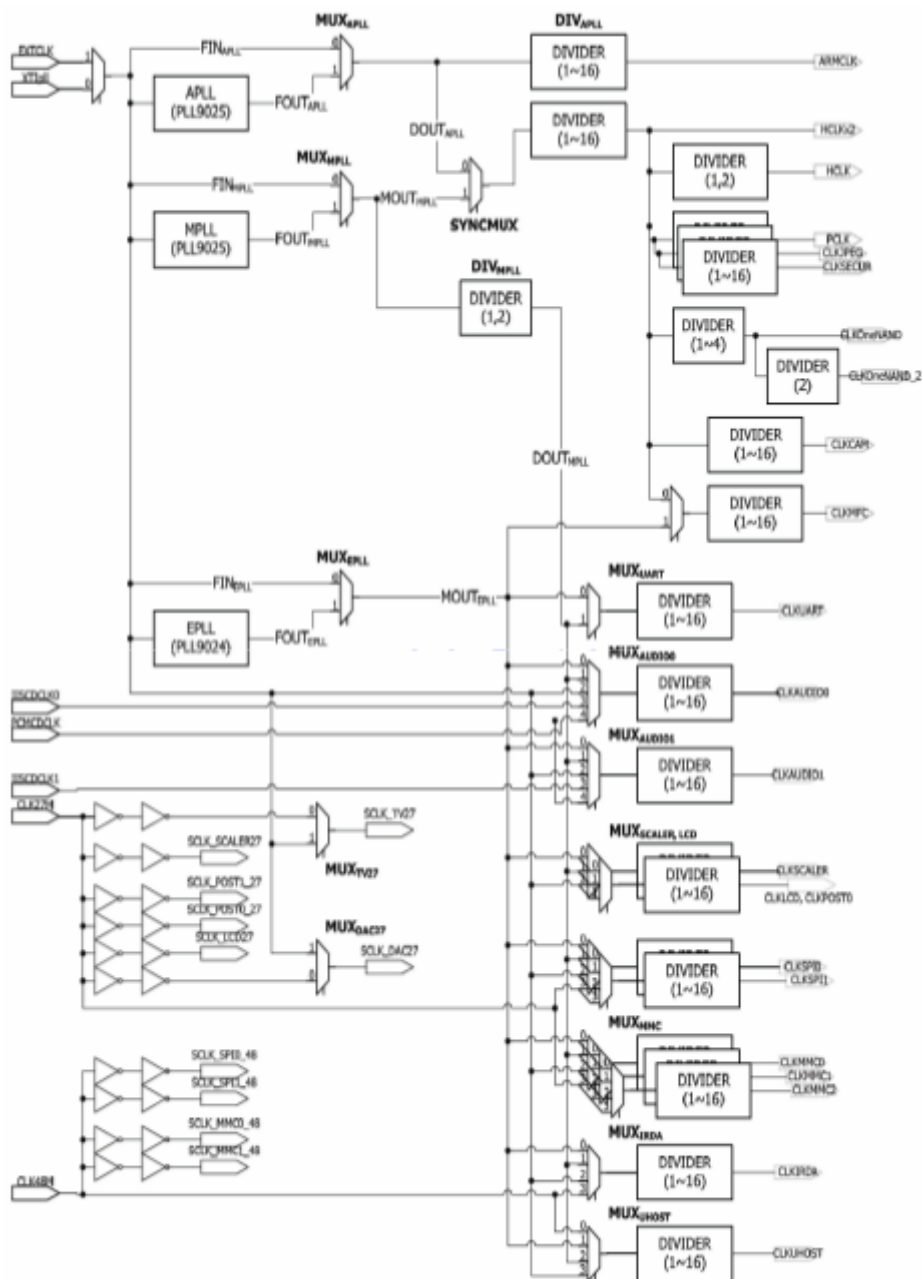
设置好分频后，我们就需要设置 PLL 了。APLL/MPLL 的启动是通过设置 APLL_CON0/MPLL_CON 寄存器：

APLL_CON / MPLL_CON	BIT	DESCRIPTION	RESET VALUE
ENABLE	[31]	PLL enable control (0: disable, 1: enable)	0
RESERVED	[30:26]	RESERVED	0x00
MDIV	[25:16]	PLL M divide value	0x190 / 0x214
RESERVED	[15:14]	RESERVED	0x0
PDIV	[13:8]	PLL P divide value	0x3 / 0x6
RESERVED	[7:3]	RESERVED	0x00
SDIV	[2:0]	PLL S divide value	0x2 / 0x3

我们将 APLL 和 MPLL 的输出(FOUT)都设置为 533MHz，由于 $FOUT = MDIV * FIN / (PDIV * 2^{SDIV})$ ，参考下面这个表格，我们取 MDIV=266, PDIV=3, SDIV=2。

FIN (MHz)	Target FOUT (MHz)	MDIV	PDIV	SDIV
12	100	400	3	4
12	200	400	3	3
12	266	266	3	2
12	400	400	3	2
12	533	266	3	1
12	667	333	3	1

5) 设置各种时钟开关



S3C6410 时钟设置参考图

在上图中，所有的 MUX 都是用来选择时钟的，相关寄存器是 CLK_SRC，见下图：

CLK_SRC	BIT	DESCRIPTION	RESET VALUE
TV27_SEL	[31]	Control MUX _{TV27} , which is the source clock of TV27MHz (0: 27MHz, 1: FIN _{EPLL})	0
DAC27_SEL	[30]	Control MUX _{DAC27} , which is the source clock of DAC27MHz (0:27MHz, 1: FIN _{EPLL})	0
SCALER_SEL	[29:28]	Control MUX _{SCALER} , which is the source clock of TVSCALER (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL})	0x0
LCD_SEL	[27:26]	Control MUX _{LCD} , which is the source clock of LCD (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL})	0x0
IRDA_SEL	[25:24]	Control MUX _{IRDA} , which is the source clock of IRDA (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 48MHz)	0x0
MMC2_SEL	[23:22]	Control MUX _{MMC2} , which is the source clock of MMC2 (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 27MHz)	0x0
MMC1_SEL	[21:20]	Control MUX _{MMC1} , which is the source clock of MMC1 (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 27MHz)	0x0
MMC0_SEL	[19:18]	Control MUX _{MMC0} , which is the source clock of MMC0 (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 27MHz)	0x0
SPI1_SEL	[17:16]	Control MUX _{SPI1} , which is the source clock of SPI1 (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 27MHz)	0x0
SPI0_SEL	[15:14]	Control MUX _{SPI0} , which is the source clock of SPI0 (00:MOUT _{EPLL} , 01: DOUT _{MPLL} , 10: FIN _{EPLL} , 11: 27MHz)	0x0
UART_SEL	[13]	Control MUX _{UART0} , which is the source clock of UART (0:MOUT _{EPLL} , 1: DOUT _{MPLL})	0
AUDIO1_SEL	[12:10]	Control MUX _{AUDIO1} , which is the source clock of IIS1, PCM1, and AC97 1 (000:MOUT _{EPLL} , 0 01: DOUT _{MPLL} , 010:FIN _{EPLL} , 011: IISCDCLK1, 100: PCMCDCCLK)	0x0
AUDIO0_SEL	[9:7]	Control MUX _{AUDIO0} , which is the source clock of IIS0, PCM0, and AC97 0 (000:MOUT _{EPLL} , 001: DOUT _{MPLL} , 010:FIN _{EPLL} , 011: IISCDCLK0, 10x: PCMCDCCLK)	0x0
UHOST_SEL	[6:5]	Control MUX _{UHOST} , which is the source clock of USB Host	0x0
		(00:48MHz, 01:MOUT _{EPLL} , 10: DOUT _{MPLL} , 11:FIN _{EPLL})	
MFCCLK_SEL	[4]	Control MUX _{MFC} , which is the source clock of MFC	0
RESERVED	[3]	RESERVED	0
EPLL_SEL	[2]	Control MUX _{EPLL} (0:FIN _{EPLL} , 1:FOUT _{EPLL})	0
MPLL_SEL	[1]	Control MUX _{MPLL} (0:FIN _{MPLL} , 1:FOUT _{MPLL})	0
APLL_SEL	[0]	Control MUX _{APLL} (0:FIN _{APLL} , 1:FOUT _{APLL})	0

参考 S3C6410 时钟设置参考图，设置各种时钟开关：

APLL_SEL=1，使用 FOUTAPLL

MPLL_SEL=1，使用 FOUTMPLL

其余都为 0，所以 CLK_SRC0=0x03；

3. main.c

在 main 函数中实现 LED 闪烁的功能，与前面的代码大同小异。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

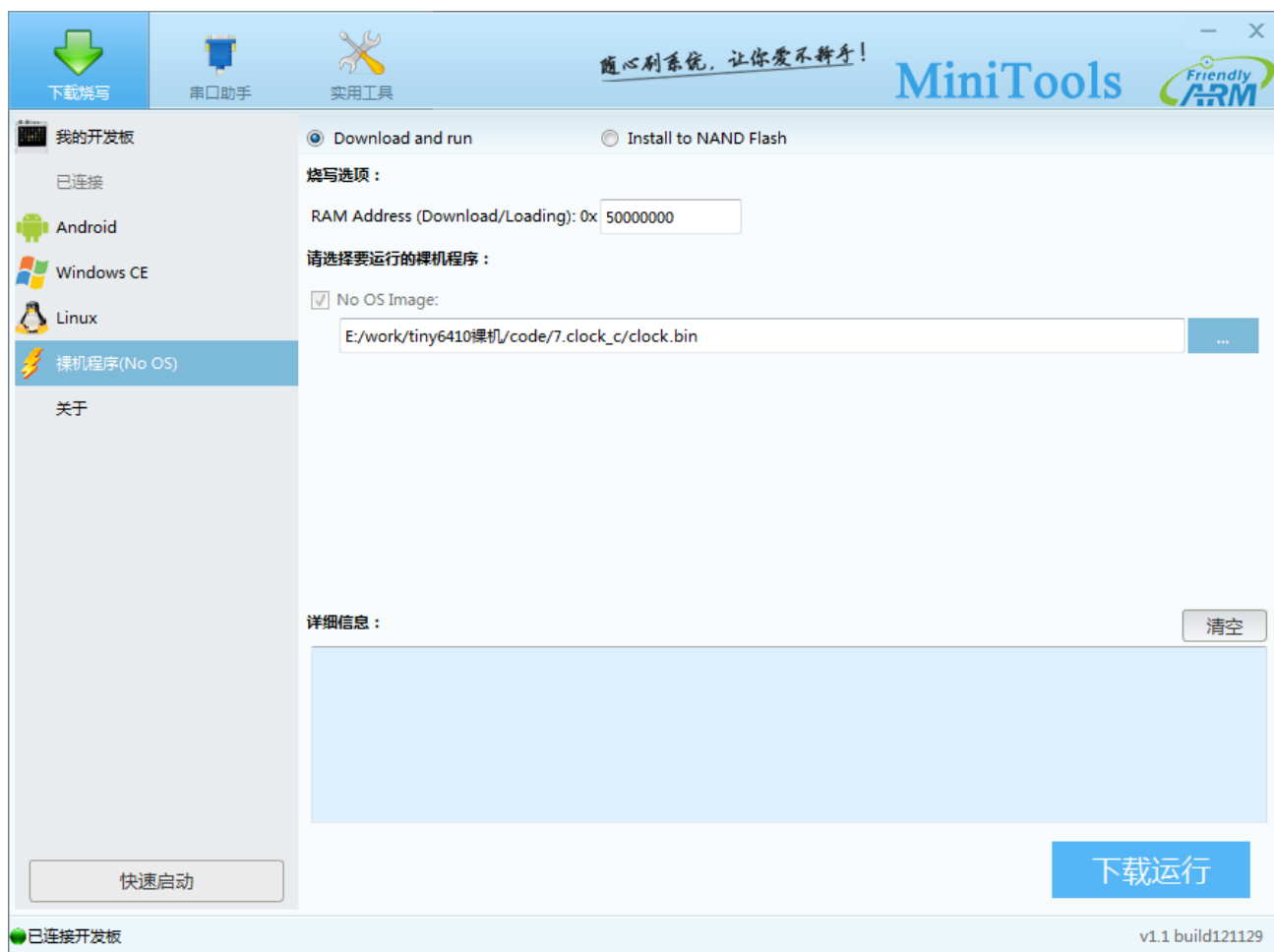
```
# cd 7.clock_c
```

```
# make
```

执行 make 后会生成 clock.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址”RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击”下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击”下载运行”就可以马上看到开发板上裸机程序的运行效果了。



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

第四节 实验现象

初始化时钟后，LED 闪烁得更快。本章的时钟设置主要是针对 APLL 和 MPLL，这足以让大部分硬件正常工作了，下一章我们将初始化 UART 以达到在终端输入输出字符的目的，这将使用到我们本章所设置好的时钟。

第八章 串口设置之输入输出字符

第一节 S3C6410 UART 相关说明

通用异步收发器简称 UART，即 UNIVERSAL ASYNCHRONOUS RECEIVER AND TRANSMITTER，它用来传输串行数据。发送数据时，CPU 将并行数据写入 UART，UART 按照一定的格式在一根电线上串行发出；接收数据时，UART 检测另一根电线的信号，将串行收集在缓冲区中，CPU 即可读取 UART 获得这些数据。S3C6410 的 UART 提供了四个独立的异步串行 I / O（SIO）端口。每个异步串行 I/O（SIO）端口通过中断或者直接存储器存取（DMA）模式来操作。在 CPU 和 UART 之间传输数据的。该 UART 使用系统时钟的时间可以支持的比特率最高为 115.2kb/s。每个 UART 的通道包含了两个 64 字节收发 FIFO 存储器。S3C6410 的 UART 结构图如下：

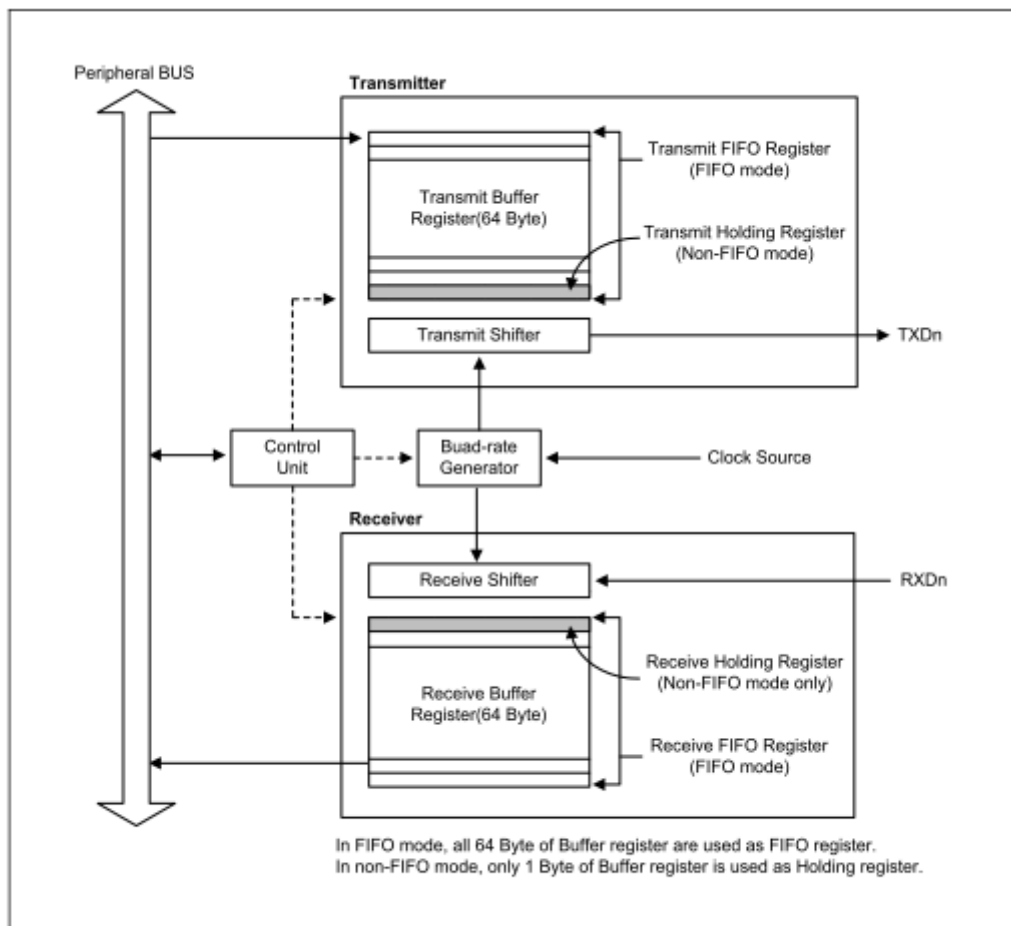
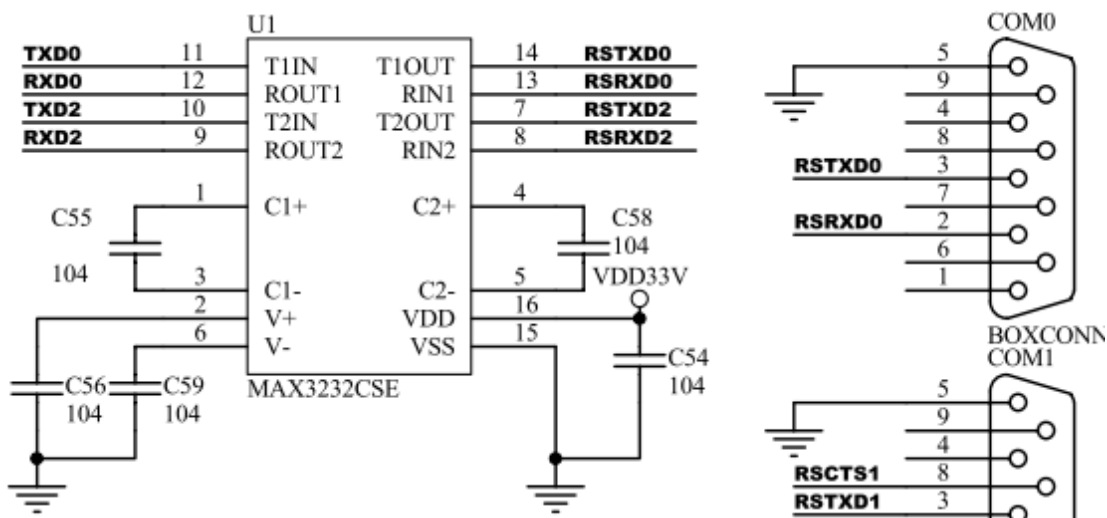


Figure 31-1. UART Block Diagram

S3C6410 的 UART 结构图

UART 使用标准的 TTL/CMOS 逻辑电平来表示数据，为了增强数据抗干扰能力和提高传输长度，通常将 TTL/CMOS 逻辑电平转换为 RS-232 逻辑电平，查看原理图可知 Tiny6410 使用的是

MAX3232SOP 芯片,使用的是 TX0 和 DX0:



搜索“txd0”,可知:

RXD0	D20	XuRXD0/GPA0
TXD0	A23	XuTXD0/GPA1
	G16	XuCTS0/GPA2
	A22	XuRTSn0/GPA3
	J15	XuRXD1/GPA4
RXD1	B22	XuTXD1/GPA5
TXD1	H15	XuCTS1/GPA6
CTS1	C22	XuRTSn1/GPA7
RTSn1	D19	XuRXD2/ExdREQ/IrRXD/GPB
RXD2	A21	XuTXD2/ExdACK/IrTXD/GPB
TXD2	J14	XuRXD3/IrRXD/ExdREQ/GPB
RXD3	B21	XuTXD3/IrTXD/ExdACK/GPB
TXD3	G15	XirSDBW/XcamFIELD/BUF_D
GPB4		

UART 引脚连接图

通过设置 UART 相关寄存器,我们就可以驱动 UART 工作,达到发送和接收字符的目的。

第二节 程序相关讲解

完整代码见目录 8.uart_putchar,对比前一个目录 7.clock_c,区别在于 main.c 和多了一个 uart.c 文件。

1. main.c

完整代码如下:

```
int main()
{
```

```
char c;
init_uart();          // 初始化串口
while (1)
{
    c = getc ();       // 接收一个字符 c
    putc(c+1);         // 发送字符 c+1
}

return 0;
}
```

在 start.S 中初始化时钟后执行 bl main 跳转到 main 函数, 在 main 函数中, 先会调用 uart_init() 初始化 UART, 然后使用 getchar 接收字符, 再调用 putchar() 将该字符+1 发送出去。

2. uart.c

init_uart() 代码如下:

```
void init_uart(void)
{
    /* 1. 配置引脚 */
    GPACON &= ~0xff;
    GPACON |= 0x22;

    /* 2. 设置数据格式等 */
    ULCON0 = 0x3;           // 数据位:8, 无校验, 停止位: 1, 8n1
    UCON0 = 0x5;            // 时钟: PCLK, 禁止中断, 使能 UART 发送、接收
    UFCON0 = 0x01;          // FIFO ENABLE
    UMCN0 = 0;              // 无流控

    /* 3. 设置波特率 */
    UBRDIV0 = 35;
    UDIVSL0T0 = 0x1;
}
```

上述代码共有 3 个步骤, 下面我们来一一讲解每一个步骤:

1) 配置引脚用于 RX/TX 功能

参考 UART 引脚连接图, 我们需要设置 GPACON 寄存器使 GPA 引脚用于 UART 功能。

GPACon	Bit	Description	Initial State
GPA0	[3:0]	0000 = Input 0010 = UART RXD[0] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [0]	0000
GPA1	[7:4]	0000 = Input 0010 = UART TXD[0] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [1]	0000
GPA2	[11:8]	0000 = Input 0010 = UART CTSn[0] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [2]	0000
GPA3	[15:12]	0000 = Input 0010 = UART RTSn[0] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [3]	0000
GPA4	[19:16]	00 = Input 10 = UART RXD[1] 0100 = Reserved 0110 = Reserved 01 = Output 11 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [4]	0000
GPA5	[23:20]	0000 = Input 0010 = UART TXD[1] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [5]	0000
GPA6	[27:24]	0000 = Input 0010 = UART CTSn[1] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [6]	0000
GPA7	[31:28]	0000 = Input 0010 = UART RTSn[1] 0100 = Reserved 0110 = Reserved 0001 = Output 0011 = Reserved 0101 = Reserved 0111 = External Interrupt Group 1 [7]	0000

GPACon 寄存器图

2) 设置数据格式等

ULCON0 用来设置数据格式，见下图：

ULCONn	Bit	Description	Initial State
Reserved	[7]		0
Infra-Red Mode	[6]	Determine whether or not to use the Infra-Red mode. 0 = Normal mode operation 1 = Infra-Red Tx/Rx mode	0
Parity Mode	[5:3]	Specify the type of parity generation and checking during UART transmit and receive operation. 0xx = No parity 100 = Odd parity 101 = Even parity 110 = Parity forced/checked as 1 111 = Parity forced/checked as 0	000
Number of Stop Bit	[2]	Specify how many stop bits are to be used for end-of-frame signal. 0 = One stop bit per frame 1 = Two stop bit per frame	0
Word Length	[1:0]	Indicate the number of data bits to be transmitted or received per frame. 00 = 5-bit 01 = 6-bit 10 = 7-bit 11 = 8-bit	00

Word Length = 11, 8bit 的数据;
 Number of Stop Bit = 0, 1bit 的停止位;
 Parity Mode = 000, 无校验;
 Infrared Mode = 0, 使用普通模式;
 所以 ULCON0=0x3

UCON0 是 UART 的配置寄存器, 见下图

UCONn	Bit	Description	Initial State
Clock Selection	[11:10]	Select PCLK or EXT_UCLK0 ¹⁾ or EXT_UCLK1 ⁴⁾ clock for the UART baud rate. x0 = PCLK : $DIV_VAL^{1)} = (PCLK / (bps \times 16)) - 1$ 01 = EXT_UCLK0: $DIV_VAL^{1)} = (EXT_UCLK0 / (bps \times 16)) - 1$ 11 = EXT_UCLK1: $DIV_VAL^{1)} = (EXT_UCLK1 / (bps \times 16)) - 1$	0
Tx Interrupt Type	[9]	Interrupt request type. 0 = Pulse (Interrupt is requested as soon as the Tx buffer becomes empty in Non-FIFO mode or reaches Tx FIFO Trigger Level in FIFO mode.) 1 = Level (Interrupt is requested while Tx buffer is empty in Non-FIFO mode or reaches Tx FIFO Trigger Level in FIFO mode.)	0
Rx Interrupt Type	[8]	Interrupt request type. ²⁾ 0 = Pulse (Interrupt is requested the instant Rx buffer receives the data in Non-FIFO mode or reaches Rx FIFO Trigger Level in FIFO mode.) 1 = Level (Interrupt is requested while Rx buffer is receiving data in Non-FIFO mode or reaches Rx FIFO Trigger Level in FIFO mode.)	0
Rx Time Out Enable	[7]	Enable/Disable Rx time-out interrupts when UART FIFO is enabled. The interrupt is a receive interrupt. ³⁾ 0 = Disable 1 = Enable	0
Rx Error Status Interrupt Enable	[6]	Enable the UART to generate an interrupt upon an exception, such as a break, frame error, parity error, or overrun error during a receive operation. 0 = Do not generate receive error status interrupt. 1 = Generate receive error status interrupt.	0
Loop-back Mode	[5]	Setting loop-back bit to 1 cause the UART to enter the loop-back mode. This mode is provided for test purposes only. 0 = Normal operation 1 = Loop-back mode	0
Send Break Signal	[4]	Setting this bit causes the UART to send a break during 1 frame time. This bit is automatically cleared after sending the break signal. 0 = Normal transmit 1 = Send break signal	0

UCONn	Bit	Description	Initial State
Transmit Mode	[3:2]	Determine which function is currently able to write Tx data to the UART transmit buffer register. 00 = Disable 01 = Interrupt request or polling mode 10 = DMA request (DMA_UART0) 11 = DMA request (DMA_UART1)	00
Receive Mode	[1:0]	Determine which function is currently able to read data from UART receive buffer register. 00 = Disable 01 = Interrupt request or polling mode 10 = DMA request (DMA_UART0) 11 = DMA request (DMA_UART1)	00

Receive Mode = 01，使用中断模式或者轮询模式；
 Transmit Mode = 01，使用中断模式或者轮询模式；
 Send Break Signal = 0，普通传输；
 Loop-back Mode = 0，不使用回环方式；
 我们采用轮询的方式接受和发送数据，不使用中断，所以 bit[6-9] 均为 0；
 Clock Selection = 0，使用 PCLK 作为 UART 的工作时钟；
 我们不使用 DMA，所以 bit[16] 和 bit[20] 均为 0；
 所以 UCON0 = 0x5

UFCON0 和 UMCN0

这两个寄存器比较简单，UFCON0 用来使能 FIFO，UMCN0 用来设置无流控。

3) 设置波特率

波特率即每秒传输的数据位数，涉及两个寄存器：UBRDIV0 和 UDIVSLOT0

UBRDIV n	Bit	Description	Initial State
UBRDIV	[15:0]	Baud rate division value (When UART clock source is PCLK, UBRDIVn must be more than 0 (UBRDIVn > 0))	-

NOTE: If UBRDIV value is 0, UART baudrate is not affected by UDIVSLOT value.

UDIVSLOT n	Bit	Description	Initial State
Reserved	[31:16]	Reserved	0
UDIVSLOTn	[15:0]	Select the slot where clock generator divide clock source	0x0000

波特率设置相关公式： $UBRDIVn + (\text{num of 1's in } UDIVSLOTn) / 16 = (PCLK / (bps \times 16)) - 1$ 。
 PCLK = 66.5MHz，我们的波特率 bps 设置为 115200，所以
 $(66.5\text{MHz} / (115200 \times 16)) - 1 = 35.08 = UBRDIVn + (\text{num of 1's in } UDIVSLOTn) / 16$ ，所以我们设置 UBRDIV0=35，UDIVSLOT0=0x1。

另外，getchar() 和 putchar() 代码如下：

```
char getchar(void)           // 接收一个字符
{
    while ((UFSTAT0 & 0x7f) == 0); // 如果 RX FIFO 空，等待
    return URXH0;               // 取数据
}

void putchar(char c)         // 发送一个字符
{
    while (UFSTAT0 & (1<<14)); // 如果 TX FIFO 满，等待
    UTXH0 = c;                // 写数据
}
```

UTXHn	Bit	Description	Initial State
TXDATAn	[7:0]	Transmit data for UARTn	-

UART 数据发送寄存器

URXHn	Bit	Description	Initial State
RXDATAn	[7:0]	Receive data for UARTn	0x00

UART 数据接收寄存器

UFSTATn	Bit	Description	Initial State
Reserved	[15]		0
Tx FIFO Full	[14]	Set to 1 automatically whenever transmit FIFO is full during transmit operation 0 = 0-byte ≤ Tx FIFO data ≤ 63-byte 1 = Full	0
Tx FIFO Count	[13:8]	Number of data in Tx FIFO	0
Reserved	[7]		0
Rx FIFO Full	[6]	Set to 1 automatically whenever receive FIFO is full during receive operation 0 = 0-byte ≤ Rx FIFO data ≤ 63-byte 1 = Full	0
Rx FIFO Count	[5:0]	Number of data in Rx FIFO	0

发送/接收状态寄存器

通过读 UTRSTAT0 发送/接收状态寄存器，当 Rx FIFO FULL 为 1 或者 Rx FIFO Count>0 时说明接收到数据，读 URXH0 寄存器可以得到 8bit 的数据；当如果 TX FIFO FULL=1 时，暂停发送数据，否则写 8bit 的数据到 UTXH0。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

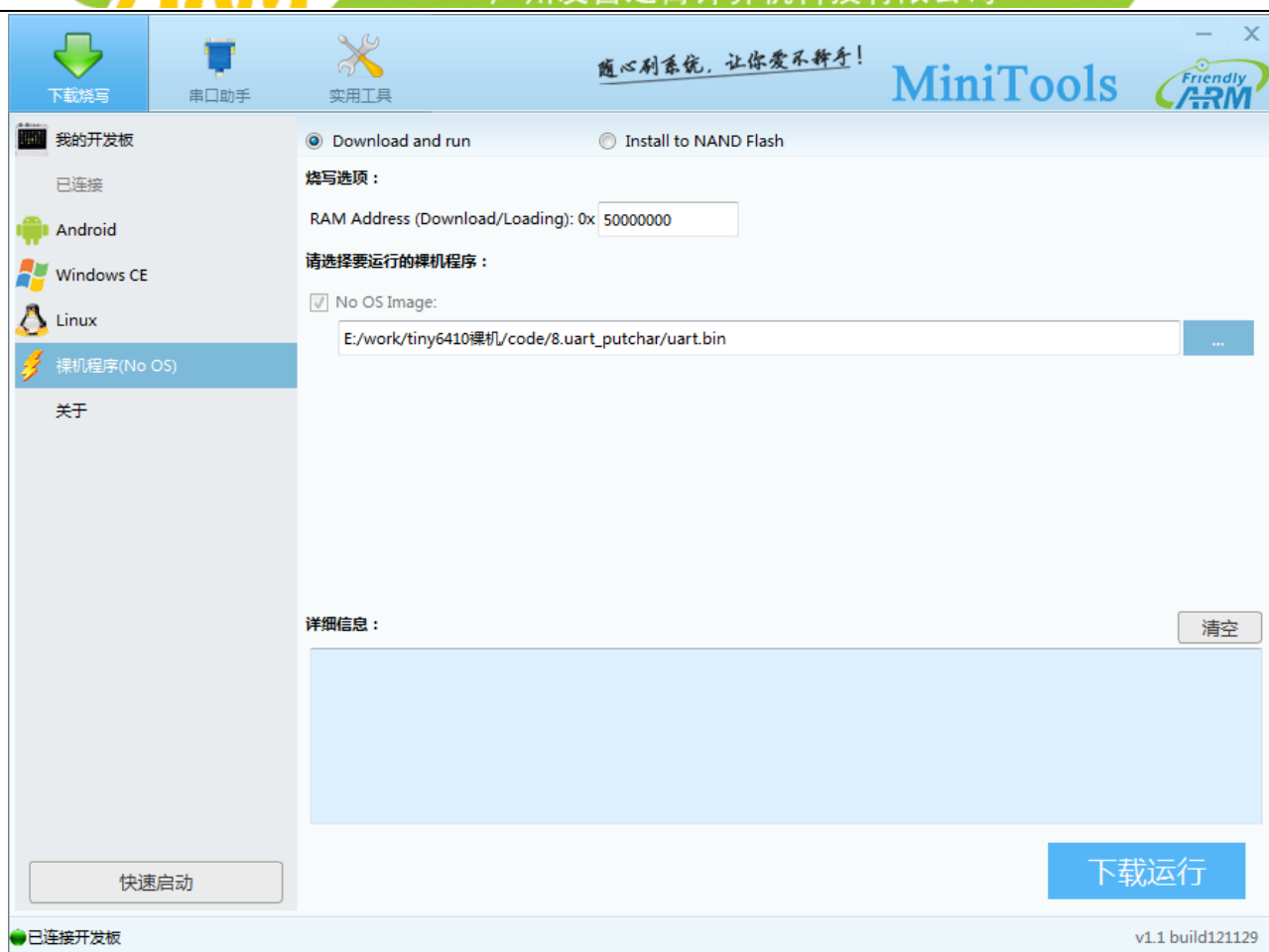
```
# cd 8. uart_putchar
```

```
# make
```

执行 make 后会生成 uart.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 DRAM，其设置方式如下：



先选中上方的” Download and run “，设置好下载地址”RAM Address(Download/Loading)”，选择要运行的裸机程序，再点击”下载运行”就可以了。MiniTools 首先会把裸机程序下载到 DRAM 的地址 0x50000000 处，然后跳转到该地址上运行裸机程序，所以只要 PC 上再点击”下载运行”就可以马上看到开发板上裸机程序的运行效果了。

第四节 实验现象

连接好串口终端后，在 PC 键盘上输入一个字符，则串口终端会显示该字符在 ASCII 表中的下一字符，例如键盘输入”abcd”，终端上会显示”bcde”，效果如下：

```
USB Mode: waiting...
Hello USB Loop
USB Mode: Connected
Downloading User-Bin...
Downloading User-Bin succeed
Run UserBin
bcde
```

通过本章对 UART 的初始化，我们已经能在终端上打印字符了，这对我们调试代码有极大的帮助。但是目前代码的功能太单一，只能简单地输入或者输出一个字符，在后面的章节中，我们将



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

为程序添加功能强大的 printf() 和 scanf() 功能。

第九章 重定位代码到 SRAM+4096

第一节 两个不同的地址概念

对于程序而言，我们需要理解两个概念，一是程序当前所处的地址，即程序在运行时，所处的当前地址；二是程序应该位于的运行地址，即编译程序时，所指定的程序的链接地址。对于 S3C6410 而言，启动时只会从 NAND Flash/MMC 等启动设备中拷贝前 8K 的代码到 SRAM 中，然后跳到 SRAM 中运行我们的代码。那么当我们的程序超过 8K 怎么办？那就需要我们在前 8K 的代码中将整个程序完完整整地拷贝到 DRAM 等其他更大存储空间，然后再跳转到 DRAM 中继续运行我们的代码，这个拷贝然后跳转的过程就叫重定位。在前面章节里，我们都是直接将 bin 文件下载到 DRAM 中运行的，所以不需要我们进行重定位。而在本章节，我们将通过 NAND 启动然后进行重定位的方式来运行我们的程序。

第二节 程序相关讲解

完整代码见目录 9.link_4096, 该目录下的代码与上一章的代码的差别在于修改了 start.S 和多了链接脚本 link.lds，我们首先分析 link.lds。

1. link.lds

什么是链接脚本？链接脚本就是程序链接时的参考文件，其主要目的是描述如何把输入文件中的段 (SECTION) 映射到输出文件中，并控制输出文件的存储布局。链接脚本的基本命令式 SECTIONS 命令，一个 SECTIONS 命令内部包含一个或多个段，段 (SECTION) 是链接脚本的基本单元，它表示输入文件中的某个段是如何放置的。链接脚本的标准格式如下：

```
SECTIONS
{
    sections-command
    sections-command
}
```

下面我们配合 link.lds 进行具体讲解：

```
SECTIONS
{
    . = 0x1000;
    .text : {
        start.o
        * (.text)
    }

    .data : {
```



```
* (.data)
}

bss_start = .;
.bss : {
    * (.bss)
}
bss_end = .;
}
```

1) 在链接脚本中, 单独的点号(.)代表了当前位置, . = 0x1000;表示我们的代码的运行地址是 0x1000;

2) link.lds 中的 .text / .data / .bss 分别是 text 段、data 段、bss 段的段名(这些段名并不是固定的,是可以随便起的)。。.text 段包含的内容是 start.o 和其余代码中所有的 text 段, .data 段包含的内容是代码中所有的 data 段, .bss 段包含的内容是代码中所有的 bss 段。

3) bss_start 和 bss_end 保存的是 bss 段的起始地址和结束地址, 在 start.S 中会被用到。

下面解释一下什么是 data、text、bss 段:

data 段:

数据段 (data segment) 通常是指用来存放程序中已初始化的全局变量的一块内存区域。数据段属于静态内存分配。

text 段:

代码段通常是指用来存放程序执行代码的一块内存区域。这部分区域的大小在程序运行前就已经确定, 并且内存区域通常属于只读, 某些架构也允许代码段为可写, 即允许修改程序。在代码段中, 也有可能包含一些只读的常数变量, 例如字符串常量等。

bss 段:

指用来存放程序中未初始化的全局变量的一块内存区域。BSS 是英文 Block Started by Symbol 的简称。当我们的程序有全局变量是, 它是放在 bss 段的, 由于全局变量默认初始值都是 0, 一般需要我们手动清 bss 段。

2. start.S

在 start.S 中, 有 4 个需要注意的地方:

1) 将栈顶指针 SP 指向 8*1024, NAND Flash 启动时, S3C6410 的内部 8K 的 SRAM 被映射到 0x0, 而 ARM 默认的栈是递减的, 所以可以让 SP 指向 8*1024;

2) 重定位, 代码如下:

```
adr r0, _start           @ _start 的当前地址
ldr r1, =_start          @ _start 的链接地址
ldr r2, =bss_start
cmp r0, r1
beq clean_bss
```

```
copy_loop:
    ldr r3, [r0], #4
    str r3, [r1], #4
    cmp r1, r2
    bne copy_loopp
```

首先获得_start 标号的当前地址(即 0x0), 然后获取_start 标号的链接地址(即 0x1000), 因为 bin 文件中不需要保存 bss 段, 所以拷贝的长度为 bss_start 的链接地址减去_start 的链接地址。

注: adr 指令获取的是代码当前位于的地址, 而 ldr 指令获取的是代码的链接地址。

3) 清 bss, 代码如下:

```
clean_bss:
    ldr r0, =bss_start
    ldr r1, =bss_end
    mov r2, #0
clean_loop:
    str r2, [r0], #4
    cmp r0, r1
    bne clean_loop
```

首先获取 bss_start 的链接地址, 然后获取 bss_end 的链接地址, 使用 clear_loop 将该部分的内存清 0, bss_start 和 bss_end 的链接地址由 link.lds 决定。

4) 跳转, 代码如下:

```
ldr pc, =main
```

由于 ldr 指令获取的是 main 函数的链接地址, 所以执行 ldr pc, =main 后, 程序就跳转到 0x1000+main 函数的 offset 的地址处了

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

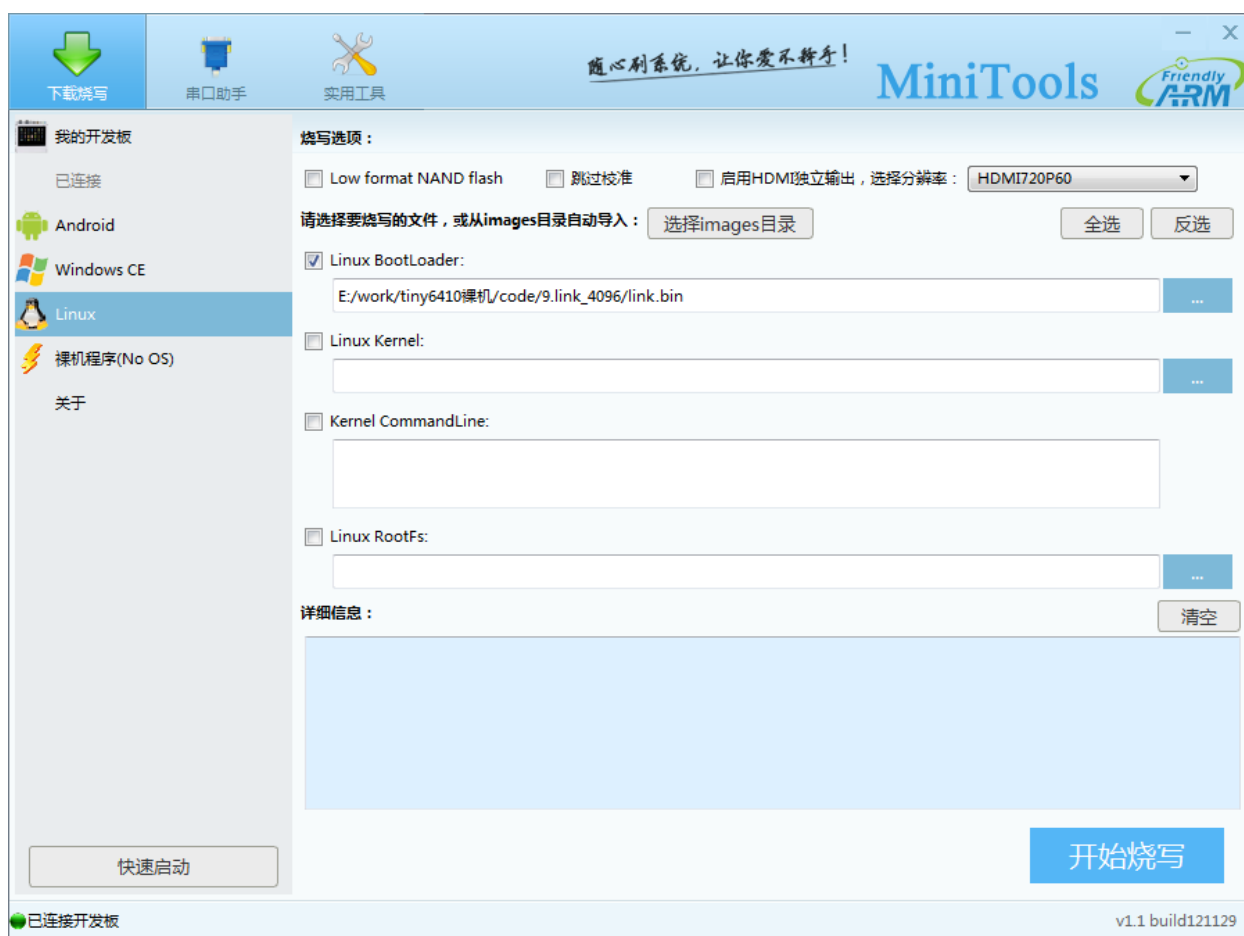
```
# cd 9.link_4096
```

```
# make
```

执行 make 后会生成 link.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址, 其设置方式如下:



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

开发板选择 NAND 启动，然后在 PC 键盘上输入一个字符，则终端会显示该字符在 ASCII 表中的下一字符，例如键盘输入“abcd”，终端上会显示“bcde”，该现象与第八章的代码运行效果一模一样，但是程序的运行过程却有了很大的区别。通过本章的学习，我们已经知道了如何对代码进行重定位，这为我们下一章节将代码重定位到 DRAM 奠定了基础。

第十章 重定位代码到 DRAM

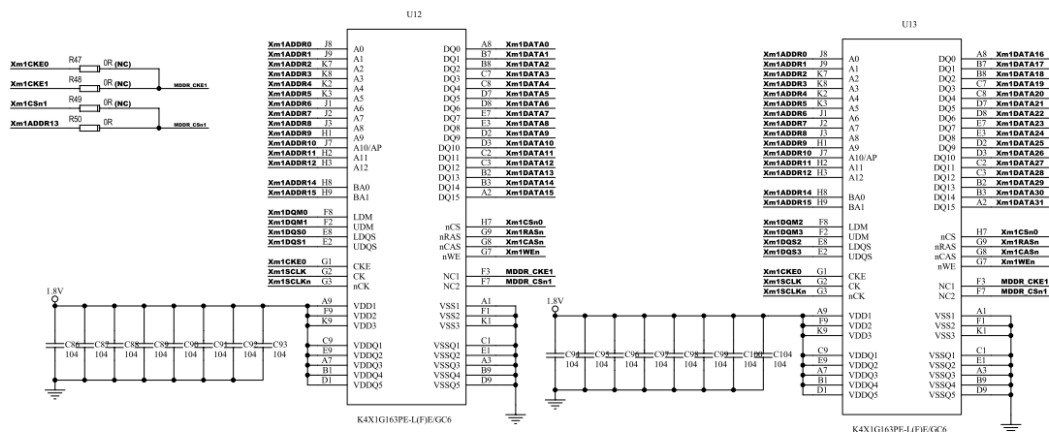
第一节 关于 DRAM

上一章我们讲解了如何对代码进行重定位，但是将代码重定位到 SRAM 中的作用不大。正确的做法是将代码重定位到容量更大的主存中，即 DRAM。S3C6410 的 DRAM 控制器最多只能支持两个同一类型的芯片，每个芯片可分配最多 256 MB 的地址空间。所有芯片在相同的端口共享所有引脚，除了时钟启动信号和片选信号。例如：DDR SDRAM 存储器接口的连接。动态 DDR SDRAM 可连接类似的 DDR SDRAM 内存。

Table 2-2. Device Specific Address Space

Address		Size(MB)	Description	Note
0x0000_0000	0x07FF_FFFF	128MB	Booting Device Region by XOM Setting	Mirrored Region
0x0800_0000	0x0BFF_FFFF	64MB	Internal ROM	
0x0C00_0000	0x0FFF_FFFF	64MB	Stepping Stone (Boot Loader)	
0x1000_0000	0x17FF_FFFF	128MB	SROMC Bank0	
0x1800_0000	0x1FFF_FFFF	128MB	SROMC Bank 1	
0x2000_0000	0x27FF_FFFF	128MB	SROMC Bank 2	
0x2800_0000	0x2FFF_FFFF	128MB	SROMC Bank 3	
0x3000_0000	0x37FF_FFFF	128MB	SROMC Bank 4	
0x3800_0000	0x3FFF_FFFF	128MB	SROMC Bank 5	
0x4000_0000	0x47FF_FFFF	128MB	Reserved	
0x4800_0000	0x4FFF_FFFF	128MB		
0x5000_0000	0x5FFF_FFFF	256MB	DRAM Controller of the Memory Port1	
0x6000_0000	0x6FFF_FFFF	256MB		

查阅 Tiny6410 的原理图：



Tiny6410 DRAM 原理图

Tiny6410 的 256M 的 DRAM 是由 2 片大小为 128M 的 DRAM 芯片组合而成。如果才能使用 DRAM？对应 Tiny6410 而言，我们需要初始化 DRAM 控制器。

第二节 程序相关讲解

完整代码见目录 10. ddr, 该目录下的代码与上一章的代码的差别在于修改了 start.S 和链接脚本 leds.lds, 以及多了一个初始化 DRAM 控制器的 sdram.c。我们首先分析 link.lds。

1. link.lds

修改了链接地址, 将其改为 0x50000000, 程序的正常运行需要我们执行如下两个步骤:

- 1) 初始化 DRAM 控制器, 使以 0x50000000 为起始的 256M 内存可用;
- 2) 将代码重定位到 0x50000000 处;

2. start.S

相比上一章的代码, 本章的 start.S 多了两个步骤:

- 1) 调用 sdram_init 函数初始化内存, 函数的实现位于 sdram.c;
- 2) 重定位, 把 SRAM 上 0 地址开始处的代码拷贝到内存 0x50000000 处。代码如下:

```
adr r0, _start           // r0 = _start 的当前地址
ldr r1, =_start          // r1 = _start 的链接地址
ldr r2, =bss_start       // r2 - r1 = 程序的长度
cmp r0, r1
beq clean_bss

copy_loop:               // 循环拷贝
    ldr r3, [r0], #4
    str r3, [r1], #4
    cmp r1, r2
    bne copy_loop
```

3. sdram.c

S3C6410 芯片手册里已经告诉我们如何初始化 DDR 类型的 DRAM, 主要分为两大步骤:

- 1) 初始化 DRAM 控制器; 具体步骤见下图:

5.4.1 DRAM CONTROLLER INITIALIZATION SEQUENCE

- Program memc_cmd to '3'b100', which makes DRAM Controller enter 'Config' state.
- Write memory timing parameter, chip configuration, and id configuration registers.
- Wait 200us to allow SDRAM power and clock to stabilize. However, when CPU starts working, power and clock would already be stabilized.
- Execute memory initialization sequence.
- Program memc_cmd to '3'b000', which makes DRAM Controller enter 'Ready' state.
- Check memory status field in memc_stat until memory status becomes '2'b01', which means 'Ready'.

- 2) 初始化 DRAM;

5.4.3 DDR/MOBILE DDR SDRAM INITIALIZATION SEQUENCE

- Program mem_cmd in direct_cmd to '2'b10', which makes DRAM Controller issue 'NOP' memory command.
- Program mem_cmd in direct_cmd to '2'b00', which makes DRAM Controller issue 'Prechargeall' memory command.
- Program mem_cmd in direct_cmd to '2'b11', which makes DRAM Controller issue 'Autorefresh' memory command.
- Program mem_cmd in direct_cmd to '2'b11', which makes DRAM Controller issue 'Autorefresh' memory command.
- Program mem_cmd to '2'b10' in direct_cmd, which makes DRAM Controller issue 'MRS' memory command
 - Bank address for EMRS must be set.
- Program mem_cmd to '2'b10' in direct_cmd, which makes DRAM Controller issue 'MRS' memory command.
 - Bank address for MRS must be set.

sdram.c 就是参考了上述相关知识进行了内存的初始化, 只要严格按照上述步骤, 并且参考 sdram 的芯片手册就可以驱动内存。sdram.c 主要进行了下面 4 个步骤的工作:

- 1) 使 dramc 进入“config”状态;
- 2) 初设置 timing parameter, chip configuration, id configuration registers;
- 3) 初始化 sdram;
- 4) 使 dramc 进入“ready”状态;

具体设置可以参考代码, 代码本身已经附有详细的注释。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

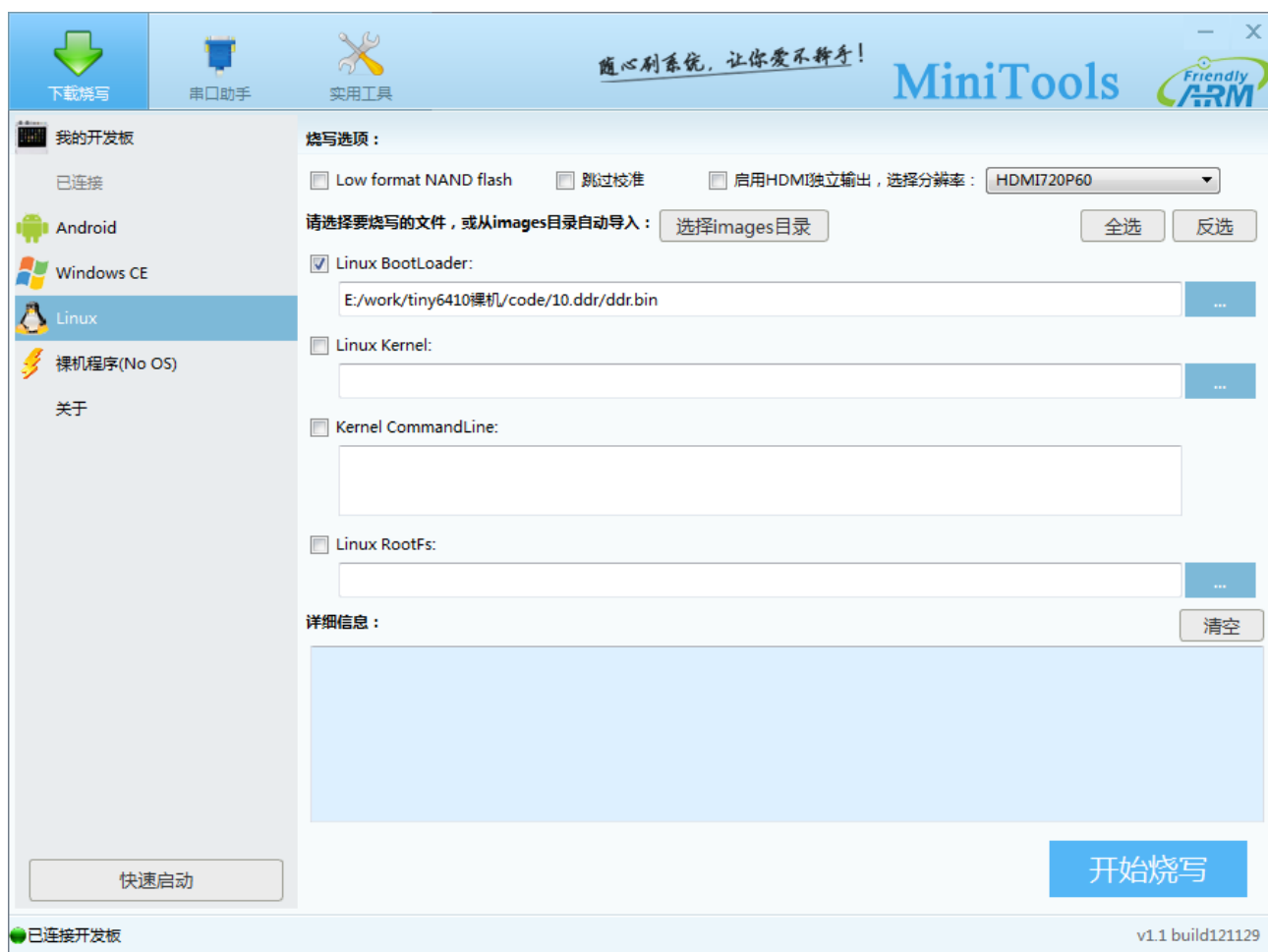
```
# cd 10. ddr
```

```
# make
```

执行 make 后会生成 ddr.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址, 其设置方式如下:



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

开发板选择 NAND 启动，然后在 PC 键盘上输入一个字符，则终端会显示该字符在 ASCII 表中的下一字符，例如键盘输入“abcd”，终端上会显示“bcde”，该现象与第九章的代码运行效果一模一样。下一章将讲解如何从 NAND Flash 中拷贝代码到 DRAM 中。

第十一章 NAND Flash 控制器

第一节 关于 NAND Flash

S3C6410 的 NAND Flash 控制器有如下特点：

- 1) 自动导入模式：复位后，引导代码被送入 8KB 的 STEPPINGSTONE 中，引导代码移动完毕，引导代码将在 STEPPINGSTONE 中执行。在导入期间，NAND FLASH 控制器不支持 ECC 纠正。
- 2) NAND FLASH 控制器 I/F：支持 512 字节和 2KB 页。
- 3) 软件模式：用户可以直接访问 NAND FLASH 控制器。例如这个特性可以用于读/擦/编程 NAND FLASH 存储器。
- 4) 接口：8 位 NAND FLASH 存储器接口总线。
- 5) 硬件 ECC 产生、检测和标志（软件纠正）。
- 6) 支持 SLC 和 MLC 的 NAND FLASH 控制器：1 位 ECC 用于 SLC，4 位 ECC 用于 MLC 的 NAND FLASH。
- 7) 特殊功能寄存器 I/F：支持字节/半字/字数据的访问和 ECC 的数据寄存器，用字来访问其他寄存器。
- 8) STEPPINGSTONE I/F：支持字节/半字/字的访问。
- 9) 8KB 内部 SRAM 缓冲器 STEPPINGSTONE，在 NAND FLASH 引导后可以作为其他用途使用。

本教程中，Tiny6410 的 NAND Flash 类型为 MLC，大小为 2G，型号为 K9GAG08U0E-S。

注：本章的内容是针对 MLC 类型的 NAND Flash，并不适用 SLC 类型的 NAND Flash。

第二节 程序相关讲解

完整代码见目录 11. nand。

1. start.S

相比上一章的 start.S 的，本章的 start.S 有两点不同：

- 1) 在重定位之前调用了 nand_init() 初始化 NAND Flash；
- 2) 重定位是通过调用 copy2ddr() 函数将代码从 SRAM 拷贝到 DRAM 中，copy2ddr() 的具体实现位于 nand.c；

2. nand.c

1> nand_init() 函数

该函数用于初始化 NAND Flash，代码如下：

```
void nand_init(void)
{
    NFCONF_REG = ( (0x2<<12) | (0xf<<8) | (0x7<<4) );
    NFCONT_REG |= (0x3<<0);
}
```



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

Register	Address	R/W	Description	Reset Value
NFCONF	0x70200000	R/W	NAND Flash Configuration register	0xX000100X

NFCONF	Bit	Description	Initial State
Reserved			
MLCClkCtrl	[30]	Clock control for 4-bit ECC & 8-bit ECC engine.(Hidden Spec.) 0: Recommended when system clock is more than 66MHz. 1: Recommended when system clock is less than 66MHz	0
Reserved	[29:26]	Reserved	0000
MsgLength	[25]	Message (Data) length for 4/8 bit ECC 0: 512-byte 1: 24-byte	0
ECCType	[24:23]	This bit indicates what kind of ECC should be used. 00: 1-bit ECC 10: 4-bit ECC 01 : 8-bit ECC Note. Don't confuse the value of 4-bit ECC and 8-bit ECC.	H/W Set (CfgBootEcc)
Reserved	[22:15]	Reserved	000000000
TACLS	[14:12]	CLE & ALE duration setting value (0~7) Duration = HCLK x TACLS	001
Reserved	[11]	Reserved	0
TWRPH0	[10:8]	TWRPH0 duration setting value (0~7) Duration = HCLK x (TWRPH0 + 1)	000
Reserved	[7]	Reserved	0
TWRPH1	[6:4]	TWRPH1 duration setting value (0~7) Duration = HCLK x (TWRPH1 + 1)	000
Reserved			
Reserved	[2]	Reserved. Must be written 1	1
Reserved			
Reserved	[0]	Reserved. Must be written 0.	0

TWRPH1/TWRPH0/TACLS 是关于访问时序的设置，需对照 NAND Flash 芯片手册设置，这里不再详细解释，分别取 TWRPH1=0x7，TWRPH0=0xF，TACLS=0x2；
其余位均使用默认值；

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

NFCONT	Bit	Description	Initial State
EnbIllegalAccINT	[10]	Illegal access interrupt control 0: Disable interrupt 1: Enable interrupt Illegal access interrupt will occurs when CPU tries to program or erase locking area (the area setting in NFSBLK (0x70200020) to NFEBLK (0x70200024).	0
EnbRnBINT	[9]	RnB status input signal transition interrupt control 0: Disable RnB interrupt 1: Enable RnB interrupt	0
RnB_TransMode	[8]	RnB transition detection configuration 0: Detect rising edge 1: Detect falling edge	0
MainECCLock	[7]	Lock Main area ECC generation 0: Unlock Main area ECC 1: Lock Main area ECC Main area ECC status register is NFMECC0/1(0x70200034/38),	1
SpareECCLock	[6]	Lock Spare area ECC generation. 0: Unlock Spare ECC 1: Lock Spare ECC Spare area ECC status register is NFSECC(0x7020003C),	1
InitMECC	[5]	1: Initialize main area ECC decoder/encoder (write-only) Caution : In case of 8bit ECC, you must set this bit carefully. If you set this bit before completing current encoding/decoding, it cause some trouble to 8bit ECC module. If you want to stop current work and start encoding/decoding for new data, you must set 8bitStop(NFCONT[11]) before this bit.	0
InitSECC	[4]	1: Initialize spare area ECC decoder/encoder (write-only)	0
Reserved	[3]	Reserved	0
Reg_nCE1	[2]	NAND Flash Memory Xm0CSn3 signal control 0: Force Xm0CSn3 to low(Enable chip select) 1: Force Xm0CSn3 to High(Disable chip select) Note: Even Reg_nCE1 and Reg_nCE0 are set to zero simultaneously, only one of them is asserted.	1
Reg_nCE0	[1]	NAND Flash Memory Xm0CSn2 signal control 0: Force Xm0CSn2 to low(Enable chip select) 1: Force Xm0CSn2 to High(Disable chip select) Note: This value is only valid while MODE bit is 1	1
MODE	[0]	NAND Flash controller operating mode 0: NAND Flash Controller Disable (Don't work) 1: NAND Flash Controller Enable	0

MODE = 1, 使能 NAND Flash;

Reg_nCE0 = 1, 暂时不选中 NAND Flash;

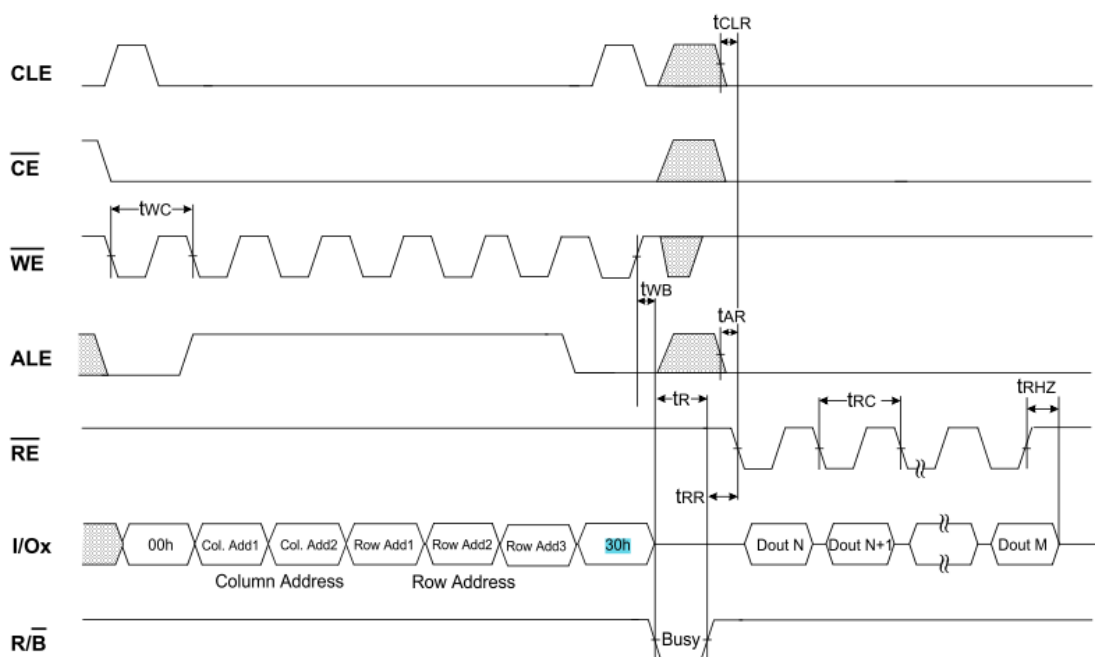
其余位使用默认值;

2> nand11_read_page() 函数

该函数用于从 NAND Flash 中读一页的数据, 对于 Tiny6410 的 NAND Flash, 一页即 2048Byte。

参考 NAND Flash 的芯片手册:

4.7 Read Operation



读一页 NAND Flash 的操作如下：

- 1) 发片选，即设置 NFCONT 的 bit1 为 0；
- 2) 发读命令：0x00，即往 NFCMD 写 0x00；
- 3) 发 5 个周期的地址，即往 NFADDR 寄存器写入地址，因为 NFADDR 一次只能接受 8bit 的数据，所以分 5 次来写；
- 4) 发读命令：0x30，即往 NFCMD 写 0x30；
- 5) 连续读 2048 个字节，即连续读 NFDATA 2048 次；

3> copy2ddr() 函数

该函数负责调用 nand11_read_page() 进行读页操作，知道 bin 文件完全地从 NAND Flash 中拷贝到 DRAM 中。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

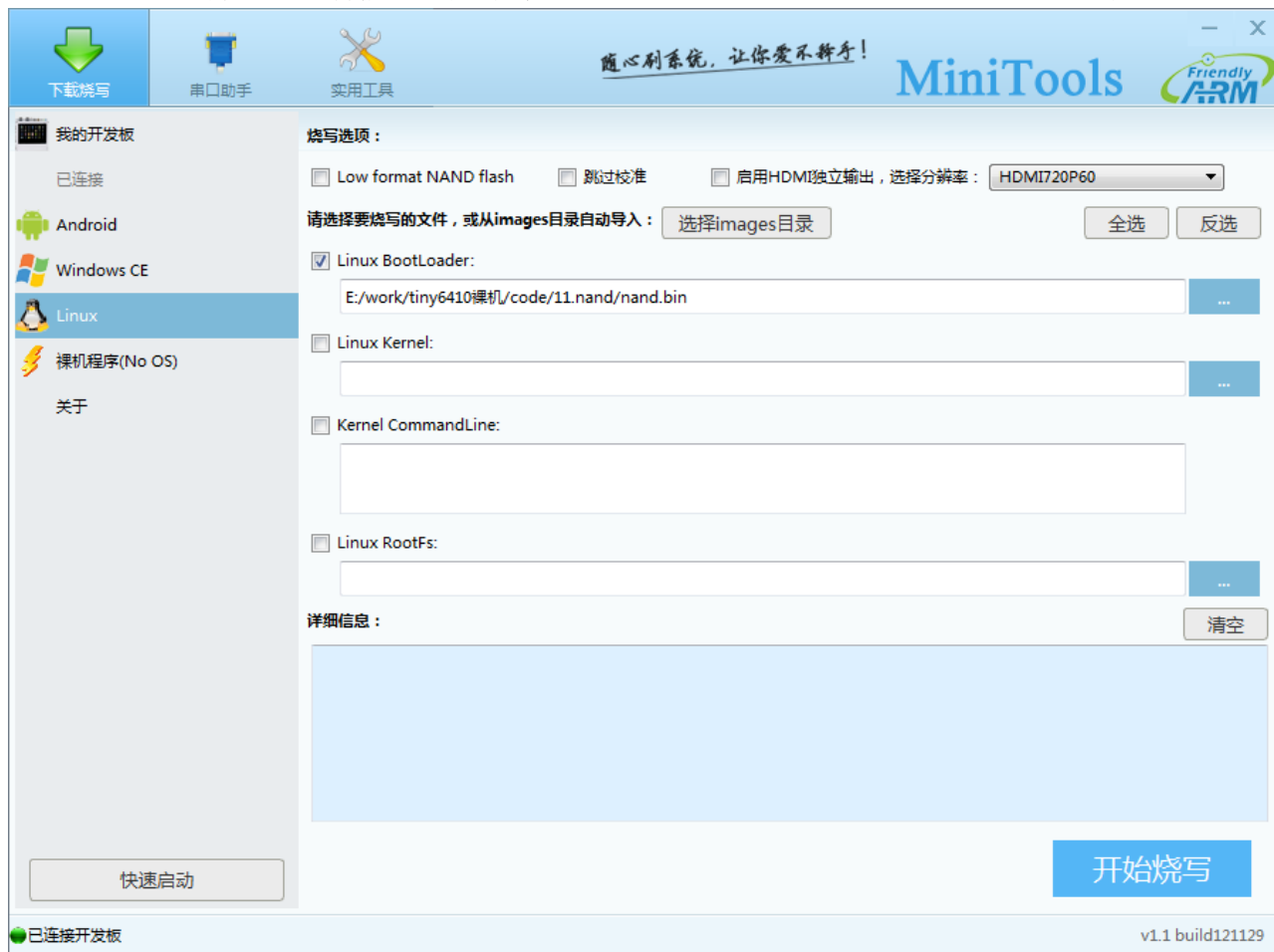
```
# cd 11.nand
```

```
# make
```

执行 make 后会生成 nand.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

开发板选择 NAND 启动，然后在 PC 键盘上输入一个字符，则终端会显示该字符在 ASCII 表中的下一字符，例如键盘输入“abcd”，终端上会显示“bcde”，该现象与第十章的代码运行效果一模一样。

第十二章 内存管理单元 MMU

第一节 关于 MMU

内存管理单元(Memory Management Unit)简称 MMU, 它负责虚拟地址到物理地址的映射, 并提供硬件机制的内存访问权限检查。现代的多用户多进程操作系统通过 MMU 使得各个用户进程都拥有自己独立的地址空间: 地址映射功能使得各个进程拥有“看起来”一样的地址空间, 而内存访问权限的检查可以保护每个进程所用的内存不会被其他进程破坏。这里不会详细讲解地址映射的知识, 我们直接来看程序, 程序涉及了什么知识我们就讲解什么知识, 避免陷入各种复杂的理论知识里。

第二节 程序相关讲解

本章涉及的代码有两套, 包括 12. leds_mmu_s (使用汇编初始化 MMU) 和 13. leds_mmu_c (使用 C 语言初始化 MMU), 两者的本质是一样的, 初始化 MMU 所达到的效果也一样, 想巩固汇编指令的话, 可以阅读 12. leds_mmu_s 里的相关代码。由于 C 语言编写的代码思路更清晰, 更便于阅读和理解, 下面将以 13. leds_mmu_c 里的代码为参考进行讲解。

1. mmu.lds

```
SECTIONS {  
    . = 0xc0000000;  
  
    .text : {  
        start.o  
        sdram.o  
        mmu.o  
        clock.o  
        * (.text)  
    }  
    .rodata : {  
        * (.rodata)  
    }  
    ... // 省略  
}
```

链接地址是 0xc0000000, 这是个虚拟地址, 从后面的代码讲解可知, 虚拟地址 0xc0000000 对应物理地址 0x50000000。

2. start.S

本章的 start.S 有 3 个需要注意的地方:

调用 C 函数 `copy2ddr()` 进行重定位;
调用了 mmu 初始化函数 `mmu_init()`;
`copy2ddr()` 的定义位于 `sdram.c` 中, 代码如下:

```
void copy2ddr(void)
{
    unsigned char *dest = (unsigned char *)0x50000000;
    unsigned char *src = (unsigned char *)0;
    int i = 0;

    while (i < 8*1024)
    {
        dest[i] = src[i];
        i++;
    }
}
```

这段代码的作用就是将我们的代码从 0 地址拷贝到 0x50000000。

3. mmu.c

该文件的核心就是 `mmu_init()` 函数, 它的作用就是设置 mmu 和启动 mmu, 具体的步骤如下:

1) 设置页表, 代码如下:

```
table[0x100] = 0x7f000000 | MMU_SECDISC_NCNB;
table[0] = 0 | MMU_SECDISC_WB;
table[0xc00] = 0x50000000 | MMU_SECDISC_WB;
```

什么是页表? CPU 发出虚拟地址后, MMU 最终是如何找到对应的物理地址的, 靠的就是页表。页表是有一个个页表项组成的, 分为一级页表和二级页表, 我们的程序里使用的是一级页表, 即一个页表项所映射的地址空间大小为 1M。另外, 页表存放于内存, 负责存放页表的内存的起始地址即页表基地址。

CPU 发出虚拟地址 VA, 然后 MMU 根据“页表项地址 = 页表基地址+VA>>20”找到对应的页表项, 最后再由页表项的高 12bit 和 VA 的低 20bit 组成 32bit 的物理地址, 这样就完成了从虚拟地址到物理地址的映射。

在代码中, 我们根据上述规律构造了 3 个页表项, 依次实现了下面 3 个映射:

物理地址 0x7F000000 起的 1M 空间=>虚拟地址 0x1000000 起的 1M 空间

物理地址 0x0 起的 1M 空间=>虚拟地址 0x0 起的 1M 空间

物理地址 0xc0000000 起的 1M 空间=>虚拟地址 0x50000000 起的 1M 空间

其中, 1) 是为了映射寄存器地址空间, 在 `main.c` 中我们通过操作虚拟地址 0x1000000 达到点灯的目的。2) 是为了保证 MMU 使能前后的代码所在的地址空间一致。(4) 由于我们的程序是被重定位到 0x50000000 上, 但是程序的链接地址是 0xc0000000, 所以需要将 0x50000000 开始的 1M 内存映射到 0xc0000000。

2) 启动 MMU, 参考如下步骤:



使无效 ICaches 和 DCaches;

使无效指令、数据 TLB;

TLB 即 Translation Lookaside Buffers, 位于 MMU 和内存之后, 是用来存储近期用到的页表项, 以避免每次地址转换时都到主存取寻找。

设置页表基地址;

设置 domain, 用于控制访问权限;

使能 ICaches 和 Dcaches;

使能 MMU;

4. main.c

main() 函数的代码跟前面的点灯程序的代码差不多, 唯一的驱动在于对寄存器的操作使用的是虚拟地址:

```
volatile unsigned long *gpkcon0 = (volatile unsigned long *)0x10008800;  
volatile unsigned long *gpkdat = (volatile unsigned long *)0x10008808;
```

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

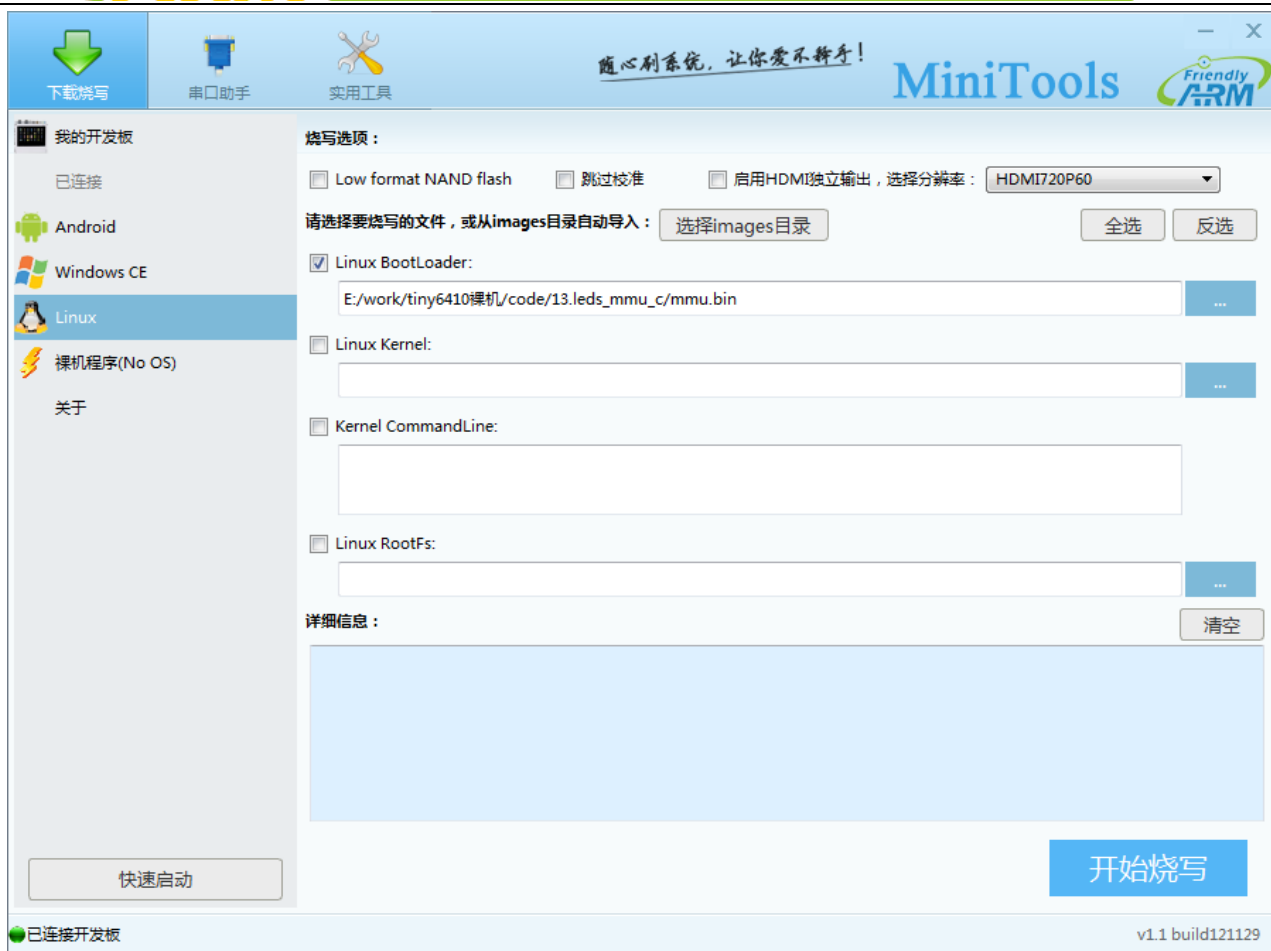
```
# cd 13.leds_mmu_c
```

```
# make
```

执行 make 后会生成 mmu.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址, 其设置方式如下:



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

LED 又欢快地闪烁起来了，没什么不同的，只不过这次我们使用的是虚拟地址。下一章节我们将会为我们的代码添加 printf 和 scanf 功能。

第十三章 移植 printf 和 scanf 功能

第一节 移植的途径

对于如何移植 printf 和 scanf，我们有许多选择：

- 1) 移植 linux 的 printf 功能，版本越新越难移植，但是功能也越强大；
- 2) 移植 uboot 的 printf 和 scanf 功能，实际 uboot 也是从 linux 内核中移植而来的；
- 3) 完全自己编写，但是功能比较弱。

在保证整个裸机其他代码部分没有任何问题，上述三种方法都是可行的。下面我们只是直接利用网友移植好的相关文件，为我们的裸机代码增加上该部分功能。

第二节 移植步骤

本次的移植是在代码 11.nand 的基础上进行修改，首先将代码 11.nand 复制一份，并改名为 14.uart_stdio，并进行如下操作：

- 1) 解压 printf.rar 到 14.uart_stdio 目录，解压成功后会多出 include 和 lib 两个目录，其中 include 放的是相关头文件，lib 放的是 printf 和 scanf 相关的代码。
- 2) 修改 14.uart_stdio 目录下的 makefile，将 lib 目录下的代码编译链接成 lib.a，然后将 lib.a 编译进 bin 中，具体修改见源码。
- 3) 编写 main 函数进行测试。

第三节 程序相关讲解

完整代码见目录 14.uart_stdio。

1. /lib/printf.c

1) printf() 的定义如下：

```
int printf(const char *fmt, ...)
{
    int i;
    int len;
    va_list args;                // va_list 即 char *

    va_start(args, fmt);
```

```
len = vsprintf(g_pcOutBuf, fmt, args); // 内部使用了 va_arg()
va_end(args);
for (i = 0; i < strlen(g_pcOutBuf); i++)
{
    putc(g_pcOutBuf[i]);
}
return len;
}
```

2> 首先, printf 函数是个变参函数, 什么是变参函数:

可变参数函数的原型声明为 `type VAFunction(type arg1, type arg2, ...)`; 参数可以分为两部分: 个数确定的固定参数和个数可变的可选参数。函数至少需要一个固定参数, 固定参数的声明和普通函数一样; 可选参数由于个数不确定, 声明时用“...”表示。固定参数和可选参数共同构成一个函数的参数列表。

3> 再来看 printf 函数, 该函数涉及了 3 个十分重要的宏:

宏1: `#define va_start(ap, A) (void) ((ap) = (((char *) &(A)) + (_bnd (A, _AUPBND))))`

宏2: `#define va_arg(ap, T) (*(T *)(((ap) += (_bnd (T, _AUPBND))) - (_bnd (T, _ADNBND))))`

宏3: `#define va_end(ap) (void) 0`

在这些宏中, va 就是 variable argument(可变参数)的意思;

ap:是指向可变参数表的指针;

A:指可变参数表的前一个固定参数;

T:可变参数的类型。

va_list 也是一个宏, 其定义为 `typedef char * va_list`, 实质上是一 char 型指针。

4> 下面逐个来分析这三个宏的作用:

(1) va_start 宏

✚ 作用:

根据 v 取得可变参数表的首指针并赋值给 ap, 方法: 最后一个固定参数 A 的地址 + 第一个变参对 A 的偏移地址, 然后赋值给 ap, 这样 ap 就是可变参数表的首地址。

✚ 举例:

如果有变参函数的声明是 `void va_test(char a, char b, char c, ...)`, 则它的固定参数依次是 a, b, c, 最后一个固定参数为 c, 因此就是 `va_start(ap, c)`。

(2) va_arg 宏

✚ 作用:

指取出当前 ap 所指的可变参数并将 ap 指针指向下一可变参数。

(3) va_end 宏

✚ 作用:

结束可变参数的获取。va_end (list) 实际上被定义为空, 没有任何真实对应的代码, 用于代码对称, 与 va_start 对应。

5> 怎样得到可变参数个数?归纳起来有三种办法:

- (1) 函数的第一个参数, 指定后续的参数个数, 如 func(int num,...);
- (2) 根据隐含参数, 判断参数个数, 如 printf 系列的, 通过字符串中%的个数判断;
- (3) 特殊情况下(如参数都是不大于 0xFFFF 的 int), 可以一直向低处访问堆栈, 直到返回地址。

有了上述知识我们就可以看懂 printf() 函数的内容了, 首先 va_start(args, fmt); 会将可变参数的首地址保存在 args 中, 然后调用 vsprintf(g_pcOutBuf, fmt, args) 进行处理, 在 vsprintf() 中, 会调用 va_arg() 逐个的取出变参, 然后进行解析。如果是普通字符则无须转换, 直接保存在 g_pcOutBuf; 如果是字符串, 则从可变参数表中拿到指向字符串的指针, 将字符串的内容拷贝到 g_pcOutBuf; 如果是数字, 则调用 number 函数进行处理, 并把解析的结果存放在 g_pcOutBuf。所有, 最后只调用 putc 函数把 g_pcOutBuf 里的字符一个个的打印出来就可以了。另外, scanf 函数的原理和 printf 类似, 这里不再进行解释

2. main.c

完整代码如下:

```
int main()
{
    char c;
    int a, b;
    uart_init();
    printf("hello, world\r\n");
    while (1)
    {
        printf("please enter two number: \r\n");
        scanf("%d %d", &a, &b);
        printf("\r\n");
        printf("the sum is: %d\r\n", a+b);
    }
    return 0;
}
```

首先会打印 “hello, world”, 然后用终端接收两个数字, 最后输出它们的和。

第四节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

地址: 广州市天河区龙口西路龙苑大厦A1栋1705 网址: <http://www.arm9.net>

电话: +86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真: +86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

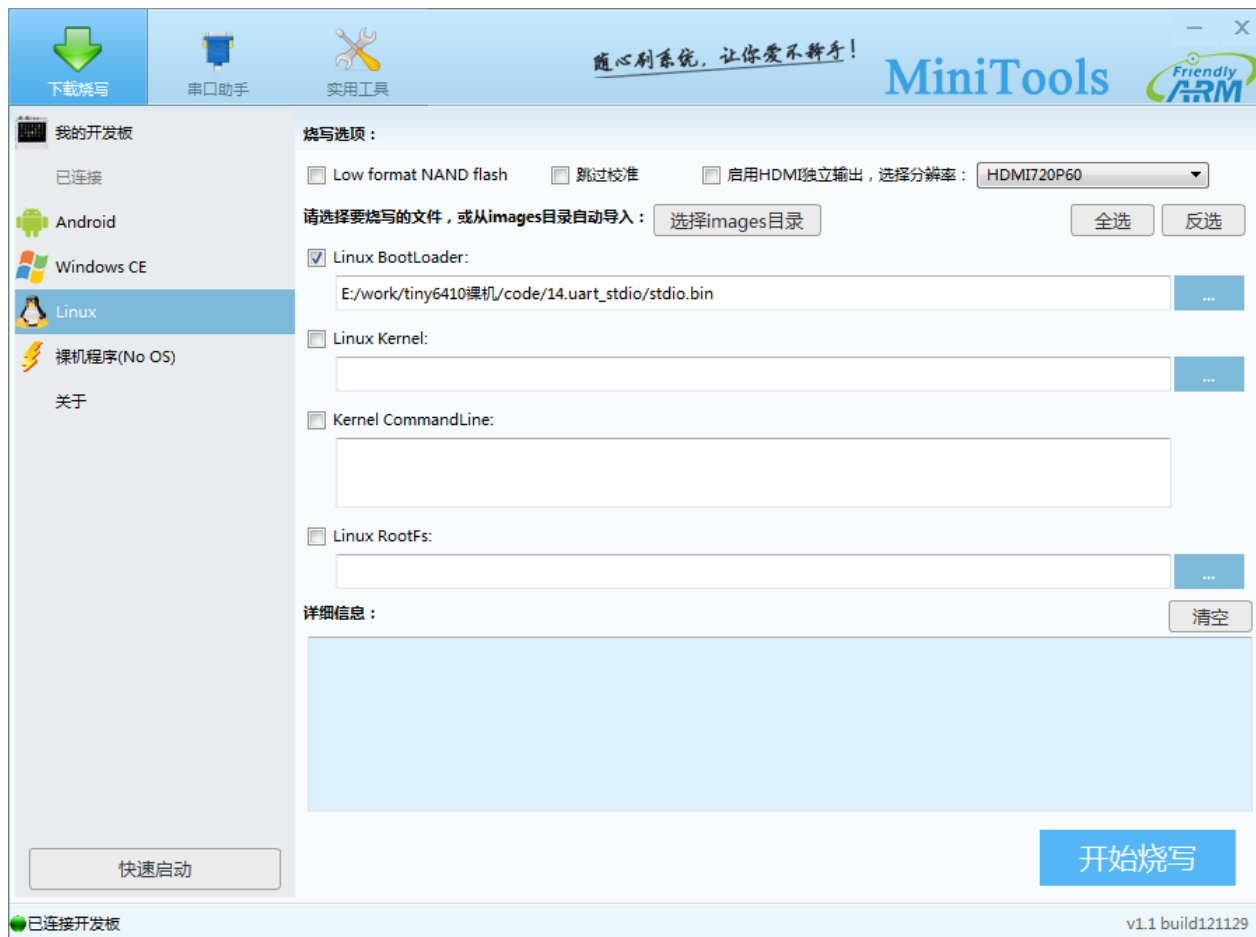
```
# cd 14.uart_stdio
```

```
# make
```

执行 make 后会生成 stdio.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第五节 实验现象

连接好串口终端后，开发板选择 NAND 启动。首先打印 hello, world，然后提醒你输入两个数字，成功输入两个数字后，会打印它们的和。效果图如下：



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

```
hello, world
please enter two number:
1 2
the sum is: 3
please enter two number:
-4 5
the sum is: 1
please enter two number:
```

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

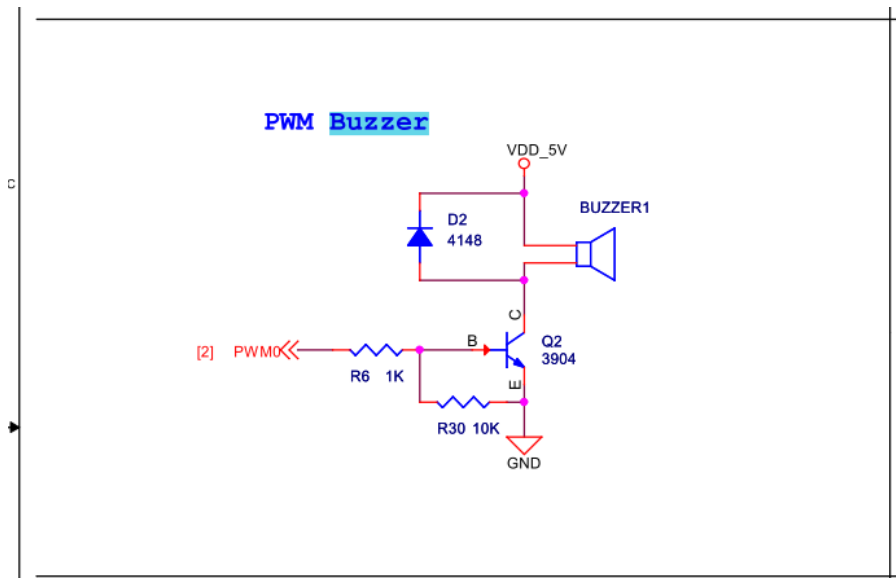
电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第十四章 控制蜂鸣器

第一节 查阅原理图

Tiny6410 带有一个蜂鸣器，十分吵闹，本章将学习如何控制蜂鸣器。首先查阅原理图：



相关引脚图：

XpwmTOUT1/GPF15	D23	PWM1
XpwmTOUT0/XCLKOUT/GPF14	H16	PWM0
XpwmECLK/GPF13	C23	USB_PWR

第二节 程序相关讲解

完整代码见目录 15. buzzer。蜂鸣器的操作十分简单，原理跟操作 LED 一样。

1. buzzer.c

完整代码如下：

```
#define GPFCON (*(volatile unsigned int *)0x7F0080A0)
#define GPFDAT (*(volatile unsigned int *)0x7F0080A4)
void buzzer_on()
{
    GPFDAT |= 1<<14;
}

void buzzer_off()
```

```
{
    GPFDAT &= ~(1<<14);
}
void buzzer_init(void)
{
    GPFCON |= 1<<28;
    GPFCON &= ~(1<<29);
}
```

函数 buzzer_init() 配置 GPIO 引脚，使 GPF14 用于输入功能；

函数 buzzer_on() 使 GPF14 输出 0，蜂鸣器响；

函数 buzzer_off() 使 GPF14 输出 1，蜂鸣器不响；

2. main.c

在 main.c 中，首先会调用 buzzer_init() 来初始化蜂鸣器，然后打印出蜂鸣器的控制菜单并等待用户输入选择，输入 n 则调用 buzzer_on() 打开蜂鸣器，输入 f 则调用 buzzer_off 关闭蜂鸣器。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

```
# cd 15.buzzer
```

```
# make
```

执行 make 后会生成 buzzer.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：

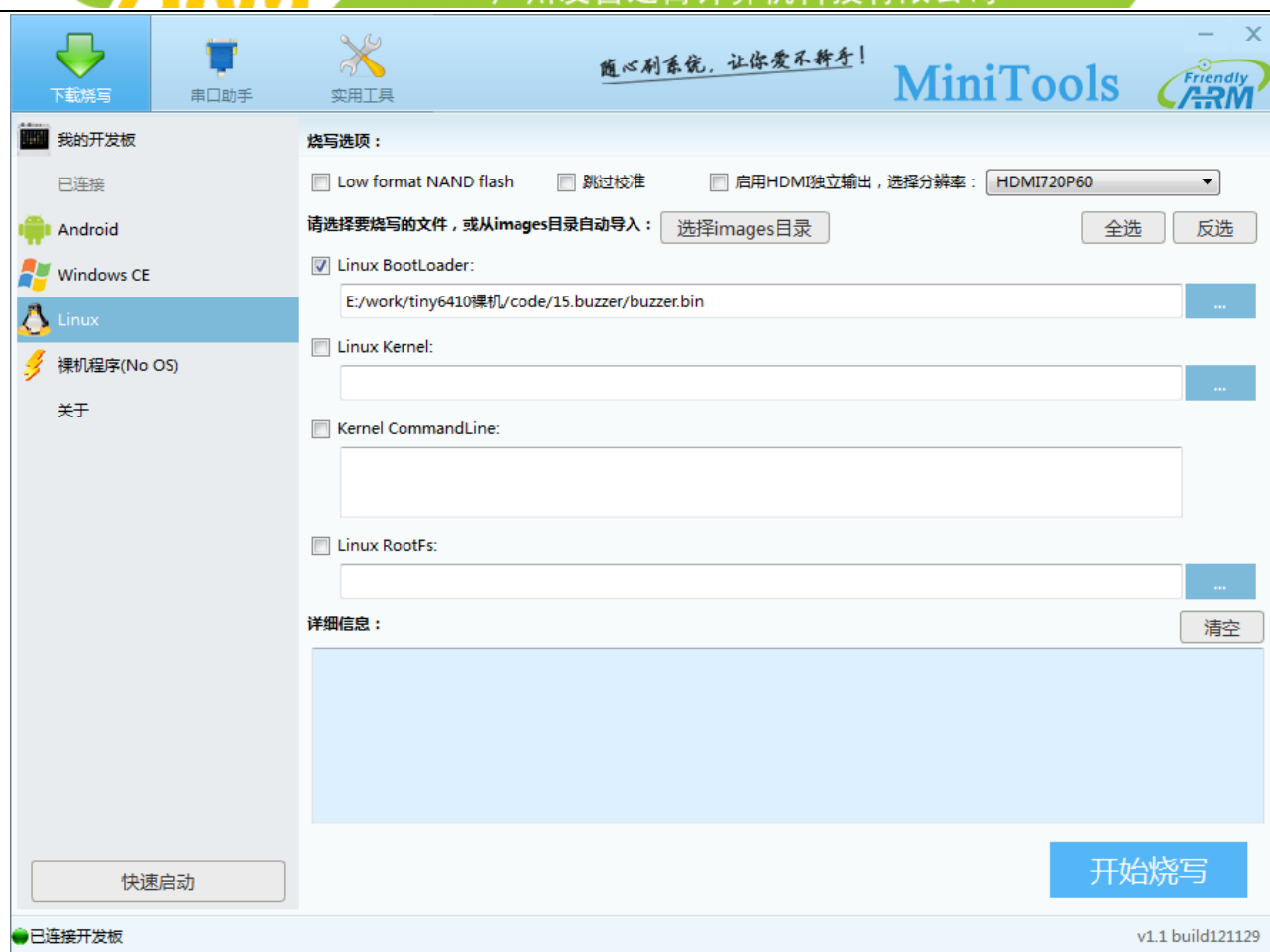


追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

地址：广州市天河区龙口西路龙苑大厦A1栋1705

网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。首先会打印蜂鸣器的操作菜单，输入 n 打开蜂鸣器，输入 f 关闭蜂鸣器，测试效果如下图：

```
#####Buzzer Test#####
[n]buzzer on
[f]buzzer off
Enter your choice:n

#####Buzzer Test#####
[n]buzzer on
[f]buzzer off
Enter your choice:f
```

第十五章 S3C6410 中断体系

第一节 关于 S3C6410 的中断体系

SS3C6410X 内的中断控制器由 2 个 VIC(矢量中断控制器, ARM PrimeCell PL192)和 2 个 TZIC(TrustZone 中断控制器, sp890)组成。两个矢量中断控制器和两个 TrustZone 中断控制器链接在一起支持 64 位中断源。

S3C6410X 内的矢量中断控制器的性能如下:

- 1) 每个 VIC 支持 32 位的矢量 IRP 中断;
- 2) 支持固定硬件中断优先级和可编程中断优先级;
- 3) 支持硬件中断优先级屏蔽和可编程中断优先级屏蔽;
- 4) 产生 IRQ 和 FIQ 中断;
- 5) 产生软件中断;
- 6) raw 中断状态;
- 7) 中断请求状态;
- 8) 支持限制访问的特权模式;

第二节 程序相关讲解

完整代码见目录 17. irq。

1. start.S

与中断相关的地方有:

- 1) 程序一开始时设置了中断向量表。当发生各种异常时, PC 会自动跳转到相应的异常向量; S3C6410 的异常向量表的起始地址是 0;
- 2) 调用 irq_init() 函数进行中断相关的初始化;
- 3) 开中断, 设置 CPSR 寄存器, 允许中断发生; 代码如下:

```
mov r0, #0x53
```

```
msr CPSR_cxsf, r0
```

- 4) 中断处理, 代码如下:

```
irq:
```

```
    ldr sp, =0x54000000           // 设置中断模式的栈
```

```
    sub lr, lr, #4
```

```
    stmdb sp!, {r0-r12, lr}       // 保存现场
```

```
    bl do_irq                     // 处理异常
```

```
    ldmia sp!, {r0-r12, pc}^      // 恢复现场
```

当发生 IRQ 中断异常时, 会根据中断向量表里的设置, 跳转到该部分代码设置中断模式下的栈, 然后保存现场, 再调用中断处理函数 do_irq(), 处理完中断后再恢复现场。do_irq() 的定义位于 irq.c, 后面会解释。

2. irq.c

中断初始化函数 `irq_init()`，代码如下：

```
void irq_init(void)
{
    GPNCON &= ~(0xffff);
    GPNCON |= 0xaaa;

    EINTOCON0 &= ~(0xffff);
    EINTOCON0 |= 0x777;

    EINTOMASK &= ~(0x3f);

    VIC0INTENABLE |= (0x3);
}
```

该函数初始化了 KEY1~6 的中断，主要步骤就是：

- 1) 配置引脚用于中断功能；
- 2) 设置中断的触发方式；
- 3) 设置禁止屏蔽中断；
- 4) 在中断控制器里使能中断；

中断处理函数 `do_irq()`，代码如下：

```
void do_irq(void)
{
    int i = 0;
    for (i = 0; i < 6; i++)
    {
        if (EINTOPEND & (1<<i))
        {
            if (GPNDAT & (1<<i))
            {
                printf("K%d released\n\r", i+1);
            }
            else
            {
                printf("K%d pressed\n\r", i+1);
            }
        }
    }
    EINTOPEND = 0x3f;
```



```
VIC0ADDRESS = 0;
```

```
}
```

主要步骤如下:

- 1) 判断是哪个按键发生了中断;
- 2) 在串口终端打印信息;
- 3) 清中断, 这样 CPU 才不会认为中断又发生了;

3. main.c

与前面的章节无异, 就是计算两个数的和。当没中断发生时, 就执行 main() 里的代码, 当发生中断时, 则跳去执行中断处理代码, 中断处理完毕后, 则继续执行 main() 里的代码;

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

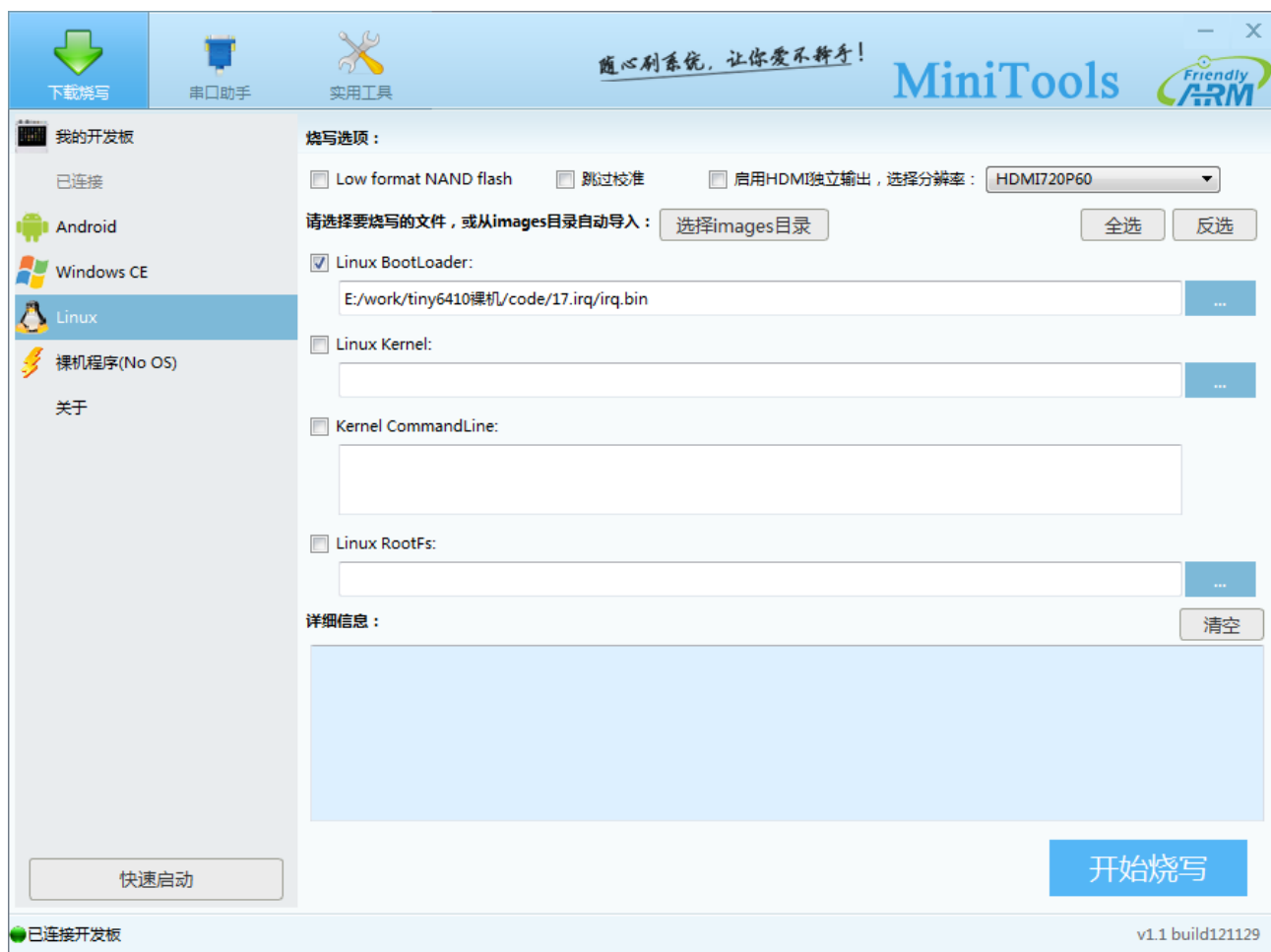
```
# cd 15.buzzer
```

```
# make
```

执行 make 后会生成 buzzer.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址, 其设置方式如下:



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。当未按下任何按键时，程序和前面的章节一样正常地计算两个数的和，当按下 KEY1~6 任意一个按键时，则会进行中断处理，效果如下：

```
please enter two number:
1 2
the sum of 1 + 2 is: 3
please enter two number:
K1 pressed
K1 pressed
K2 pressed
K2 released
K3 pressed
K3 released
```

第十六章 PWM 定时器

第一节 S3C6410 的 PWM 定时器

S3C6410 由五个 32 位定时器组成。这些计时器用来产生内部中断到 ARM 子系统。此外，定时器 0, 1, 2 和 3 包含一个 PWM 功能（脉宽调制），它可以驱动外部的 I/O 信号。定时器 0 上的 PWM 能够产生一个可选的死区发生器。它可能被用来支持大量的通用装置。定时器 4 只是一个没有输出引脚的内部定时器。相关的结构图如下：

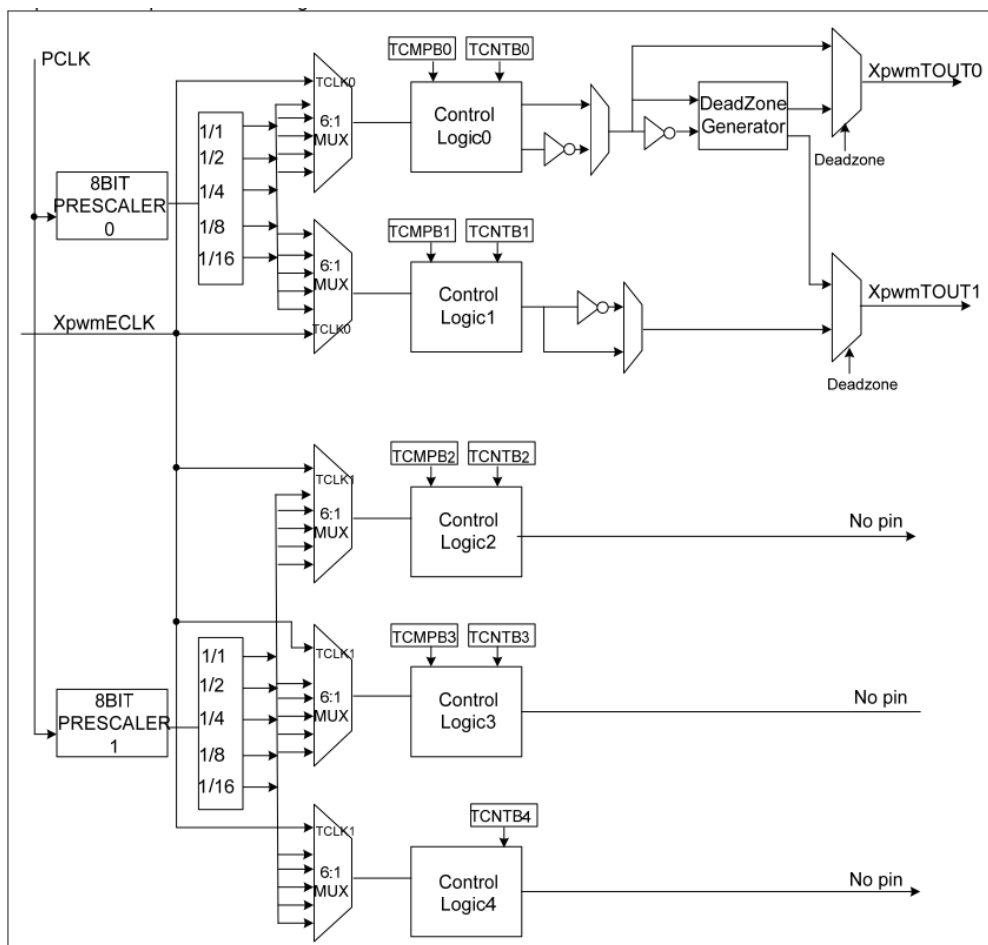


Figure 32-1. PWMTIMER Clock Tree Diagram



第二节 程序相关讲解

完整代码见目录 19.timer。

1. main.c

需要注意两点：

- 1) 调用 timer_init() 初始化 Timer0;
- 2) 死循环，不断地打印数字；

2. timer.c

1> 中断初始化函数 irq_init()

在中断控制器里使能 Timer0 中断，通过往 VICINTENABLE 的 bit23 置 1 即可。

2> Timer0 初始化函数 timer_init()

主要包括 4 个步骤：

1) 设置分频

首先设置预分频系数为 65，相关寄存器为 TCFG0, 如下：

TCFG0	Bit	R/W	Description	Initial State
Reserved	[31:24]	R	Reserved Bits	0x00
Dead zone length	[23:16]	R/W	Dead zone length	0x00
Prescaler 1	[15:8]	R/W	Prescaler 1 value for Timer 2, 3 and 4	0x01
Prescaler 0	[7:0]	R/W	Prescaler 0 value for timer 0 & 1	0x01

然后设置分配比为 16 分频，相关寄存器为 TCFG1, 如下：

TCFG1	Bit	R/W	Description	Initial State
Reserved	[31:24]	R	Reserved Bits	0x00
DMA mode	[23:20]	R/W	Select DMA Request Channel Select Bit 0000: No select 0001: INT0 0010: INT1 0011: INT2 0100: INT3 0101: INT4 0110: No select 0111: No select	0x0
Divider MUX4	[19:16]	R/W	Select Mux input for PWM Timer 4 0000: 1/1 0001: 1/2 0010: 1/4 0011: 1/8 0100: 1/16 0101: External TCLK1 0110: External TCLK1 0111: External TCLK1	0x0
Divider MUX3	[15:12]	R/W	Select Mux input for PWM Timer 3 0000: 1/1 0001: 1/2 0010: 1/4 0011: 1/8 0100: 1/16 0101: External TCLK1 0110: External TCLK1 0111: External TCLK1	0x0
Divider MUX2	[11:8]	R/W	Select Mux input for PWM Timer 2 0000: 1/1 0001: 1/2 0010: 1/4 0011: 1/8 0100: 1/16 0101: External TCLK1 0110: External TCLK1 0111: External TCLK1	0x0
Divider MUX1	[7:4]	R/W	Select Mux input for PWM Timer 1	0x0

经过上面的设置之后，就能确定 Timer0 的输入时钟了，计算方式如下：

Timer Input Clock Frequency = PCLK / ({prescaler value + 1}) / {divider value} = 66MHz / (65+1) / 16 = 62500Hz

2) 设置计数

设置寄存器 TCNTB0=62500 和 TCMPB0=0，启动 Timer0 后，TCNTB0 会逐渐减一，直到为 0 时就产生一次中断，即 1 秒产生一次 Timer0 中断。

1.5.1.4 Timer0 Counter Register (TCNTB0, R/W, Address = 0xE250_000C)

TCNTB0	Bit	Description	Initial State
Timer 0 Count Buffer	[31:0]	Timer 0 Count Buffer Register	0x0000_0000

1.5.1.5 Timer0 Compare Register (TCMPB0, R/W, Address = 0xE250_0010)

TCMPB0	Bit	Description	Initial State
Timer 0 Compare Buffer	[31:0]	Timer 0 Compare Buffer Register	0x0000_0000

3) 启动 Timer0

设置寄存器 TCON，先设置手动更新位，然后清除手动更新位，使用自动装载，最后启动 timer0

4) 使能 timer0 中断

设置寄存器 TINT_CSTAT，使能 Timer0 中断。



TINT_CSTAT	Bit	R/W	Description	Initial State
Reserved	[31:10]	R	Reserved Bits	0x00000
Timer 4 Interrupt Status	[9]	R/W	Timer 4 Interrupt Status Bit. Clears by writing '1' on this bit.	0x0
Timer 3 Interrupt Status	[8]	R/W	Timer 3 Interrupt Status Bit. Clears by writing '1' on this bit.	0x0
Timer 2 Interrupt Status	[7]	R/W	Timer 2 Interrupt Status Bit. Clears by writing '1' on this bit.	0x0
Timer 1 Interrupt Status	[6]	R/W	Timer 1 Interrupt Status Bit. Clears by writing '1' on this bit.	0x0
Timer 0 Interrupt Status	[5]	R/W	Timer 0 Interrupt Status Bit. Clears by writing '1' on this bit.	0x0
Timer 4 interrupt Enable	[4]	R/W	Timer 4 Interrupt Enable. 1 – Enabled 0 – Disabled	0x0
Timer 3 interrupt Enable	[3]	R/W	Timer 3 Interrupt Enable. 1 – Enabled 0 – Disabled	0x0
Timer 2 interrupt Enable	[2]	R/W	Timer 2 Interrupt Enable. 1 – Enabled 0 – Disabled	0x0
Timer 1 interrupt Enable	[1]	R/W	Timer 1 Interrupt Enable. 1 – Enabled 0 – Disabled	0x0
Timer 0 interrupt Enable	[0]	R/W	Timer 0 Interrupt Enable. 1 – Enabled 0 – Disabled	0x0

3> 中断处理函数 do_irq()

共 2 个步骤:

- 1) 打印信息;
- 2) 清 Timer0 的中断状态寄存器 TINT_CSTAT;

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令:

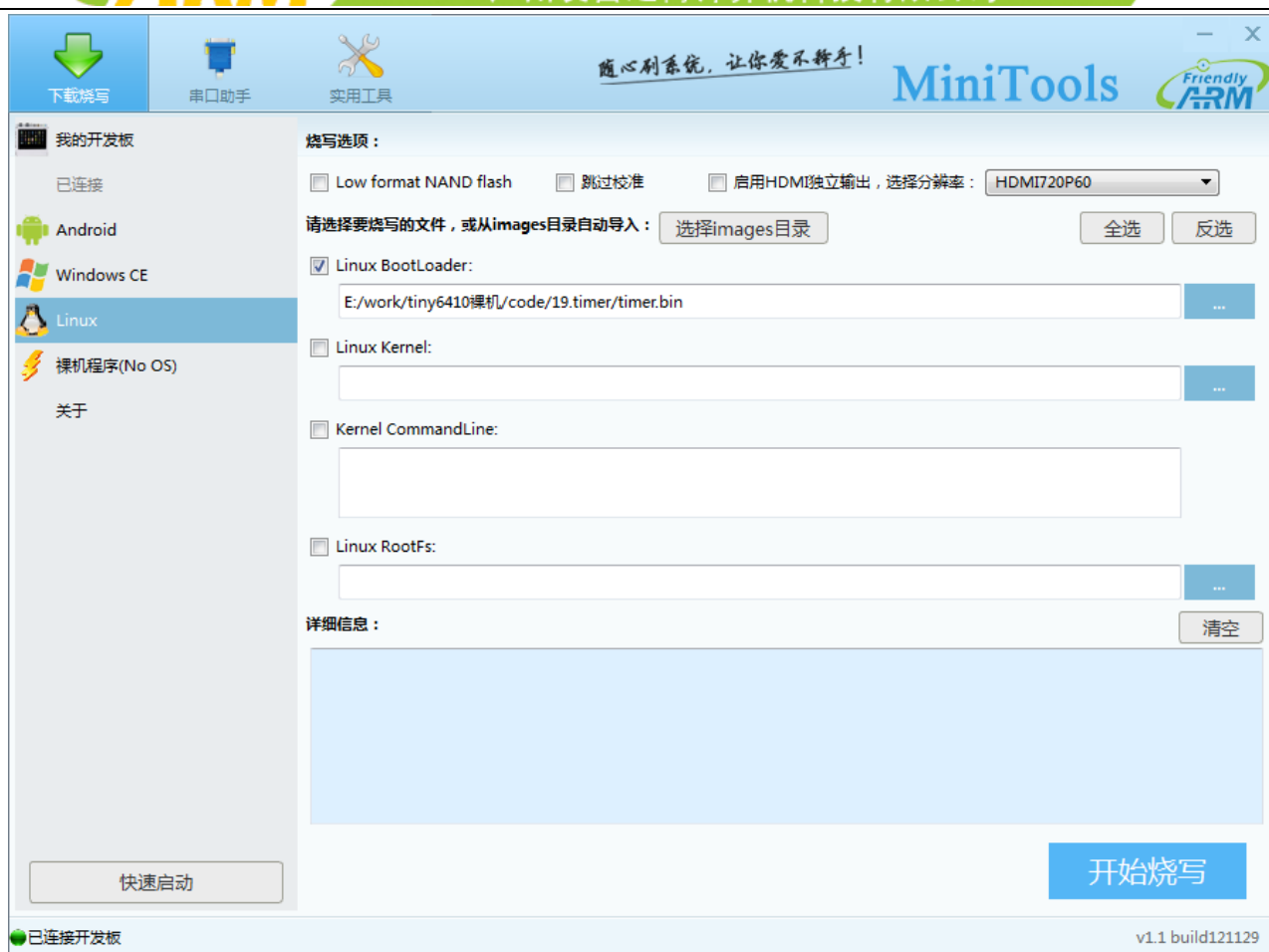
```
# cd 19.timer
```

```
# make
```

执行 make 后会生成 timer.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址, 其设置方式如下:



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。Timer0 中断没发生时，main() 会不停地打印数字，当 Timer 中断发生时，就会调用中断处理函数打印“Timer0 interrupt occur”效果如下：

```
#####timer test#####
0
1
2
3
1 Timer0 interrupt occur
4
5
6
2 Timer0 interrupt occur
7
8
```

第十七章 看门狗定时和复位

第一节 S3C6410 的看门狗定时器

S3C6410 上的看门狗定时器相当于一个普通的 16bit 的定时器，它与 PWM 定时器的区别是看门狗定时器可以产生 RESET 信号而 PWM 定时器不能，S3C6410 看门狗定时器的结构图如下：

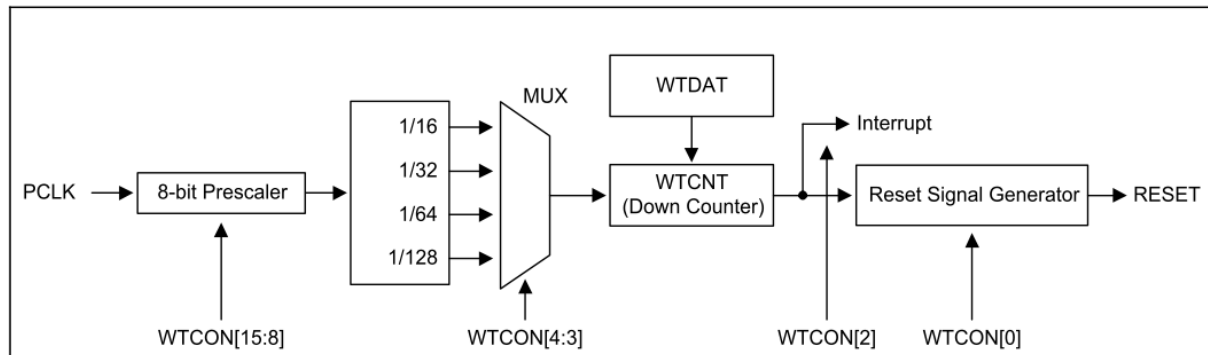


Figure 34-1. Watchdog Timer Block Diagram

第二节 程序相关讲解

完整代码见目录 20. watchdog。

1. main.c

需要注意两点：

- 1) 调用 wtd_operate() 初始化看门狗；
- 2) 死循环，不断地打印数字；

2. watchdog.c

1> 中断初始化函数 irq_init()

在中断控制器里使能看门狗中断，通过往 VIC0INTENABLE 的 bit26 置 1 即可。

2> 看门狗初始化函数 wtd_operate()

wtd_operate() 的完整代码如下：

```
void wtd_operate(unsigned long uenreset, unsigned long uenint, unsigned long uselectclk,
unsigned long uenwtd, unsigned long uprescaler, unsigned long uwtdat, unsigned long
uwtcnt)
{
    WTDAT = uwtdat;
    WTCNT = uwtcnt;
```

```
WTCN= (uenreset<<0) | (uenint<<2) | (uselectclk<<3) | (uenwtd<<5) | ((uprescaler)<<8);
}
```

首先设置计数相关的寄存器 WTDAT 和 WTCNT，寄存器 WTDAT 用来决定看门狗定时器的超时周期，在看门狗定时器启动后，寄存器 WTDAT 的值会自动传入寄存器 WTCNT，当 WTCNT 计数达到 0 时：如果中断被使能的话会发出中断，如果 RESET 功能被使用的话会发出复位信号，然后装载 WTDAT 的值并重新计数。

WTDAT	Bit	Description	Initial State
Count reload value	[15:0]	Watchdog timer count value for reload.	0x8000

WTCNT	Bit	Description	Initial State
Count value	[15:0]	The current count value of the watchdog timer	0x8000

寄存器 WTCN 进行相关配置，用来决定是否使能 RESET、是否使能中断、分频、是否启动定时器等，具体见下图：

WTCN	Bit	Description	Initial State
Prescaler value	[15:8]	Prescaler value. The valid range is from 0 to (2^8-1) .	0x80
Reserved	[7:6]	Reserved. These two bits must be 00 in normal operation.	00
Watchdog timer	[5]	Enable or disable bit of Watchdog timer. 0 = Disable 1 = Enable	1
Clock select	[4:3]	Determine the clock division factor. 00: 16 01 : 32 10: 64 11 : 128	00
Interrupt generation	[2]	Enable or disable bit of the interrupt. 0 = Disable 1 = Enable	0
Reserved	[1]	Reserved. This bit must be 0 in normal operation.	0
Reset enable/disable	[0]	Enable or disable bit of Watchdog timer output for reset signal. 1: Assert reset signal of the S3C6410 at watchdog time-out 0: Disable the reset function of the watchdog timer.	1

3> 中断处理函数 do_irq()

共 2 个步骤：

- 1) 打印信息；
- 2) 清中断状态寄存器 WTCLRINT；
- 3) 当发生了 5 次定时中断后，使能看门狗的 RESET 功能，此时系统会重启。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

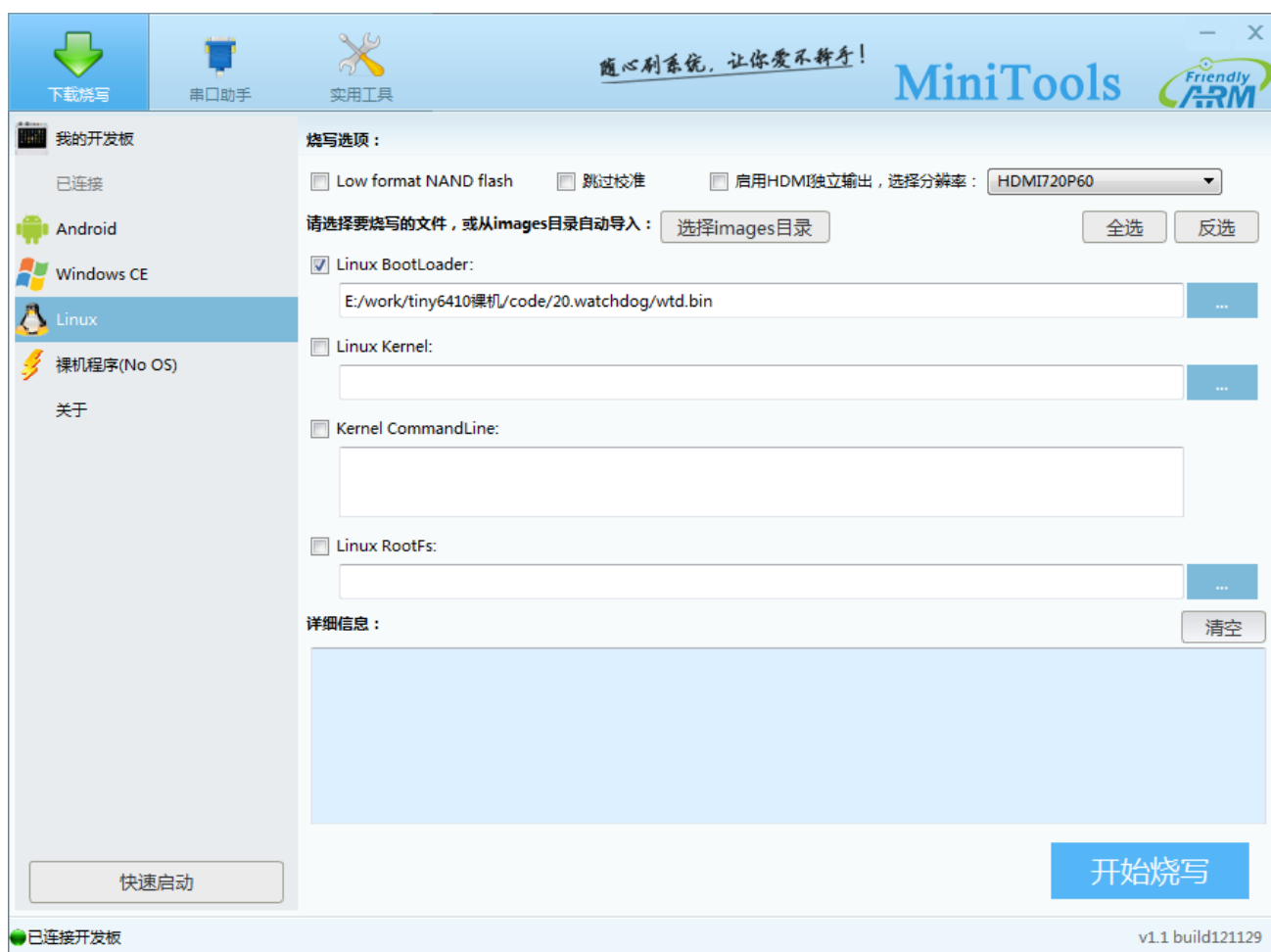
```
# cd 20.watchdog
```

```
# make
```

执行 make 后会生成 wtd.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。首先会不断地打印 1、2、3、4...，当发生看门狗定时中断时，则打印 “Watchdog interrupt occur”，当发生了 5 次看门狗定时中断后，看门



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

狗复位功能会被使能，接着系统就会重启，效果如下图：

```
#####watchdog test#####
0
1
2
watchdog interrupt occur 1
3
4
5
watchdog interrupt occur 2
6
7
8
watchdog interrupt occur 3
9
10
11
watchdog interrupt occur 4
12
13
14
watchdog interrupt occur 5
waiting system reset
15
16
17

#####watchdog test#####
```

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第十八章 RTC 读写时间

第一节 S3C6410 的 RTC

RTC 就是实时时钟芯片，用来在系统断电时，利用备用的锂电池继续记录时间。S3C6410 的 RTC 主要有如下特点：

- 1) 二进制编码数据：秒，分钟，小时，日期，日，月和年；
- 2) 闰年发生器；
- 3) 报警功能：报警中断或从断电模式中唤醒；
- 4) 时钟计数功能：时钟节拍中断或从断电模式中唤醒；
- 5) 不存在千年虫问题；
- 6) 独立地电源引脚（RTCVDD）；
- 7) 支持毫秒标记的时间中断信号，用于 RTOS 内核时间标记；

下面的 S3C6410 的 RTC 相关结构图：

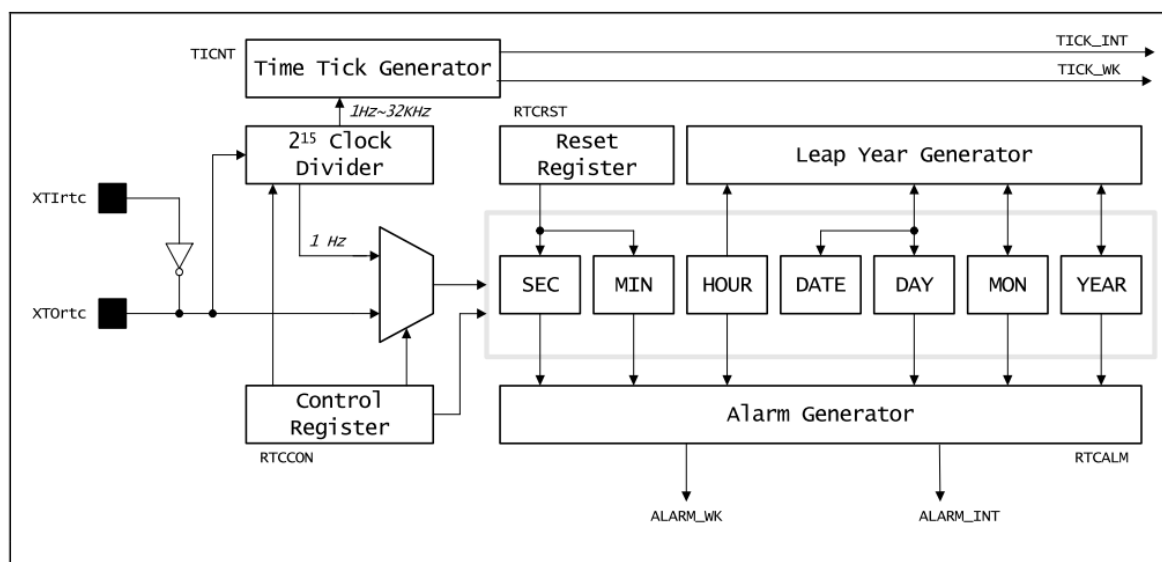


Figure 33-1. Real Time Clock Block Diagram

第二节 程序相关讲解

完整代码见目录 17.rtc。

1. main.c

main() 函数中打印了一个菜单，共两个选择，一是打印时间，调用了函数 rtc_realtime_display()；二是重置时间，调用了函数 rtc_settime()，这两个函数的定义均位于 rtc.c；

2. watchdog.c

1> 打印时间函数 rtc_realtime_display()

代码如下:

```
void rtc_realtime_display(void)
{
    int counter = 0;
    rtc_enable(true);                // 使能 rtc 控制器
    rtc_ticktime_enable(true);       // 使能 rtc tick timer

    while( (counter++) < 5)          // 打印 5 次时间
    {
        rtc_print();
        delay(0x1000000/5);
    }

    rtc_ticktime_enable(false);      // 关闭 rtc 控制器
    rtc_enable(false);              // 关闭 rtc tick timer
}
```

共 3 个步骤:

1) 使能 rtc 控制器, 调用了函数 `rtc_enable()`, 使能 rtc tick timer, 调用了 `rtc_ticktime_enable()`。两个函数的实质都是设置寄存器 `RTCCON`, 见下图:

RTCCON	Bit	Description	Initial State
TICEN	[8]	Tick timer enable 0 = Disable 1 = Enable	0
TICCKSEL	[7:4]	Tick timer sub clock selection. 4'b0000 = 32768 hz 4'b0001 = 16384 hz 4'b0010 = 8192 hz 4'b0011 = 4096 hz 4'b0100 = 2048 hz 4'b0101 = 1024 hz 4'b0110 = 512 hz 4'b0111 = 256 hz 4'b1000 = 128 hz 4'b1001 = 64 hz 4'b1010 = 32 hz 4'b1011 = 16 hz 4'b1100 = 8 hz 4'b1101 = 4 hz 4'b1110 = 2 hz 4'b1111 = 1 hz	4'b0000
CLKRST	[3]	RTC clock count reset. 0 = RTC counter(2^{15} Clock divider) enable, 1 = RTC counter reset and disable. Note: When RTCEN is enable, CLKRST affects RTC.	0
CNTSEL	[2]	BCD count select. 0 = Merge BCD counters 1 = Reserved (Separate BCD counters) Note: When RTCEN is enable, CNTSEL affects RTC.	0
CLKSEL	[1]	BCD clock select. 0 = XTAL $1/2^{15}$ divided clock 1 = Reserved (XTAL clock only for test) Note: When RTCEN is enable, CLKSEL affects RTC.	0
RTCEN	[0]	RTC control enable. 0 = Disable 1 = Enable Note: When RTCEN is enable, BCD time count setting, 2^{15} Clock divider reset, BCD counter select and BCD clock selecting.	0

2) 打印 5 次时间，调用了函数 `rtc_print()`，其作用就是读寄存器 BCDYEAR、BCDMON、BCDDATE、BCDHOUR、BCDMIN、BCDSEC、BCDDAY，分别获取了年、月、日、时、分、秒、星期几；

3) 关闭 rtc 控制器和 rtc tick timer，同样都是调用了函数 `rtc_enable()` 和 `rtc_ticktime_enable()`，不过这次是关闭功能。

2> 设置时间函数 `rtc_settime()`

共 3 个步骤：

- 1) 给 year、month、date 等时间变量赋初始值；
- 2) 将时间转化为 BCD 码；
- 3) 保存 BCD 码到寄存器 BCDYEAR、BCDMON、BCDDATE 等；

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

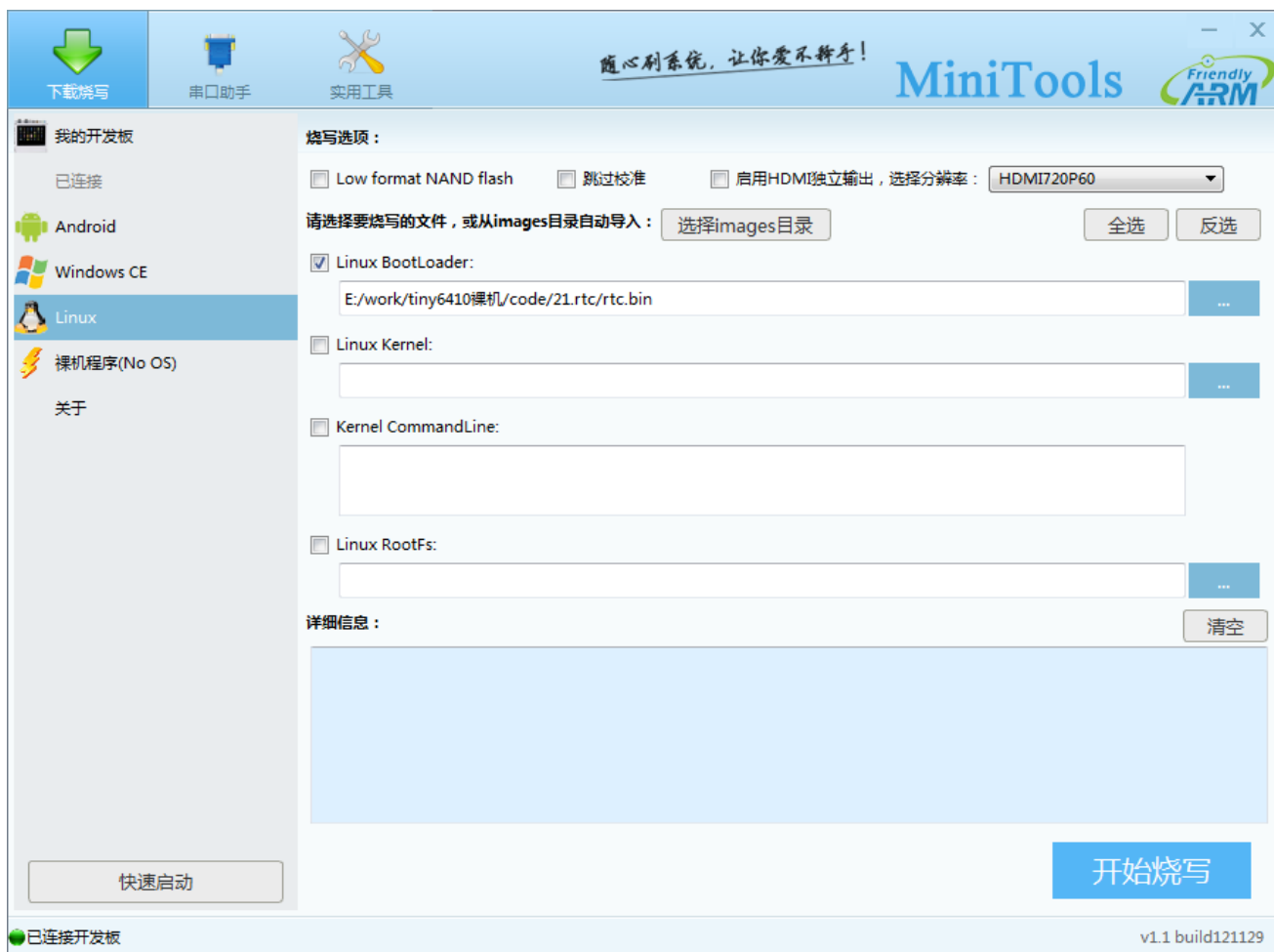
```
# cd 21.rtc
```

```
# make
```

执行 make 后会生成 `rtc.bin` 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。首先打印功能菜单，输入 s 复位时间，输入 d 显示当前时间，效果如下：



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

```
#####rtc test#####
[d] Display rtc realtime(hour:min:sec:weekday date/month/year)
[s] Reset rtc realtime(12:0:0:Tuesday 1/1/2012)
Enter your choice:s
reset success

#####rtc test#####
[d] Display rtc realtime(hour:min:sec:weekday date/month/year)
[s] Reset rtc realtime(12:0:0:Tuesday 1/1/2012)
Enter your choice:d
12 : 0 : 1    Tuesday,    5/ 1/2012
12 : 0 : 2    Tuesday,    5/ 1/2012
12 : 0 : 3    Tuesday,    5/ 1/2012
12 : 0 : 4    Tuesday,    5/ 1/2012
12 : 0 : 5    Tuesday,    5/ 1/2012
```

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第十九章 LCD 绘图和打印字符

第一节 S3C6410 LCD 控制器

要使一块 LCD 正常显示文字或图像，不仅需要 LCD 驱动器，还需要相应的 LCD 控制器。LCD 控制器的主要作用是将定位在系统存储器中的显示缓冲区中的 LCD 图像数据传送到外部 LCD 驱动器，并产生必要的控制信号，例如 VSYNC、HSYNC、VCLK。S3C6410 内部了 LCD 控制器，它结构图如下：

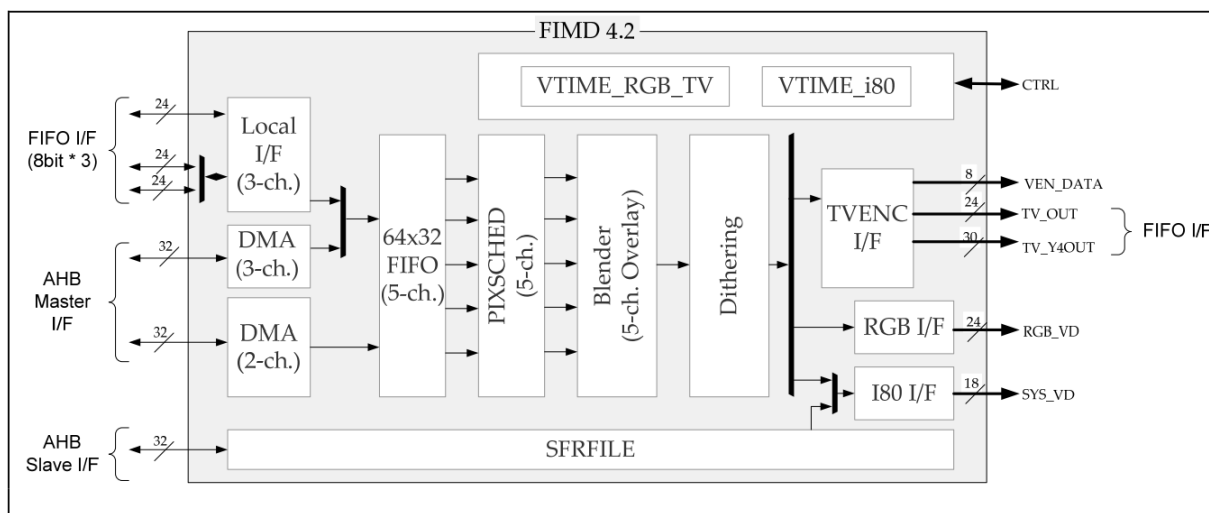


Figure 14-1. Top block diagram DISPLAY controller

LCD 控制器由 VSFR, VDMA, VPRCS, VTIME 和视频时钟产生器组成。VSFR 包括可编程寄存器设置和 2 个 256x25 的调色板存储器，VSFR 用于配置显示控制器。VDMA 专用于显示 DMA，VDMA 可以将帧存储器内的视频数据转换到 VPRCS。通过使用特殊的 DMA - VDMA，可以不使用 CPU，直接将视频数据显示在屏幕上。VPRCS 接收 VDMA 发出的视频数据，将其转换到适合的数据格式，如 8 位像素或 16 位像素后，将视频数据发送到显示设备上。VTIME 包括可编程逻辑，以支持在不同 LCD 驱动上发现的接口时序和速率的各种环境。VTIME 模块产生 RGB_VSYNC, RGB_HSYNC, RGB_VCLK, RGB_VDEN, SYS_CS, SYS_CS0 等等。

注：LCD 示例程序可同时支持 P43 和 H43 两款 LCD。

第二节 程序相关讲解

完整代码见目录 22. lcd 和 23. lcd_putchar，我们先讲解 22. lcd。

1. main.c

需要注意下面两点：

- 1) LCD 初始化；
- 2) 打印功能菜单，共 5 个选择，分别是清屏、画十字、画横向、画竖线、画圆。

2. lcd.c

LCD 初始化函数 lcd_init()

在 S3C6410 中, 给出了 LCD 的初始化序列, 见下图:

14.5 PROGRAMMER'S MODEL

14.5.1 OVERVIEW

The following registers are used to configure DISPLAY controller:

1. MOFPCON: SEL_BYPASS[3] value @ 0x7410800C must be set as '0'(normal mode) instead of '1'(by-pass mode).
2. SPCON: LCD_SEL[1:0] value @ 0x7F0081A0 must be set as '00' to use Host I/F Style or as '01' to use RGB I/F Style
3. VIDCON0: configure Video output format and display enable/disable.
4. VIDCON1: RGB I/F control signal.
5. I80IFCONx: i80-system I/F control signal.
6. ITUIFCON0 : ITU (BT.601) Interface Control
7. VIDTCNx: configure Video output Timing and determine the size of display.
8. WINCONx: each window format setting
9. VIDOSDxA, VIDOSDxB: Window position setting
10. VIDOSDxC: alpha value setting
11. VIDWxxADDx: source image address setting
12. WxKEYCONx: Color key value register
13. WINxMAP: window color control
14. WPALCON: Palette controls register
15. WxPDATAx: Window Palette Data of the each Index.

参考上述序列, 在 lcd.c 里函数 lcd_init() 对 lcd 控制器进行了初始化, 共 8 个步骤:

1) 配置相关引脚用于 LCD 功能, 相关代码如下:

```
GPECON = 0x00011111;  
GPEDAT = 0x00000001;  
GPFCON = 0x96AAAAAA;  
GPFDAT = 0x00002000;  
GPICON = 0xAAAAAAAA;  
GPJCON = 0x00AAAAAA;
```

查看 Tiny6410 的原理图可知, 涉及到的 GPIO 引脚为 GPE、GPF、GPI、GPJ。

2) 配置 MIFPCON 寄存器, 选择 normal mode;

3) 配置 SPCON 寄存器, 选择 RGB I/F;

4) 配置 VIDCONx, 设置接口类型、时钟、极性和使能 LCD 控制器等;
寄存器 VIDCON0:



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

VIDCON0	Bit	Description	Initial State
Reserved	[31:30]	Reserved	0
INTERLACE_F	[29]	Interlace or Progressive 0 : Progressive 1 : Interlace	0
Reserved	[28]	Reserved (Must be zero)	0
VIDOUT	[27:26]	It determines the output format of Display Controller 00: RGB I/F 01: TV Interface (Encoder or ITU601) 10: I80 I/F for LDI0 11: I80 I/F for LDI1	00
L1_DATA16	[25:23]	Select the mode of output data format of I80 I/F (LDI1.) (Only when, VIDOUT[1:0] == 2'b11) 000 = 16-bit mode (16 BPP) 001 = 16 + 2-bit mode (18 BPP) 010 = 9 + 9-bit mode (18 BPP) 011 = 16 + 8-bit mode (24 BPP) 100 = 18-bit mode (18BPP) 101 = 8 + 8-bit mode (16BPP)	000

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

VIDCON0	Bit	Description	Initial State
L0_DATA16	[22:20]	Select the mode of output data format of I80 CPU I/F (LDI0.) (Only when, VIDOUT[1:0] == 2'b10) 000 = 16-bit mode (16 BPP) 001 = 16 + 2-bit mode (18 BPP) 010 = 9 + 9-bit mode (18 BPP) 011 = 16 + 8-bit mode (24 BPP) 100 = 18-bit mode (18BPP) 101 = 8 + 8-bit mode (16BPP)	000
Reserved	[19]	Reserved	0
PNRMODE	[18:17]	Select the display mode. (Where, VIDOUT[1:0] == 2'b00) 00 = RGB Parallel format (RGB) 01 = RGB Parallel format (BGR) 10 = Serial Format (R->G->B) 11 = Serial Format (B->G->R) Select the display mode. (Where, VIDOUT[1:0] != 2'b00) 00 = RGB Parallel format (RGB)	00
CLKVALUP	[16]	Select CLKVAL_F update timing control 0 = always 1 = start of a frame (only once per frame)	0
Reserved	[15:14]	Reserved	
CLKVAL_F	[13:6]	Determine the rates of VCLK and CLKVAL[7:0] $VCLK = \text{Video Clock Source} / (\text{CLKVAL} + 1)$ where $\text{CLKVAL} \geq 1$ Note. 1. The maximum frequency of VCLK is 66MHz. 2. Video Clock Source is selected by CLKSEL_F register	0
VCLKFREE	[5]	VCLK Free run control (Only valid at the RGB IF mode) 0 = Normal mode (control by ENVID) 1 = Free-run mode	0
CLKDIR	[4]	Select the clock source as direct or divide using CLKVAL_F register 0 = Direct clock (frequency of VCLK = frequency of Clock source) 1 = Divided by CLKVAL_F	0x0
CLKSEL_F	[3:2]	Select the Video Clock source 00 = HCLK 01 = LCD video Clock (from SYSCON) 10 = reserved 11 = 27MHz Ext Clock input	0
ENVID	[1]	Video output and the logic immediately enable/disable. 0 = Disable the video output and the Display control signal. 1 = Enable the video output and the Display control signal.	0
ENVID_F	[0]	Video output and the logic enable/disable at current frame end. 0 = Disable the video output and the Display control signal. 1 = Enable the video output and the Display control signal. * If set on and off this bit, then you will read "H" and video controller enable until the end of current frame.	0

ENVID_F = 1, 当前帧结束后使能 LCD 控制器;

ENVID = 1, 使能 LCD 控制器;

CLKSEL_F = 0, 选择时钟源为 HCLK;

CLKDIR = 1, 选择需要分频;

VCLKFREE = 0, 选择 normal mode;

CLKVAL_F = 14, 分频系数为 9, 即 $VCLK = 133 / (9+1) = 13\text{MHz}$;

CLKVALUP = 0, 总是选择 CLKVAL_F 来更新时序控制;

RGSPSEL = 0, RGB 并行;
VIDOUT=0, 使用 RGB 接口;
未设置的 bit 均使用默认值。

5) 配置 VIDTCOnx, 设置时序和长宽等;

在讲解如何设置寄存器之前, 需要先讲讲 LCD 的时序设置, 见下图:

1.3.11.2 LCD RGB Interface Timing

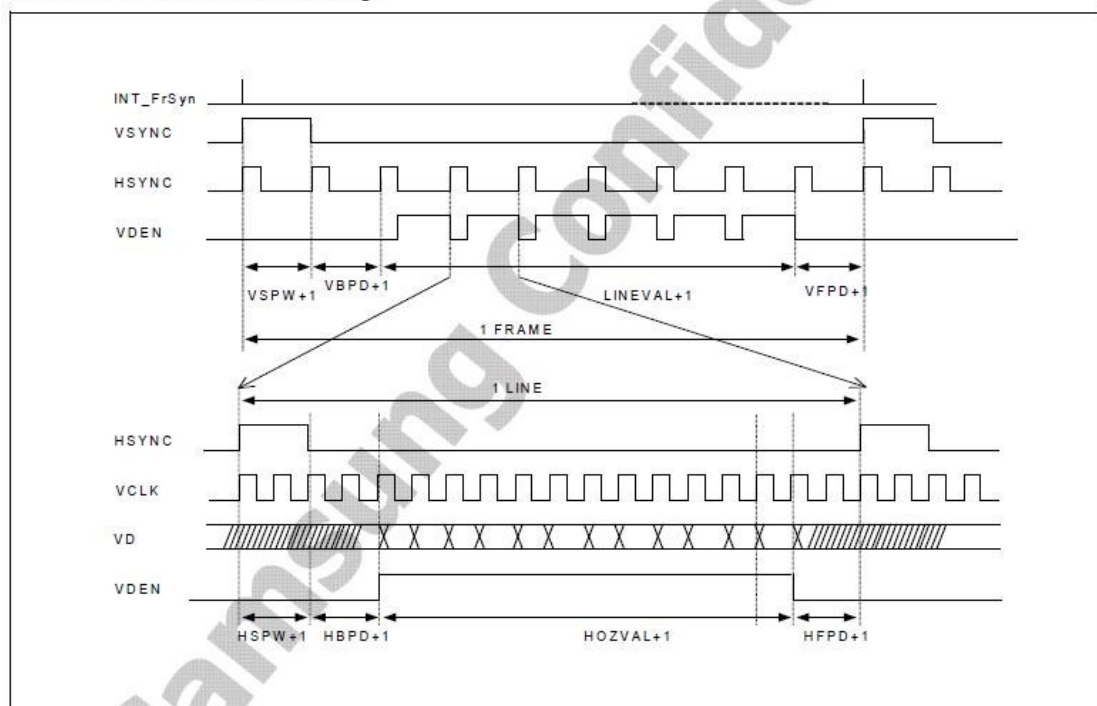


Figure 4-20 LCD RGB Interface Timing

其中各个时序的含义如下图:

- VBPD(vertical back porch): 表示在一帧图像开始时, 垂直同步信号以后的无效的行数
- VFPD(vertical front porch): 表示在一帧图像结束后, 垂直同步信号以前的无效的行数
- VSPW(vertical sync pulse width): 表示垂直同步脉冲的宽度, 用行数计算
- HBPD(horizontal back porch): 表示从水平同步信号开始到一行的有效数据开始之间的 VCLK 的个数
- HFPD(horizontal front porch): 表示一行的有效数据结束到下一个水平同步信号开始之间的 VCLK 的个数
- HSPW(horizontal sync pulse width): 表示水平同步信号的宽度, 用 VCLK 计算

每一帧的传输过程如下:

- a) VSYNC 信号有效时, 信号宽度为 (VSPW+1) 个 HSNc 信号周期, 即 (VSPW+1) 个无效行;
- b) VSYNC 信号脉冲之后, 总共还要进过 (VBPD+1) 个 HSYNC 信号周期, 有效的行数据才出现。所



以, 在 VSYNC 信号有效之后, 还要进过 (VSPW+1+VBPD+1) 个无效的行;

c) 随即发出 (LINEVAL+1) 行的有效数据;

d) 最后是 (VFPD+1) 个无效的行;

每一行中像素的传输过程如下:

a) HSNC 信号有效时, 表示一行数据的开始, 信号宽度为 (HSPW+1) 个 VCLK 信号周期, 即 (HSPW+1) 个无效像素;

b) HSYNC 信号脉冲之后, 还要经过 (HBPD+1) 个 VCLK 信号周期, 有效的像素数据才出现;

c) 随后发出 (HOZVAL+1) 个像素的有效数据;

d) 最后是 (HFPD+1) 个无效的像素;

有了上面这些知识, 设置时序相关的寄存器 VIDTC0N0 和 VIDTC0N1 就没什么问题了, 相关代码如下:

```
#define HSPW                (0)
#define HBPD                (40 - 1)
#define HFPD                (5 - 1)
#define VSPW                (0)
#define VBPD                (8 - 1)
#define VFPD                (8 - 1)
#define LINEVAL             (271)
#define HOZVAL              (479)
// 设置时序
VIDTC0N0 = VBPD<<16 | VFPD<<8 | VSPW<<0;
VIDTC0N1 = HBPD<<16 | HFPD<<8 | HSPW<<0;
// 设置长宽
VIDTC0N2 = (LINEVAL << 11) | (HOZVAL << 0);
```

首先是寄存器 VIDTC0N0, 我们参考下面两张图:

14.6.4 VIDEO TIME CONTROL 0 REGISTER

Register	Address	R/W	Description	Reset Value
VIDTCON0	0x77100010	R/W	Video time control 0 register	0x0000_0000

VIDTCON0	Bit	Description	Initial State
VBPDE	[31:24]	Vertical back porch is the number of inactive lines at the start of a frame, after vertical synchronization period. (Only for the even field of YVU interface)	0x0
VBPD	[23:16]	Vertical back porch is the number of inactive lines at the start of a frame, after vertical synchronization period.	0x0
VFPD	[15:8]	Vertical front porch is the number of inactive lines at the end of a frame, before vertical synchronization period.	0x0
VSPW	[7:0]	Vertical sync pulse width determines the VSYNC pulse's high level width by counting the number of inactive lines.	0x0

寄存器 VIDTCON0

6.3 Data Input Format

Parallel 24-bit RGB Input Timing Table

Parameters	Symbol	Min.	Typ.	Max.	Unit	Conditions
DCLK frequency	fclk	5	9	12	MHz	
VSYNC period time	Tv	277	288	400	Th	
VSYNC display area	Tvd	272			Th	
VSYNC back porch	Tvbp	3	8	31	Th	
VSYNC front porch	Tvfp	2	8	93	Th	
HSYNC period time	Th	520	525	800	DCLK	
HSYNC display area	Thd	480			DCLK	
HSYNC back porch	Thbp	36	40	255	DCLK	
HSYNC front porch	Thfp	4	5	65	DCLK	

LCD 芯片手册时序图

- VFPD:** Vertical front porch, 单位是行。LCD 芯片手册时序图中 VSYNC front porch 就是 VFPD+1, 因为时序图中 Th 表示一行需多少个 VCLK, 所有 VFPD=8-1 行 (Th);
- VBPD:** Vertical back porch, 单位是行。LCD 芯片手册时序图中 VSYNC back porch 就是 VBPD+1, 因为时序图中 Th 表示一行需多少个 VCLK, 所有 VBPD=8-1 行 (Th);
- VSPW:** Vertical sync pulse width, 单位是行。LCD 芯片手册时序图中没直接给出它的值, 需要自己计算: $VSPW = VSYNC \text{ period time} - VSYNC \text{ display area} - VSYNC \text{ back porch} - VSYNC \text{ front porch}$ $= 288 - 272 - 8 - 8 = 0$, 所有 VSPW 设为 0 即可;

然后是寄存器 VIDTCON1, 我们参考下面两张图:

14.6.5 VIDEO TIME CONTROL 1 REGISTER

Register	Address	R/W	Description	Reset Value
VIDTCON1	0x77100014	R/W	Video time control 1 register	0x0000_0000

VIDTCON1	Bit	Description	Initial state
VFPDE	[31:24]	Vertical front porch is the number of inactive lines at the end of a frame, before vertical synchronization period. (Only for the even field of YVU interface)	0x0
HBPD	[23:16]	Horizontal back porch is the number of VCLK periods between the falling edge of HSYNC and the start of active data.	0x0
HFPD	[15:8]	Horizontal front porch is the number of VCLK periods between the end of active data and the rising edge of HSYNC.	0x0
HSPW	[7:0]	Horizontal sync pulse width determines the HSYNC pulse's high level width by counting the number of the VCLK.	0x0

VIDTCON1 寄存器

6.3 Data Input Format

Parallel 24-bit RGB Input Timing Table

Parameters	Symbol	Min.	Typ.	Max.	Unit	Conditions
DCLK frequency	fclk	5	9	12	MHz	
VSYNC period time	Tv	277	288	400	Th	
VSYNC display area	Tvd	272			Th	
VSYNC back porch	Tvbp	3	8	31	Th	
VSYNC front porch	Tvfp	2	8	93	Th	
HSYNC period time	Th	520	525	800	DCLK	
HSYNC display area	Thd	480			DCLK	
HSYNC back porch	Thbp	36	40	255	DCLK	
HSYNC front porch	Thfp	4	5	65	DCLK	

LCD 芯片手册时序图

- ✚ **HBPD:** Horizontal back porch, 单位是 VCLK。LCD 芯片手册时序图中 HSYNC back porch 就是 HBPD +1, 所有 HBPD=40-1(VCLK);
- ✚ **HFPD:** Horizontal front porch, 单位是 VCLK。LCD 芯片手册时序图中 HSYNC front porch 就是 HFPD +1, 所有 VBPD=5-1(VCLK);
- ✚ **HSPW:** Horizontal sync pulse width, 单位是 VCLK。LCD 芯片手册时序图中没直接给出它的值, 需要自己计算: $HSPW+1 = HSYNC \text{ display area} - HSYNC \text{ display area} - HSYNC \text{ back porch} - HSYNC \text{ front porch} = 525 - 480 - 40 - 5 = 0$, 所有 VSPW 设为 0 即可;

最后是寄存器 VIDTCON2, 见下图:

14.6.6 VIDEO TIME CONTROL 2 REGISTER

Register	Address	R/W	Description	Reset Value
VIDTCON2	0x77100018	R/W	Video time control 2 register	0x0000_0000

VIDTCON2	Bit	Description	Initial state
LINEVAL	[21:11]	These bits determine the vertical size of display	0
HOZVAL	[10:0]	These bits determine the horizontal size of display	0

✚ HOZVAL: $HOZVAL = (\text{Horizontal display size}) - 1 = 480 - 1 = 479$

✚ LINEVAL: $LINEVAL = (\text{Vertical display size}) - 1 = 272 - 1 = 271$

6) 配置 WINCON0, 设置 window0 的数据格式;

S3C6410 的 LCD 控制器有 overlay 功能, 支持 5 个 window。这里我们只使用 window0, 相关的代码设置如下:

```
WINCON0 |= 1 << 0;
WINCON0 &= ~(0xf << 2);
WINCON0 |= 0xB << 2;
```

我们不需要使用很强大的功能, 所有只需设置如下几个 bit 即可:

✚ ENWIN_F=1, 使能;
✚ BPPMODE_F= 1011, 24bpp;

7) 配置 VIDOSD0A/B/C, 设置 window0 的坐标系;

相关代码如下:

```
VIDOSD0A = (LeftTopX << 11) | (LeftTopY << 0);
VIDOSD0B = (RightBotX << 11) | (RightBotY << 0);
VIDOSD0C = (LINEVAL + 1) * (HOZVAL + 1);
```

设置 window0 的左上角和右下角的坐标和长宽。

8) 配置 VIDW00ADD0B0 和 VIDW00ADD1B0, 设置 framebuffer 的地址;

相关代码如下:

```
VIDW00ADD0B0 = FRAME_BUFFER;
VIDW00ADD1B0 = (((HOZVAL + 1) * 4 + 0) * (LINEVAL + 1)) & (0xffffffff);
```

到这里, 代码 22. lcd 就讲解完了, 我们现在来看 23. lcd_putchar:

1. lcd.c

相比 22. lcd, 多了一个函数 lcd_draw_char(), 其主要内容如下:

- 1) 获得字模。以传进来的参数为下标, 从字模数组 fontdata_8x8 里取出对应的字模, 每一个字模用 8*8bit 的数据来表示, 数组 fontdata_8x8 的定义位于 font_8x8.c 中;
- 2) 是否需要回车换行。当遇到 '\n' 时表示换行, 当遇到 '\r' 表示回车;
- 3) 在 16*16 个像素里描绘一个字符。font_8x8.c 里定义的每一个字模都是由 8*8bit 组成, 每



1bit 对应一个像素，为 1 则描白点，为 0 则描黑点。但是为了看得更清楚，我们的代码把字体放大了两倍，即用 16*16 个像素来显示一个字符，具体方法参考代码。

4) 光标移动到下一个 16*16 的位置；

2. main.c

main() 会打印一个菜单，这个菜单没有具体的功能，只是调用了 printf(), 在 printf() 里会调用 putc(), 我们在 putc() 最后面加了一行代码：lcd_draw_char(), 把打印到终端的信息也同时打印到 LCD 上。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

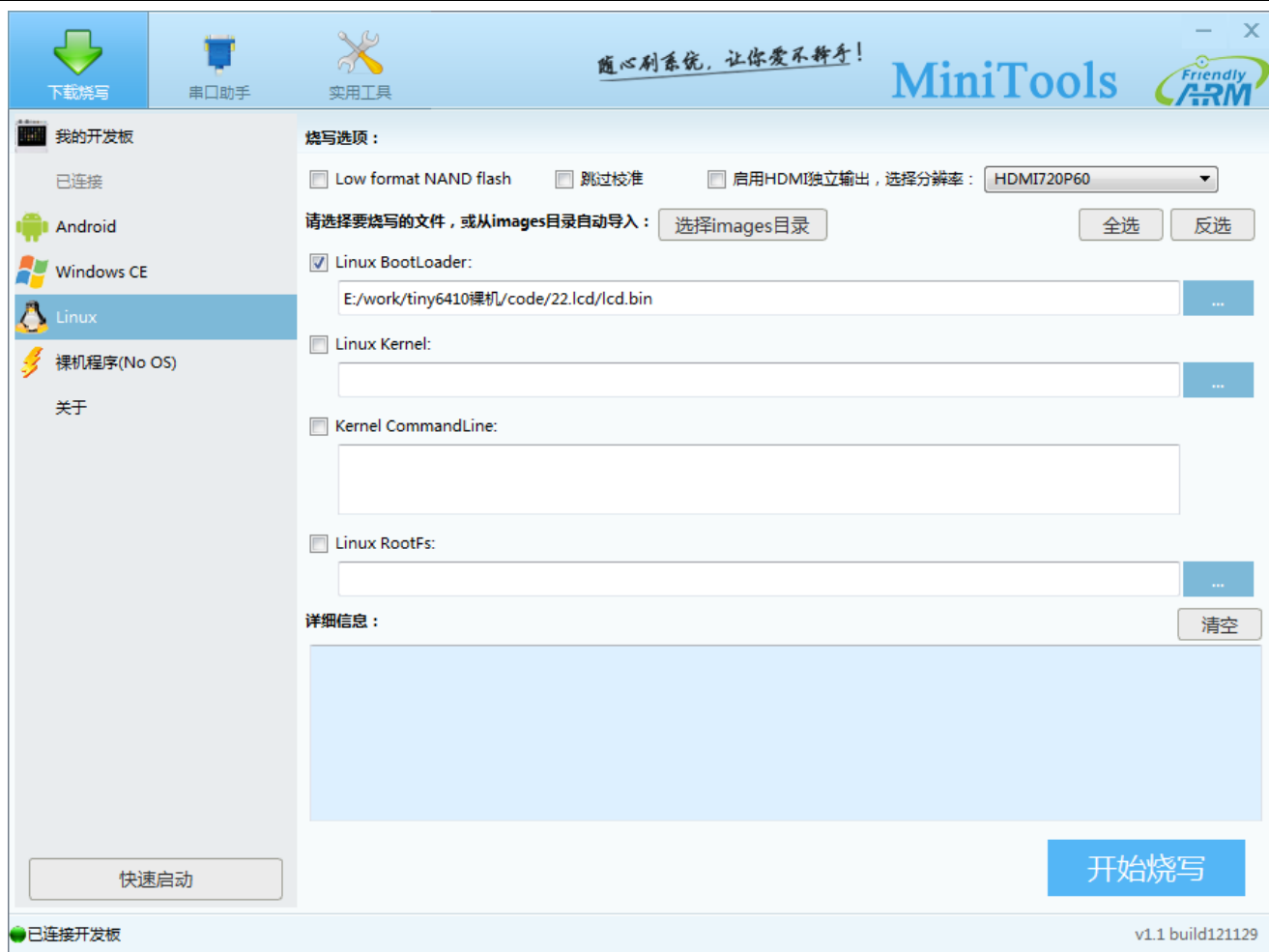
```
# cd 22.lcd
```

```
# make
```

执行 make 后会生成 lcd.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

连接好串口终端后，开发板选择 NAND 启动。对于代码 22.lcd，首先在 PC 终端会打印如下菜单：

```
#####lcd test#####
[1] lcd_clear_screen
[2] lcd_draw_cross
[3] lcd_draw_hline
[4] lcd_draw_vline
[5] lcd_draw_circle
Enter your choice:
```

输入 1：LCD 会清屏；

输入 2：LCD 会在左上角打印一个十字；

输入 3：LCD 会在正中间画一条横线；

输入 4：LCD 会在正中间画一条竖线；

输入 5：LCD 会出现同心圆；



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

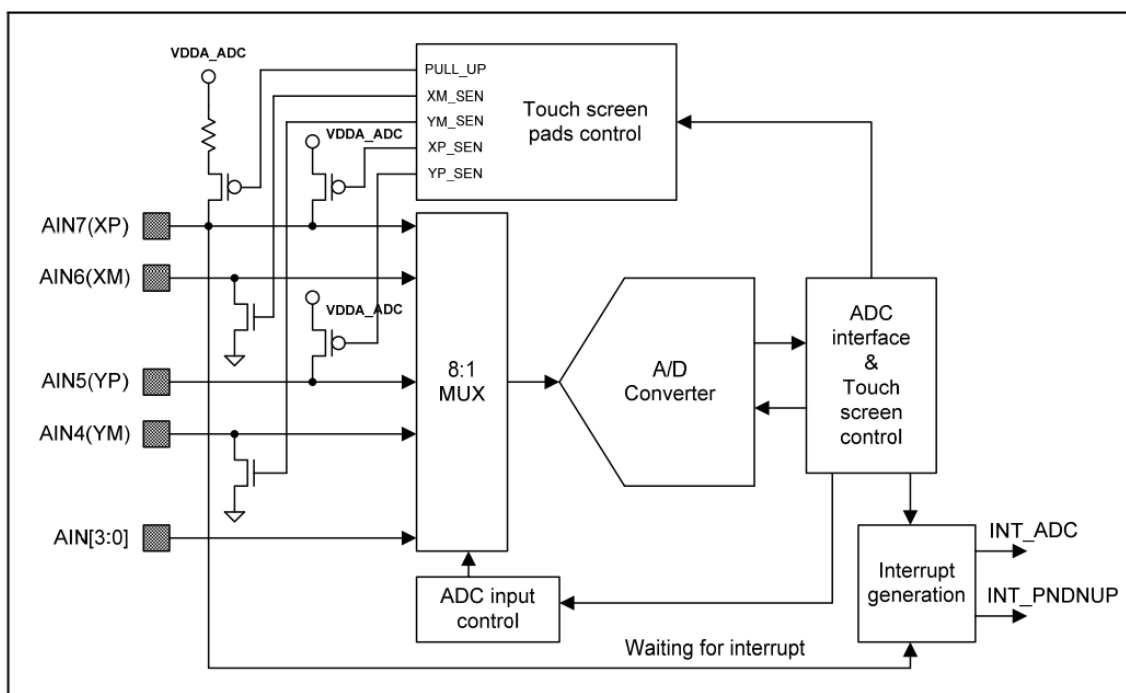
广州友善之臂计算机科技有限公司

对于代码 23. lcd_putchar，在 PC 终端和 LCD 屏幕中，都会打印出功能菜单：

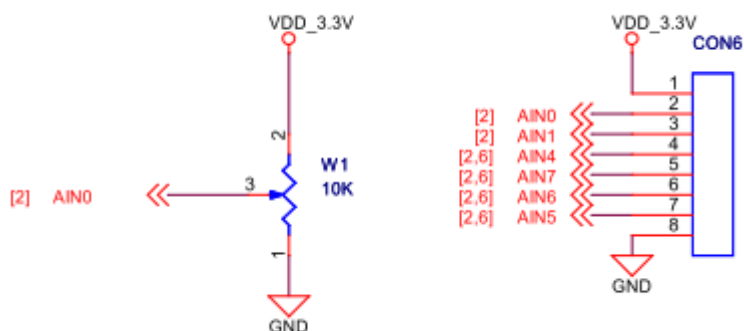
第二十章 测试 ADC 转换

第一节 S3C6410 的 ADC

S3C6410 的 ADC 可支持 10bit 和 12bit，它支持 8 路输入，然后将输入的模拟的信号转换为 10bit 或者 12bit 的二进制数字信号。在 5MHz 的时钟下，最大转换速率是 1MSPS。本章只是涉及到初步的 ADC 转换，并不会讲解触摸屏相关知识，所有我们暂时把触摸屏的相关知识屏蔽。其结构图如下：



在 Tiny6410 中，ADC 相关的原理图如下：



通道 0 的输入被接到可调电阻上，通过调节可变电阻，adc 能转换出不同的值。



第二节 程序相关讲解

完整代码见目录 24. adc。

1. main.c

main 函数很简单，就是在一个死循环里不断地读取 ADC 转换的值。

2. adc.c

核心函数是 read_adc(), 它主要包括 5 个步骤:

1) 设置时钟。相关代码如下:

```
ADCCON = (1<<16) | (1 << 14) | (65 << 6) | (ch << 3);
```

ADCCON 寄存器

ADCCON	Bit	Description	Initial State
RESSEL	[16]	A/D converter resolution selection 0 = 10-bit A/D conversion 1 = 12-bit A/D conversion	0
ECFLG	[15]	End of conversion flag(Read only) 0 = A/D conversion in process 1 = End of A/D conversion	0
PRSCEN	[14]	A/D converter prescaler enable 0 = Disable 1 = Enable	0
PRSCVL	[13:6]	A/D converter prescaler value Data value: 5 ~ 255 NOTE: Note that division factor is (N+1) when the prescaler value is N. ADC frequency should be set less than PCLK by 5 times. (Ex. If PCLK=10MHz, ADC Frequency<2MHz) This A/D converter is designed to operate at maximum 5MHz clock	0xFF
SEL_MUX	[5:3]	Analog input channel select 000 = AIN 0 001 = AIN 1 010 = AIN 2 011 = AIN 3 100 = YM 101 = YP 110 = XM 111 = XP	0
STDBM	[2]	Standby mode select 0 = Normal operation mode 1 = Standby mode	1
READ_START	[1]	A/D conversion start by read 0 = Disable start by read operation 1 = Enable start by read operation	0
ENABLE_START	[0]	A/D conversion starts by enable. If READ_START is enabled, this value is not valid. 0 = No operation 1 = A/D conversion starts and this bit is cleared after the start-up.	0

SEL_MUX = 0, 选择通道 0;

PRSCVL = 65, 时钟预分频系数为 65;

PRSCEN = 1, 使能预分频;

RESSEL = 1, 使用 12bit 的 ADC;

2) 设置模式, 代码如下:

```
ADCCON &= ~(1<<2) | (1<<1);
```

设为普通转换模式, 禁止 read start。

3) 启动转换。代码如下:



```
ADCCON |= (1 << 0);
```

4) 检查转换是否完成。代码如下：

```
while (ADCCON & (1 << 0));
```

5) 读数据，代码如下：

```
return (ADCDAT0 & 0xffff);
```

由于我们使用的 12bit 的模式，所有读寄存器 ADCDAT0 的前 12bit。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

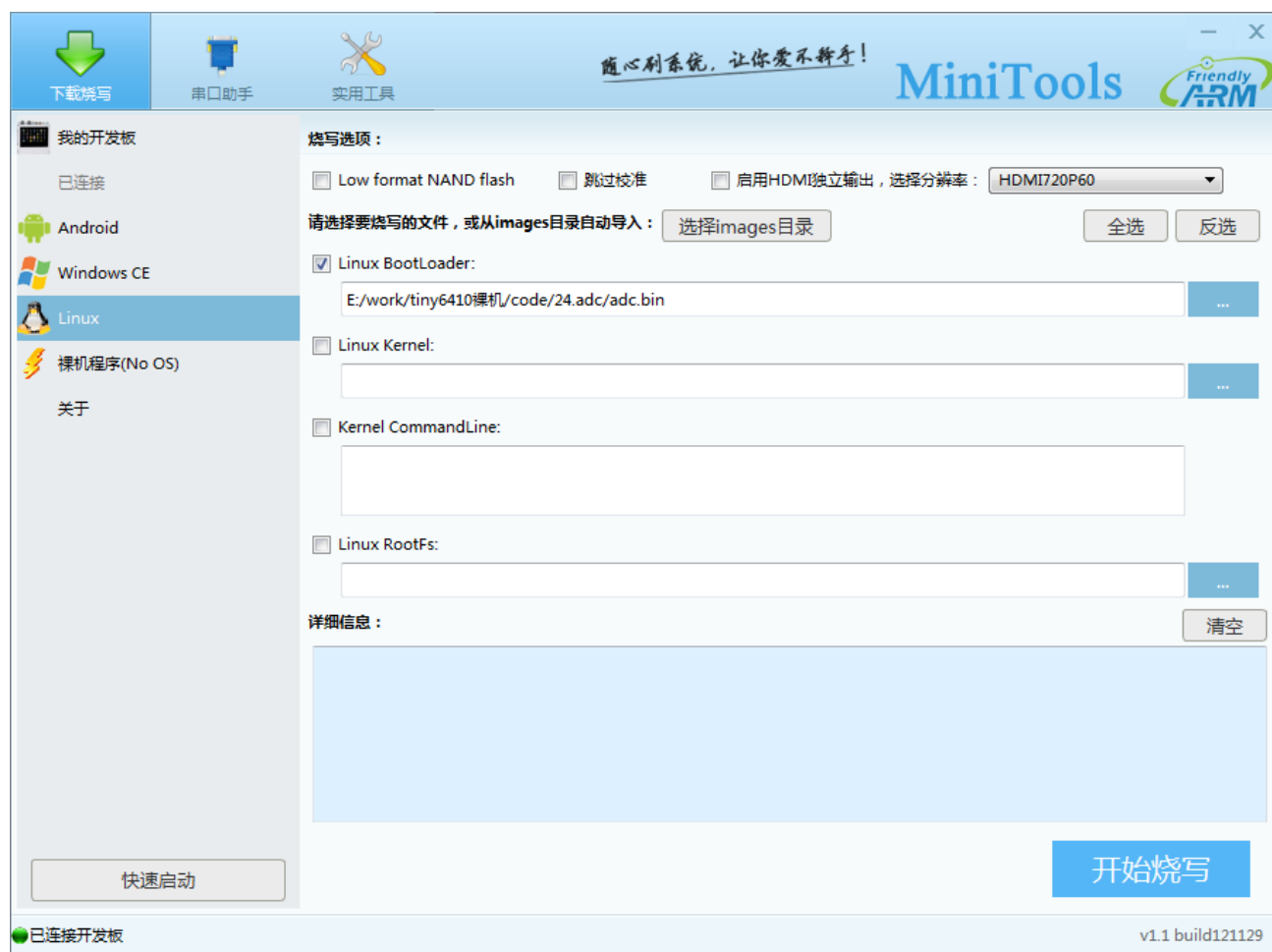
```
# cd 24.adc
```

```
# make
```

执行 make 后会生成 adc.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

PC 终端上会不断的打印出数字，数字的范围是 0~4096，这是因为我们使用的是 12bit 的 ADC。有了 ADC 转换的基础，也就为触摸屏的学习打下了基础。



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

```
#####adc test#####
```

```
adc = 0
adc = 0
adc = 36
adc = 126
adc = 585
adc = 911
adc = 1823
adc = 2013
adc = 2363
adc = 3372
adc = 3624
adc = 3664
adc = 4081
adc = 4082
adc = 4078
adc = 4084
adc = 4072
adc = 4084
adc = 4088
adc = 4081
adc = 4075
adc = 4082
adc = 4079
adc = 4081
adc = 4086
adc = 4075
adc = 4082
adc = 4073
adc = 4082
```

地址：广州市天河区龙口西路龙苑大厦A1栋1705

网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)

第二十一章 增加命令功能

第一节 关于命令功能

这里所说的命令功能类似 linux 中的 shell，输入一个命令，然后程序开始解析运行。这里我们只是象征性的实现几个简单的命令，包括：

help-提供帮助信息

md-memory display 显示内存

mw-memory write 写内存

loadb-通过串口下载 bin 文件到内存

go-运行内存中的 bin 文件

第二节 程序详细讲解

完整代码见目录 25. shell。

1. main.c

main() 函数核心代码如下：

```
while (1)
{
    printf("Tiny6410: ");
    gets(buf);                // 等待用户输入命令
    argc = shell_parse(buf, argv); // 解析命令
    command_do(argc, argv);    // 运行命令
}
```

注释已经一目了然了。函数 shell_parse() 的定义位于 shell.c，command_do() 的定义位于 command.c 中。

2. shell.c

shell_parse() 核心代码如下：

```
while (*buf)
{
    if (*buf != ' ' && state == 0) // 获得一个单词
    {
        argv[argc++] = buf;
        state = 1;
    }
}
```

```
if (*buf == ' ' && state == 1)    // 跳过空格
{
    *buf = '\0';
    state = 0;
}
buf++;
```

逐个解析 buf 里的字符，结果保存在 argc 和 argv 中。例如输入命令：go 0x51000000，最终解析的结果是 argc = 2, argv[0] = “go”, argv[1] = “0x51000000”。

3. command.c

首先来看 command_do()，其大致代码如下：

```
int command_do(int argc, char * argv[])
{
    if (argc == 0)
        return -1;
    if (strcmp(argv[0], "help") == 0)    // 执行 help 命令
        help(argc, argv);
    ...    // 省略代码
    if (strcmp(argv[0], "loadb") == 0)    // 执行 loadb 命令
        loadb(argc, argv);
    ...    // 省略代码
}
```

根据不同的命令，调用不同的执行函数。例如输入 help 命令，调用的执行函数是 help() 函数。下面来简单地解释各个命令的执行函数：

int help(int argc, char * argv[]): 打印帮助信息。

int md(int argc, char * argv[]): 读内存，读写内存都只是简单的指针操作

int mw(int argc, char * argv[]): 写内存

int loadb(int argc, char * argv[]): 从串口下载 bin 文件到内存，先获得文件大小，再获得下载地址，最后利用 getc() 函数一个字节一个字节地接受 bin 文件。

int go(int argc, char * argv[]): 执行内存中的 bin 文件，先定义一个函数指针，然后赋值调用。

第三节 编译代码和烧写运行

编译代码

在 Fedora 终端执行如下命令：

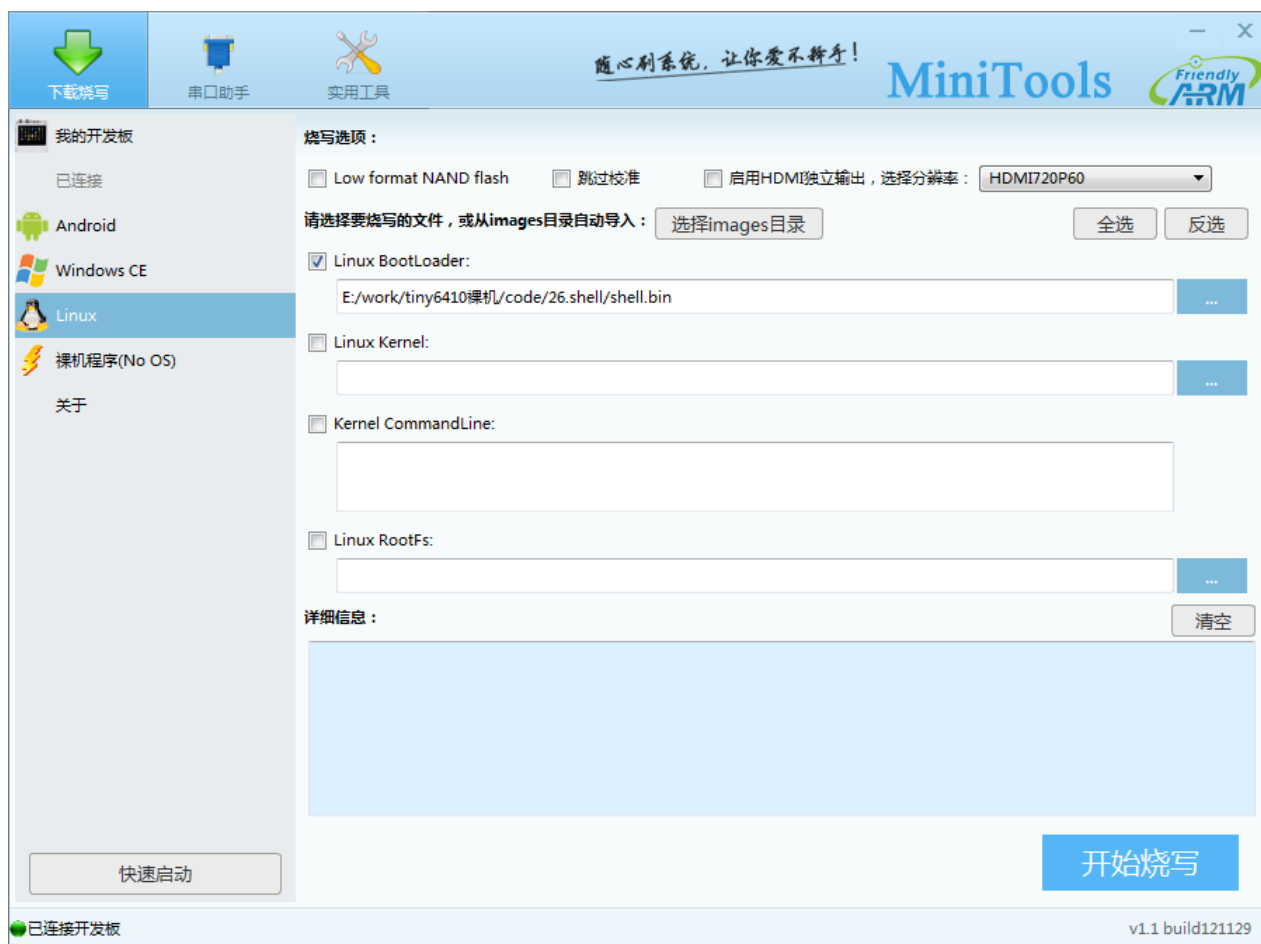
```
# cd 25.shell
```

```
# make
```

执行 make 后会生成 shell.bin 文件。

烧写运行

使用 MiniTools 将 bin 文件烧写到开发板上的 NAND Flash 的 0 地址，其设置方式如下：



点击开始烧写，MiniTools 就会把 bin 文件烧写到 NAND Flash 的 0 地址上了。

第四节 实验现象

PC 终端上会打印出 “Tiny6410”，然后输出 help 命令会打印帮助信息：



追求卓越 创造精品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

```
Tiny6410: help
help usage:
md - memory dispaly
mw - memory write
loadb - loadb filesize addr
go - go addr
Tiny6410: md
0: e3a00207 e3800013 ee0f0f92 e59f0068
10: e3a01000 e5801000 e59fd060 e3800a01
20: ee010f10 eb000024 eb00004f eb0000b1
30: e24f0038 e59f1048 e59f2048 e0422001
40: e1500001 a000002 eb0000b8 e3500000
50: 1a000008 e59f002c e59f102c e3a03000
60: e1500001 a000002 e4803004 e1500001
70: 1affffffc e59ff014 eaffffffe 7e004000
80: c002000 50000000 50002594 50002d98
90: 50000494 2541 61656100 1006962
a0: 1b 4d524105 36373131 2d465a4a
b0: 9060053 1090108 20a e59f3048
c0: e59f2048 e5832000 e5832004 e5832008
d0: e5932900 e3c220c0 e5832900 e5931900
e0: e59f2024 e3110c0f 1affffffb e3a03c33
f0: e5823020 e59f3018 e582300c e5823010
Tiny6410: 
```

地址：广州市天河区龙口西路龙苑大厦A1栋1705

网址：<http://www.arm9.net>

电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505

E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)



追 求 卓 越 创 造 精 品

TO BE BEST

TO DO GREAT

广州友善之臂计算机科技有限公司

地址：广州市天河区龙口西路龙苑大厦A1栋1705 网址：<http://www.arm9.net>
电话：+86-20-85201025(售前、售后咨询) 技术支持(Tel): 13719442657 传真：+86-20-85261505
E-Mail: capbily@163.com(商务或项目合作) dev_friendlyarm@163.com (技术支持)